الجـــمهوريـــة الجزائريـــة الديمـــقراطيـــــة الشـعبيـة
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵜⴰⵣⵣⴰⵢⵔⵉⵜ ⵜⴰⵎⴰⵙⵙⴰⵡⵉⵜ ⵜⴰⵖⵔⴰⵎⵉⵜ
République Algérienne Démocratique et Populaire
وزارة التعليـــم العـــالي والبـــحث العلمـــي
ⵓⵢⵏⵉⵖ ⵏ ⵜⵓⵎⵔⵉⵜ ⵜⵉⵏⵉⵖ ⴷ ⵜⵓⴷⵉⵜ ⵜⵓⵎⵓⵏⵉⵏ
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

ECOLE NATIONALE SUPÉRIEURE D'INFORMATIQUE

المدرسة الوطنية العليا للإعلام الآلي
ⵓⵢⵓⵎⵓⵣ ⵓⵢⵓⵏⵓⵍ ⵜⵉⵏⵉⵖ ⵏ ⵜⵓⵏⵓⵎⵓⵔⵉⵜ
École nationale Supérieure d'Informatique

Final Dissertation

For obtaining the State Engineer Diploma in Computer Science

Option : Intelligent Systems and Data (SD)
Option : Computer Systems (SIQ)

Theme :

Redesigning Refactal – A Serious Game for Learning Advanced Object-Oriented Programming Principles

---

Completed by :

– Abdellah Arrouche
– Ahmed Abdelhadi Hambli

Supervised by :

– Dr. Nabila Bousbia

Host organization : Ecole Nationale Supérieure d'Informatique (ESI)

Cohort : 2025 / 2026

To our beloved families, whose endless support, patience, and love made this achievement possible.

To our teachers, supervisors, and friends who guided us throughout our studies and PFE development.

Acknowledgements

First and foremost, we express our deep gratitude to Almighty Allah for granting us the health, strength, and perseverance to successfully complete this graduation project.

We would like to express our deepest gratitude to our supervisor, **Dr. Nabila Bousbia**, for her valuable guidance, constant encouragement, and insightful feedback. Her expertise and dedication have been crucial to the completion of this final year project.

We are highly appreciative to the members of the jury for their time and effort in evaluating this dissertation. We would also like to thank all the professors, administrative staff, and colleagues at the **École Nationale Supérieure d'Informatique (ESI)** for their invaluable support throughout our university studies.

Finally, we express our profound gratitude to our parents, siblings, and friends for their unconditional love, continuous moral support, and motivation during this challenging and rewarding journey.

Abstract

Serious games for programming education have grown substantially as a research area over the past decade, yet the vast majority of existing titles target introductory concepts (basic syntax, control structures, and elementary algorithmic reasoning). Purpose-built games for advanced object-oriented programming and software design patterns remain rare. This work addresses that gap through a complete architectural redesign of Refactal, an educational coding game developed at the École Nationale Supérieure d'Informatique (ESI) across multiple projects since 2022, whose prior versions suffered from severe early dropout and a limited data collection infrastructure.

The redesigned game, Refactal : Reimagined (RE :RE), is a 2D side-scrolling puzzle-platformer built in Godot 4.6 in which players interact with a malfunctioning space station by writing real, executable code. Three advanced OOP concepts are covered : the Single Responsibility Principle, the Builder design pattern, and the Memento design pattern. The core mechanic is enabled by a fully custom interpreter embedded within the game engine, dispatching method calls directly onto objects in the game world and producing immediate, visible consequences. Player behaviour is tracked automatically and asynchronously.

The game was evaluated with ESI computer science students across multiple year groups. Fifty-two students played the game and ten submitted completed responses. Evaluation through the MEEGA+ educational game evaluation framework showed Satisfaction (4.22/5.0) and Usability (3.95/5.0) as the top dimensions, with fun and intent to replay scoring 4.50/5.0 each.

These results suggest that Refactal : Reimagined successfully addresses the documented limitations of its predecessors and constitutes a meaningful contribution to the small body of serious games targeting advanced software engineering education.

Keywords : Serious Games, Object-Oriented Programming, Design Patterns, Player Tracking, Game-Based Learning.

Résumé

Les jeux sérieux destinés à l'enseignement de la programmation ont connu une croissance importante en tant que domaine de recherche au cours de la dernière décennie. Cependant, la grande majorité des titres existants se concentre sur des concepts introductifs (syntaxe de base, structures de contrôle et raisonnement algorithmique élémentaire). Les jeux conçus spécifiquement pour l'apprentissage de la programmation orientée objet avancée et des patrons de conception logicielle demeurent rares. Ce travail vise à combler cette lacune à travers une refonte architecturale complète de Refactal, un jeu éducatif de programmation développé à l'École Nationale Supérieure d'Informatique (ESI) au fil de plusieurs projets depuis 2022, dont les versions précédentes souffraient d'un fort taux d'abandon précoce ainsi que d'une infrastructure limitée de collecte de données.

Le jeu repensé, Refactal : Reimagined (RE), est un jeu de plateforme et d'énigmes en défilement horizontal en 2D développé avec Godot 4.6, dans lequel les joueurs interagissent avec une station spatiale défaillante en écrivant du code réel et exécutable. Trois concepts avancés de la programmation orientée objet sont abordés : le principe de responsabilité unique (Single Responsibility Principle), le patron de conception Builder et le patron de conception Memento. La mécanique principale repose sur un interpréteur entièrement personnalisé intégré au moteur du jeu, qui distribue les appels de méthodes directement aux objets du monde du jeu et produit des conséquences immédiates et visibles. Le comportement des joueurs est suivi automatiquement et de manière asynchrone.

Le jeu a été évalué auprès d'étudiants en informatique de l'ESI issus de plusieurs années d'étude. Cinquante-deux étudiants ont joué au jeu et dix ont soumis des réponses complètes. L'évaluation réalisée à l'aide du cadre d'évaluation des jeux éducatifs MEEGA+ a montré que la Satisfaction (4,22/5,0) et l'Utilisabilité (3,95/5,0) constituaient les dimensions les mieux notées, tandis que l'aspect ludique et l'intention de rejouer ont obtenu chacun une note de 4,50/5,0.

Ces résultats suggèrent que Refactal : Reimagined répond avec succès aux limites documentées de ses prédécesseurs et constitue une contribution significative au corpus encore restreint des jeux sérieux consacrés à l'enseignement avancé du génie logiciel.

Mots-clés : Jeux Sérieux, Programmation Orientée Objet, Patrons de Conception, Suivi des Joueurs, Pédagogie par le jeu.

ملخص

شهدت الألعاب الجادّة المخصّصة لتعليم البرمجة نمواً ملحوظاً كمجال بحثي خلال العقد الماضي. ومع ذلك، فإن الغالبية العظمى من الألعاب الحالية تركّز على المفاهيم التمهيدية (الصياغة الأساسية، بُنى التحكّم، والتفكير الخوارزمي الأولي). ولا تزال الألعاب المصمّمة خصيصاً لتعليم البرمجة كائنية التوجّه المتقدّمة وأنماط التصميم البرمجي نادرة. يهدف هذا العمل إلى سدّ هذه الفجوة من خلال إعادة تصميم معمارية كاملة للعبة Refactal ، وهي لعبة تعليمية للبرمجة تم تطويرها في المدرسة الوطنية العليا للإعلام الآلي (ESI) عبر عدة مشاريع منذ عام ٢٠٢٢، حيث عانت إصداراتها السابقة من معدلات انسحاب مبكر مرتفعة وبنية تحتية محدودة لجمع البيانات.

اللعبة المعاد تصميمها، Refactal: Reimagined ، هي لعبة ألغاز ومنصّات ثنائية الأبعاد ذات تمرير جانبي تم تطويرها باستخدام Godot 4.6 ، حيث يتفاعل اللاعبون مع محطة فضائية متعطّلة من خلال كتابة شيفرة برمجية حقيقية وقابلة للتنفيذ. تغطي اللعبة ثلاثة مفاهيم متقدّمة في البرمجة كائنية التوجّه: مبدأ المسؤولية الواحدة (Single Responsibility Principle)، ونمط التصميم Builder ، ونمط التصميم Memento . تعتمد الآلية الأساسية للعبة على مفسّر مخصّص بالكامل ومضمّن داخل محرّك اللعبة، يقوم بتوجيه استدعاءات الدوال مباشرةً إلى الكائنات داخل عالم اللعبة ويُنتج نتائج فورية ومرئية. كما يتم تتبّع سلوك اللاعبين تلقائياً وبشكل غير متزامن.

تم تقييم اللعبة بمشاركة طلبة الإعلام الآلي في ESI من عدة سنوات دراسية. لعب ٥٢ طالباً اللعبة، وقدّم عشرة منهم استبيانات مكتملة. أظهر التقييم باستخدام إطار تقييم الألعاب التعليمية MEEGA+ أن الرضا (٢٢.٤ من ٠.٥) وقابلية الاستخدام (٩٥.٣ من ٠.٥) كانا أعلى الأبعاد

تقييماً، بينما حصل كل من عامل المتعة والرغبة في إعادة اللعب على تقييم قدره ٥٠٠٤ من ٥٠٠٠.

تشير هذه النتائج إلى أن Refactal: Reimagined نجحت في معالجة القيود الموثّقة في الإصدارات السابقة، وتمثل مساهمة مهمة ضمن المجال المحدود نسبياً للألعاب الجادّة الموجّهة لتعليم هندسة البرمجيات المتقدّمة.

الكلمات المفتاحية : الألعاب الجادة، البرمجة الكائنية التوجه، أنماط التصميم، تتبع اللاعب، التعلم المبني على اللعب

Table des matières

Dedication i

Acknowledgements ii

Abstract iii

Résumé iv

ملخص v

List of Figures xiv

List of Tables xv

List of Equations xvi

List of Abbreviations xviii

General Introduction 1

I State of the Art 4

1 Definitions and Concepts 5

1.1 Introduction 5

1.2 Definitions 6

1.2.1 Game-Based Learning 6

1.2.2 Advanced Object-Oriented Programming Concepts 6

1.3 Serious Games Development and Evaluation 8

1.3.1 Serious Games Design Frameworks 8

1.3.2 Evaluation of Serious Games and MEEGA+ 10

1.4 Serious Games for Advanced Notions 11

1.4.1 Relics of Chaos 11

1.4.2 UMLCraft . . . 14
1.5 Analysis and Discussion . . . 17
1.6 Conclusion . . . 17

2 Player Tracking in Games **19**
2.1 Introduction . . . 19
2.2 Engagement . . . 19
2.2.1 Motivation and Engagement . . . 19
2.2.2 Flow Theory . . . 20
2.2.3 Reward Systems and Progression Loops . . . 21
2.2.4 Player Behaviour and Game Characteristics . . . 22
2.3 Knowledge Tracing Models . . . 23
2.3.1 Learning Analytics . . . 23
2.3.2 Learner Modeling . . . 23
2.3.3 Knowledge Tracing Concepts . . . 24
2.4 Discussion and Synthesis . . . 26
2.5 Conclusion . . . 27

3 Games for Programming as Engaging Learning Tools **28**
3.1 Introduction . . . 28
3.2 Existing Games for Programming . . . 28
3.2.1 Robocode . . . 29
3.2.2 TIS-100 . . . 30
3.2.3 Screeps . . . 31
3.2.4 SHENZHEN I/O . . . 32
3.2.5 The Farmer Was Replaced . . . 34
3.3 Comparison of the Games . . . 36
3.4 Conclusion . . . 38

II Contribution **39**

4 Background Analysis **40**
4.1 Introduction . . . 40
4.2 Presentation of Refactal . . . 40
4.2.1 Storyline . . . 41
4.2.2 Refactal Versions . . . 41
4.3 Educational Objectives of Refactal . . . 45
4.4 Gameplay Overview . . . 45
4.5 Existing Mechanics and Progression Systems . . . 47
4.6 Dashboard and Player Data Analysis . . . 48
4.7 Analysis . . . 49
4.7.1 Analysis of Player Retention . . . 49
4.7.2 Analysis of Learning Difficulties . . . 51
4.7.3 Analysis of Engagement Issues . . . 52
4.7.4 Identification of Design Limitations . . . 52
4.8 Comparative Positioning with Existing Games for Programming . . . 52

4.9 Discussion and Synthesis . . . 53
4.10 Conclusion . . . 54

5 System Design **55**
5.1 Introduction . . . 55
5.2 Design Objectives and Redesign Strategy . . . 55
5.2.1 Redesign Objectives . . . 55
5.2.2 Scope . . . 56
5.3 Global Design Framework . . . 56
5.3.1 Core Design Philosophy . . . 56
5.3.2 Pedagogical Design . . . 57
5.3.3 Choice of Game Genre and Experience Direction . . . 57
5.4 Gameplay and Progression Design . . . 58
5.4.1 Exploration and Ability-Based Progression . . . 58
5.4.2 Puzzle Design . . . 58
5.4.3 Boss Design and Pattern Representation . . . 59
5.4.4 Narrative Design . . . 59
5.4.5 Room Design Principles . . . 59
5.4.6 Game Loop Design . . . 60
5.5 System and Technical Architecture . . . 62
5.5.1 System Architecture Overview . . . 62
5.5.2 Core System Components . . . 63
5.5.3 Data Flow and System Interactions . . . 64
5.5.4 Player Tracking and Analytics Design . . . 65
5.6 User Experience Design . . . 66
5.6.1 User Interface Design . . . 66
5.6.2 Interaction and Feedback Design . . . 67
5.7 Conclusion . . . 67

6 Implementation **68**
6.1 Introduction . . . 68
6.2 Technical Stack & Development Environment . . . 68
6.2.1 Engine Configuration & GDScript . . . 68
6.2.2 Cloud Integration & Local Persistence . . . 69
6.2.3 Development Tools . . . 69
6.2.4 Additional Software and Technologies Used . . . 69
6.3 Scene Organization and Systems . . . 70
6.3.1 Node Hierarchy Within a Room . . . 70
6.3.2 The Signal System . . . 71
6.4 Gameplay Mechanics Implementation . . . 72
6.4.1 Player Movement and Physics . . . 72
6.4.2 Interactive and Programming Objects . . . 72
6.4.3 Room Transitions and Progression . . . 74
6.5 Graphics Design . . . 74
6.5.1 Art Direction . . . 74
6.5.2 Master Colour Palette . . . 75

6.5.3 Tile System and Grid . . . 75
6.5.4 Parallax Background Layers . . . 75
6.5.5 Player Character and Object Sprites . . . 76
6.6 Puzzle System and Room Implementation . . . 77
6.6.1 The Code Execution System . . . 77
6.6.2 The Object Registry . . . 78
6.6.3 Advanced OOP Concept-Specific Validation . . . 78
6.6.4 Room Structure . . . 79
6.7 Enemy and Boss Implementation . . . 81
6.8 Progression and Unlock Systems . . . 81
6.8.1 The AutoSave System . . . 81
6.8.2 The Guide Progress System . . . 82
6.8.3 The Code Editor Lock . . . 82
6.8.4 Extras Gallery System . . . 83
6.9 Dialogue and Narrative Systems . . . 83
6.10 User Interface Implementation . . . 84
6.10.1 The Code Editor . . . 84
6.10.2 The Guide System . . . 85
6.10.3 The Quiz UI . . . 86
6.10.4 The Start Menu . . . 87
6.11 Tracking and Analytics Features . . . 88
6.12 Administrative Dashboard . . . 89
6.13 Conclusion . . . 91

7 Evaluation **92**
7.1 Introduction . . . 92
7.2 Evaluation Objectives . . . 92
7.3 Experimental Protocol . . . 93
7.3.1 Cohort-Based Version Distribution . . . 93
7.3.2 MEEGA+ Questionnaire . . . 94
7.3.3 Protocol Limitations . . . 96
7.4 Evaluation Metrics . . . 97
7.5 MEEGA+ Dimensions Evaluation . . . 97
7.5.1 Dimension-Level Results . . . 97
7.5.2 Per-Item Results . . . 98
7.5.3 Analysis . . . 100
7.6 MEEGA+ Pre/Post Knowledge Assessment Evaluation . . . 100
7.6.1 Results . . . 100
7.6.2 Analysis . . . 101
7.7 Comparative Analysis with Original Refactal . . . 102
7.8 Discussion of Results . . . 103
7.9 Threats to Validity . . . 104
7.10 Conclusion . . . 104

General Conclusion **106**

References **109**

Webography **112**

Appendices **112**

A Past Players' Feedback **113**

B Learning Difficulties **117**

C External Motivation **119**

D MEEGA+ Questionnaire **120**

Table des figures

1.1 A class diagram for the Adapter design pattern (Gamma, Helm, Johnson, & Vlissides, 1995). 7
1.2 Decomposition of the MEEGA+ model (Petri, von Wangenheim, & Borgatto, 2016). 10
1.3 Screen capture of Relics of Chaos, one of the questions about the last instructional objective (Vahldick & Nolli, 2026). 12
1.4 Screen capture of Relics of Chaos, example of an explanation screen (Vahldick & Nolli, 2026). 13
1.5 Bar chart comparing individual pre-test and post-test scores across all 14 students (Vahldick & Nolli, 2026). 14
1.6 Screen capture of UMLCraft, showing a mini-game from class diagram use case (Gkouma, Karpouzis, & Bibi, 2026). 15
1.7 Screen capture of UMLCraft, showing an exercise from class diagram use case (Gkouma et al., 2026). 16

2.1 Game interface. Left : level and difficulty selection screen. Center : screenshot of a play round. Right : player statistics accessible from the main menu (Alexandrovsky, Friehs, Grittner, et al., 2021). 20
2.2 Individual's feelings toward an experience, categorized by level of challenge and abilities needed (track2training, 2025). 21
2.3 The BKT model architecture, where latent variables for skills at step t are denoted as kt and their correspondingly observed variables for answers at the time step t are denoted as yt (Abdelrahman, Wang, & Nunes, 2023). . 25

3.1 Robocode logo (Robocode, 2001). 29
3.2 Screen capture of an instance of Robocode gameplay (Robocode, 2001). . . 29
3.3 TIS-100 game interface (Zachtronics, 2015a). 30
3.4 Screeps game world showing multiple player colonies (Screeps, LLC, 2016). 31
3.5 Screeps code editor and API documentation (Screeps, LLC, 2016). 32
3.6 SHENZHEN I/O circuit design interface (Zachtronics, 2016). 33
3.7 The Farmer Was Replaced gameplay showing the start of the game tutorial (Koster, 2024b). 34
3.8 The Farmer Was Replaced gameplay showing the code editor and farm grid (Koster, 2024b). 35
3.9 Technology progression tree showing unlockable features (Koster, 2024b). . 35
3.10 ACM Computer Science Curricula 2013 knowledge areas (ACM/IEEE-CS Joint Task Force, 2013). 37

4.1 Screen Capture of Refactal (Mouheb, 2023). 41

4.2 General Composition of Refactal version 1.0 (Morsli, 2022). 42
4.3 Ranking of players based on their scores (Morsli, 2022). 42
4.4 General Composition of Refactal version 2.0 (Mouheb, 2023), (New additions in red, already existing parts in blue). 43
4.5 General Composition of Refactal version 2.0 with complimentary additions (Makhloufi & Kasmi, 2024 ; Arrouche & Tirouche, 2024), (New additions in red, already existing parts in blue). 44
4.6 General Composition of Refactal version 3.0 (Hamdane, 2025), (New additions in red, already existing parts in blue). 45
4.7 Screen Capture of Refactal version 2.0 (Mouheb, 2023), showing the Tutorial 2 level. 46
4.8 Screen Capture of Refactal version 2.0 (Mouheb, 2023), showing the code editor with example partial code. 46
4.9 Screen Capture of Refactal version 2.0 (Mouheb, 2023), showing one of the hint slides in the Tutorial 1 level. 47
4.10 A sample of collected player data (Arrouche & Tirouche, 2024), missing rows are indicated in yellow. 48
4.11 A screen capture of the administrative dashboard, showing examples of charts in the Engagement tab (Arrouche & Tirouche, 2024). 49
4.12 Percentage pie chart of abandoning players by level (Arrouche & Tirouche, 2024). 50
4.13 Retention graphs of players by total time played and by level (Arrouche & Tirouche, 2024). 51

5.1 UML state diagram representing the puzzle design and room based progression. 59
5.2 A state machine representing the game loop, with states grouped into two distinct parts : Main Game Loop and Loop Breaks. 60
5.3 The grouping of OOP terms per category of difficulty. 61
5.4 General system architecture of the redesigned Refactal 63
5.5 General system components diagram of the redesigned Refactal 64
5.6 Sequence Diagram, code execution flow from player input through the interpreter to world response. 65

6.1 Generic Node Hierarchy within a Room. 71
6.2 Difference between the door object states. 73
6.3 A programmable platform (grey programming object) with an actively shrinking outline. 73
6.4 The evolution of the graphics of RE :RE over time. 74
6.5 Part of the tile grid and texture layout. 75
6.6 Foreground and background parallax separation. 76
6.7 The evolution of RE :RE's player character's design. 77
6.8 Some programming objects from RE :RE. 77
6.9 Screen capture of RE :RE, showing a tutorial room and gameplay interaction. 80
6.10 Screen capture of RE :RE, showing the boss room. 80
6.11 Screen caputre of RE :RE, showing the boss fight mid-action. 81
6.12 Initial concept art of the player character, shown to the player as an extra unlock. 83

6.13 The code editor panel, with annotated UI components. 85
6.14 The guide book rendered in its SubViewport overlay. 86
6.15 The quiz UI during a Find the Odd One Out question. 86
6.16 The quiz UI during a cloze question. 87
6.17 The main start menu screen. 88
6.18 The database schema, eight tables covering player identity, room navigation, quiz performance, etc. 89
6.19 Metabase visual SQL query builder and dashboard metrics visualization. . 90
6.20 Chart visualization of the quer built in Figure 6.19. 90
6.21 Screen capture of the main page of the administrative dashboard. 91
6.22 Screen capture of part of the Retention & Engagement tab of the administrative dashboard. 91

B.1 Average Completion Times per Level (Arrouche & Tirouche, 2024). 117
B.2 Screen capture of the administrative dashboard (Arrouche & Tirouche, 2024), showing average completion times per level. 117
B.3 Screen capture of the administrative dashboard (Arrouche & Tirouche, 2024), showing quiz answer distributions. 118

C.1 Recruitment email sent to students for beta-testing Refactal. 119

D.1 Screen capture of the start of the MEEGA+ questionnaire. 120
D.2 Screen capture of part of the MEEGA+ questionnaire. 121

Liste des tableaux

I Classification of games according to programming language, genre, platform and type of graphics . . . 36
II Comparative table of characteristics per game . . . 37
III Classification of games according to educational content . . . 38

I Results of the quiz experiment. . . . 61
II Expected vs. actual level of OOP understanding based on the quiz experiment. 62

I Master colour palette and primary roles. . . . 75
II Fields stored in user ://autosave.json. . . . 82
III The extras and their in-game unlock conditions. . . . 83

I Cohort-versioned builds distributed by institutional email. . . . 93
II MEEGA+ questionnaire items with dimension assignments (−2 to +2 scale). 94
III Self-assessment structure. . . . 96
IV MEEGA+ dimension means (n = 10, normalized 1–5 scale). . . . 98
V Per-item MEEGA+ results (n = 10). Normalized = raw + 3. . . . 98
VI Pre/post self-assessment (n = 10, scale 1–5). . . . 101
VII Comparative analysis between Refactal (ver 2.0.3) and Refactal : Reimagined pilot results (n = 10). . . . 102

I Responder feedback on general experience elements . . . 113
II Responder feedback on finishing the game (Only for players who haven't finished the game) . . . 113
III Responder feedback on general appreciations, annoyances and potential improvements . . . 114

List of Equations

4.1 Refactal Scoring Formula 48
7.1 Perceived Gain Formula 96
7.2 Hake Normalized Gain Formula 100

List of Abbreviations

AFM Additive Factor Model
AI Artificial Intelligence
API Application Programming Interface
AUC Area Under the Curve
BKT Bayesian Knowledge Tracing
CS Computer Science
CS-BKT Cross-Skill Bayesian Knowledge Tracing
DDE Design, Development, Evaluation
DIP Dependency Inversion Principle
DKT Deep Knowledge Tracing
DRY Don't Repeat Yourself
ESI Ecole Nationale Supérieure d'Informatique
F4ASG Framework for the Analysis and Design of Serious Games
FSM Finite State Machine
GBL Game-Based Learning
GoF Gang of Four
GRASP General Responsibility Assignment Software Patterns
HTTP Hypertext Transfer Protocol
I/O Input/Output
IDE Integrated Development Environment
IRT Item Response Theory
ISP Interface Segregation Principle
JSON JavaScript Object Notation
KT Knowledge Tracing
KTM Knowledge Tracing Machines
LSP Liskov Substitution Principle
MDA Mechanics, Dynamics, Aesthetics
MEEGA+ Model for the Evaluation of Educational Games and Gamification
MOOC Massive Open Online Course
N/A Not Applicable
NPC Non-Player Character
OCP Open-Closed Principle

OOP Object-Oriented Programming
OST Original Soundtrack
PFA Performance Factor Analysis
RE:RE Refactal: Reimagined
REST Representational State Transfer
RMSE Root Mean Square Error
RPG Role-Playing Game
SG Serious Game
SOLID Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
SQL Structured Query Language
SRP Single Responsibility Principle
UCS Universal Coding Scanner
UI User Interface
UML Unified Modeling Language

General Introduction

Context and Motivation

Explaining advanced concepts of software engineering, especially software design patterns, is difficult. In introductory courses that teach students how to program, there is an immediate feedback loop with them writing some code and then executing it. Students quickly see the result of their learning. Design patterns are a higher level of abstraction than writing lines of code. Reading about patterns in a lecture or a book doesn't allow students to fully appreciate them. The true test is having them use design patterns without any guidance.

Serious games have been proposed and studied as a means of bridging this gap. By embedding learning objectives directly within interactive gameplay, they create conditions for active experimentation and reflective practice that conventional instruction cannot easily replicate. A substantial and growing body of literature confirms their effectiveness as a complement to traditional teaching, more than one hundred relevant studies were published between 2015 and 2024 alone (Ishaq et al., 2024), spanning contexts from primary education to advanced university-level courses (Miljanovic & Bradbury, 2018). Yet the overwhelming majority of this work targets introductory concepts : basic syntax, control structures, and elementary algorithmic reasoning. Purpose-built serious games for advanced object-oriented programming and software design patterns remain comparatively rare.

This project situated itself directly within that gap. It builds upon Refactal, an educational coding game developed at the École Nationale Supérieure d'Informatique (ESI) across multiple End-of-Studies projects since 2022 (Morsli, 2022 ; Mouheb, 2023 ; Hamdane, 2025). Prior versions of Refactal demonstrated both the promise of the concept and a set of concrete, measurable limitations : severe early dropout (34% of players abandoned the game at the very first tutorial level, and 83% had quit by the end of the second chapter (Arrouche & Tirouche, 2024)), alongside a poorly calibrated difficulty curve, a static and non-adaptive hint system and a 3D interface that players reported as hardware-intensive and visually monotonous

Problem Statement

The central question this work addresses is the following : how can a video game serve as an effective pedagogical environment for teaching advanced software design patterns to students with limited prior exposure, while maintaining genuine player engagement

and collecting meaningful, structured learning analytics ? Prior analysis of Refactal cohort data (Arrouche & Tirouche, 2024) identified that the game's shortcomings were not isolated failures but deeply interconnected structural limitations spanning onboarding design, progression rigidity, engagement mechanics, and data infrastructure. These findings validate the need for a principled redesign rather than incremental patching, and establish the specific design targets that this project addresses.

Research Objectives

The redesign is driven by three primary objectives, reflecting the three axes along which prior versions fell short. The first is educational : the game must require players to actively write and apply real code that embodies the design patterns under study. Understanding must be demonstrated through correct program construction. The second objective is engagement : the redesigned game must address the onboarding and retention failures of prior iterations through a progressive tutorial sequence, contextual programming help accessible at all times, a coherent visual and narrative identity, and a game loop that sustains motivation across the full session, replacing the abrupt drop-off that characterised earlier cohorts. The third objective is analytic : the game must automatically collect structured, granular player behaviour data (room events, code resets, quiz responses) and transmit it asynchronously to a cloud storage solution. This replaces the limited two-metric evaluation (completion time and lines of code) of prior versions with a system capable of supporting genuine learning analytics.

Methodology

The redesigned Refactal (called Refactal : Reimagined) was prototyped and developed as a 2D side-scroller puzzle-platformer in Godot 4.6. While Refactal originally launched as a 3D first-person experience, Refactal : Reimagined takes place in a 2D lateral viewport. This design choice was made so that the player can see interactable objects, their spatial results, and code editor output simultaneously in one viewport, rather than having it split among multiple screens like its predecessor was criticised for. To facilitate writing real, functioning code that can interact with tangible objects in the world, a custom interpreter was developed inside the engine that can take players' programs through a full pipeline and emit them as live method calls on scene components. All player behaviour is logged automatically and asynchronously to a database, with each action recorded with a version string that contains the cohort the player session came from. The Refactal : Reimagined was deployed to students from each of ESI's coursework cohorts. Our analysis utilised a mixed-method approach drawing from two main instruments : the MEEGA+ education-specific game evaluation framework (Petri et al., 2016) and Hake's normalised gain calculation (Hake, 1998) to compare conceptual knowledge gain to a participant's individual baseline.

Report Structure

The remainder of this dissertation is organized into two main parts, subdivided into chapters :

- **Part I : State of the Art**
  - **Chapter 1 : Definitions and Concepts** introduces the underlying theories, including game-based learning, serious games, advanced object-oriented programming concepts, game design patterns, the MDA and F4ASG design methodologies, and the MEEGA+ evaluation tool, and surveys two state-of-the-art serious games developed in recent years for learning advanced programming concepts.
  - **Chapter 2 : Player Tracking in Games** presents a literature review of existing studies about players' engagement and retention, as well as knowledge tracing.
  - **Chapter 3 : Games for Programming as Engaging Learning Tools** discusses existing games for programming that are, first and foremost, engaging.
- **Part II : Contribution**
  - **Chapter 4 : Background Analysis** presents an analysis of the Refactal family (versions 1.0, 2.0 and 3.0), capturing the design issues observed in previous versions through gameplay data as well as qualitative user feedback.
  - **Chapter 5 : System Design** describes the redesign of Refactal, including design goals, genre, design philosophy, learning decisions, system architecture, and UI mockups.
  - **Chapter 6 : Implementation** documents the technical implementation of the redesigned game as well as its subsystems.
  - **Chapter 7 : Evaluation** discusses the evaluation that was carried out with the redesigned Refactal, presenting the results of the MEEGA+ questionnaire, as well as a comparison with metrics originally captured for Refactal.
- Finally, a **General Conclusion** summarizes the main contributions, details research limitations, and outlines future avenues of development and perspectives.

Première partie

State of the Art

Chapitre 1

Definitions and Concepts

1.1 Introduction

Serious games have been widely adopted in programming education as a means to address long-standing challenges related to learner engagement, motivation, and the comprehension of abstract computational concepts. Review studies (Miljanovic & Bradbury, 2018 ; Ishaq et al., 2024 ; Gundersen & Lampropoulos, 2025) highlighted the pedagogical potential of serious games to complement traditional instruction by offering interactive environments, immediate feedback, and contextualized problem solving. In particular, the 2018 review of serious games for programming education emphasizes that game-based approaches were primarily used to make introductory programming concepts more accessible and engaging, especially for novice learners who often struggle with the perceived difficulty and abstraction of programming courses (Miljanovic & Bradbury, 2018).

Across the body of reviewed literature, a clear evolution can be observed in the pedagogical focus of serious games for programming. Early studies primarily concentrated on introductory syntax, basic control structures, and elementary algorithmic reasoning, often relying on block-based or highly guided environments. More recent work, however, emphasizes personalization, adaptive feedback, and structured learning progressions that align with learners' cognitive development. These newer approaches aim to teach programming constructs without neglecting to support self-regulated learning, problem decomposition, and incremental mastery of concepts through carefully designed gameplay mechanics.

This growing research interest is also reflected in the scale of recent literature surveys. More than one hundred relevant studies on gamified and game-based learning in programming education were published between 2015 and 2024 (Ishaq et al., 2024), indicating sustained and increasing academic attention to the topic. These studies span a wide range of educational contexts, from primary and secondary education to introductory university-level courses (Miljanovic & Bradbury, 2018), and collectively suggest that serious games can be effective tools for improving both engagement and learning performance when they are pedagogically well designed.

In this chapter, we establish some definitions necessary for understanding the subsequent discussion, such as game-based learning, advanced OOP concepts and serious

game development and evaluations. We will also discuss some recent serious games, chosen for tackling advanced notions (programming paradigm specific concepts, advanced OOP concepts, elements of software design, ... etc).

1.2 Definitions

1.2.1 Game-Based Learning

Game-Based Learning (GBL) refers to the pedagogical use of games to achieve defined learning outcomes. Rather than simply motivating students through play, GBL integrates game mechanics directly aligned with educational objectives, creating environments that promote exploration, experimentation, and reflective learning.

GBL supports both knowledge acquisition and behavioral transformation (Ahmed & Sutton, 2017), encouraging learners to think critically and adaptively. Learners build understanding through experience, trial, and error. When properly designed, these environments mimic real-world scenarios, allowing learners to internalize abstract concepts.

— **Gamification :** may be defined as the process of integrating game theory and design, game elements, game aesthetics, and game mechanics into a learning experience in order to create motivation and a sense of enjoyment within non-game contexts (Ahmed & Sutton, 2017 ; Dörner, Göbel, Effelsberg, & Wiemeyer, 2016).

— **Serious Games :** may be defined as games in which education, training, or another practical objective constitutes the primary goal rather than pure entertainment (Michael & Chen, 2005). The emphasis isn't put on the narrative, art, nor software, rather serious games inherently involve pedagogy through activities designed to educate or instruct, thereby imparting knowledge or skills, which is why they are serious in nature (Zyda, 2005). In this sense, serious games are characterized by an explicit and carefully considered educational purpose and are not intended to be played primarily for amusement, although this does not imply that they cannot or should not be entertaining (Abt, 1987).

1.2.2 Advanced Object-Oriented Programming Concepts

Advanced Object-Oriented Programming (OOP) concepts represent a critical transition point in programming education, where learners move from writing functional code to designing maintainable, extensible, and professionally structured software systems. We will introduce the formal notion of design patterns as reusable solutions to recurring problems, then explore complementary principles that collectively guide developers toward higher-quality software design.

Design Patterns

A Software Design Pattern or Design Pattern is a general, reusable solution to a commonly occurring problem in many contexts in software design. In *Design Patterns :*

Elements of Reusable Object-Oriented Software (Gamma et al., 1995), Erich Gamma et al borrow Christopher Alexander's definition , "*Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice*".

Gamma et al. classify design patterns into three broad categories according to their purpose : creational, structural, and behavioral patterns.

Creational Patterns focus on object creation mechanisms, aiming to decouple a system from the specific classes it instantiates. A representative example is the **Builder** pattern, which lets you construct complex objects step by step, allowing you to produce different types and representations of an object using the same construction code. (Gamma et al., 1995). The five Creational Patterns are **Abstract Factory**, **Builder**, **Factory Method**, **Prototype** and **Singleton** (Gamma et al., 1995).

Structural Patterns deal with the composition of classes and objects to form larger structures while keeping them flexible and efficient. The **Adapter** pattern (see Figure 1.1) exemplifies this category by enabling incompatible interfaces to work together. It acts as a bridge between two otherwise incompatible components, allowing an existing class to be reused in a new context without modification (Gamma et al., 1995). The seven Structural Patterns are **Adapter, Bridge, Composite, Decorator, Facade, Flyweight** and **Proxy** (Gamma et al., 1995).

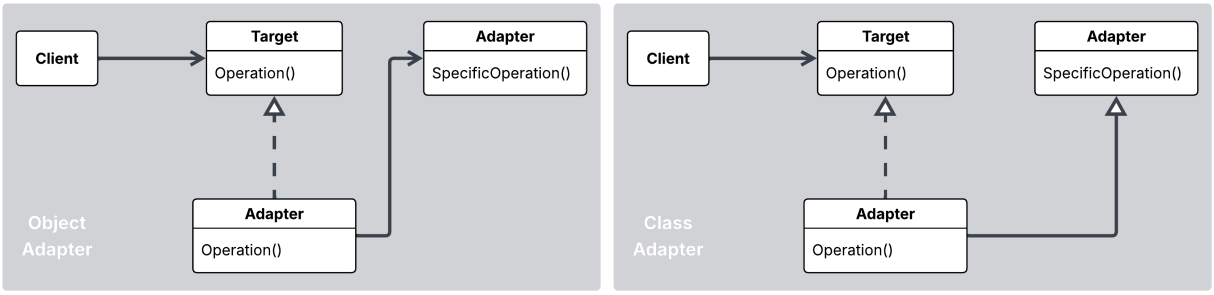


FIGURE 1.1 – A class diagram for the Adapter design pattern (Gamma et al., 1995).

Behavioral Patterns are concerned with communication and responsibility distribution among objects, emphasizing how objects interact and collaborate to perform tasks. A canonical example is the **Memento** pattern, which lets you save and restore the previous state of an object without revealing the details of its implementation (Gamma et al., 1995). The eleven Behavioural Patterns are **Chain of Responsibility, Command, Iterator, Mediator, Memento, Observer, State, Strategy, Template Method** and **Visitor** (Gamma et al., 1995).

SOLID Principles

While basic object-oriented programming teaches classes, objects, inheritance, and polymorphism, advanced OOP concepts provide principles and guidelines that help write better, more maintainable code. These concepts serve as a bridge between understanding the syntax of OOP and becoming a proficient software designer. Having gone over Design

Patterns above, in this section, we explore one additional advanced OOP concept : SOLID Principles.

SOLID is an acronym representing five fundamental principles of object-oriented design, introduced by Robert C. Martin (Martin, 2013). These principles, when applied together, make software designs more understandable, flexible, and maintainable. Each principle addresses a specific aspect of good design and helps developers avoid common pitfalls that lead to rigid, fragile code :

- **S - Single Responsibility Principle (SRP) :** a class should have one, and only one, reason to change. In other words, a class should have only one job or responsibility. This doesn't mean a class should only have one method ; rather, all of its methods should relate to a single, well-defined purpose (Martin & Sommerville, 2004).
- **O - Open-Closed Principle (OCP) :** software entities (classes, modules, functions) should be open for extension but closed for modification. This means we should be able to add new functionality without changing existing code (Martin & Sommerville, 2004).
- **L - Liskov Substitution Principle (LSP) :** formulated by Barbara Liskov, states that objects of a superclass should be replaceable with objects of its subclasses without breaking the application. In simpler terms, derived classes must be substitutable for their base classes (Martin & Sommerville, 2004).
- **I - Interface Segregation Principle (ISP) :** no client should be forced to depend on methods it doesn't use. Large, generalized interfaces should be split into smaller, more specific ones so that implementing classes only need to concern themselves with methods relevant to them (Martin & Sommerville, 2004).
- **D - Dependency Inversion Principle (DIP) :** high-level modules should not depend on low-level modules ; both should depend on abstractions. Additionally, abstractions should not depend on details ; details should depend on abstractions (Martin & Sommerville, 2004).

1.3 Serious Games Development and Evaluation

1.3.1 Serious Games Design Frameworks

A framework in game design is more than a conceptual convenience ; it is an operative tool that structures the iterative decisions made throughout a game's development lifecycle. From determining the rules of interaction to predicting how those rules will produce meaningful player experiences, frameworks help designers reason explicitly about the relationship between their creative decisions and the ultimate effect those decisions will have on their audience. Two frameworks have proven particularly foundational in both entertainment and applications in serious game design : MDA and F4ASG, distinguishing itself from MDA by being expressly made for serious game design.

The MDA Framework

The Mechanics, Dynamics, and Aesthetics (MDA) framework is a formal approach to understanding games. Introduced by Hunicke, LeBlanc, and Zubek in 2004, it aims to bridge the gap between game design, development, research and criticism (Hunicke, LeBlanc, & Zubek, 2004). The MDA framework splits games into three components to uncover how design decisions influence player experience.

— **Mechanics** are the various actions, behaviors and control mechanisms afforded to the player within a game context (Hunicke et al., 2004). For example, the mechanics of card games include shuffling, trick-taking and betting.
— **Dynamics** refers to game mechanics' interactions and behaviour based on player inputs and each others' outputs over time (Hunicke et al., 2004).
— **Aesthetics** describes the desirable emotional responses evoked in the player, when they interact with the game system (Hunicke et al., 2004).

To concretize the concept of aesthetics, Hunicke et al. proposed a vocabulary of eight aesthetic categories that different games may pursue to varying degrees : **Sensation** (game as sense-pleasure), **Fantasy** (game as make-believe), **Narrative** (game as drama), **Challenge** (game as obstacle course), **Fellowship** (game as social framework), **Discovery** (game as uncharted territory), **Expression** (game as self-discovery), and **Submission** (game as pastime) (Hunicke et al., 2004). Rather than a fixed taxonomy, these categories function as a descriptive compass, a vocabulary that enables designers to articulate design goals and evaluate whether particular dynamics and mechanics actually serve those goals.

The Framework for Adaptive Serious Games (F4ASG)

The most comprehensive framework specifically designed to address adaptivity in serious games is the Framework for Adaptive Serious Games (F4ASG), developed by Pistono, Santos, Baptista, and Mamede (2024) using the Design Science Research (DSR) methodology. F4ASG was constructed in response to a gap identified in the literature : while numerous frameworks exist that are fit for the design and evaluation of serious games, none systematically relates game elements to learning outcomes while also accommodating the forms of adaptation that are critical to effectiveness in professional training contexts.

F4ASG is based on a systematic literature review of 53 publications on serious games in professional training, supplemented by two focus groups (one with developers of training solutions, one with human resources and learning professionals) and two expert interviews with university professors experienced in game-based learning (Pistono, dos Santos, Baptista, & Mamede, 2024). This multi-faceted approach allowed the framework's authors to integrate both theoretical perspectives and practitioner experience into its design.

F4ASG is organized around two sets of dimensions, Learning and Game, each of which is articulated through four dimensions. The Learning dimensions are : **L1** (Content/Context/Framing), **L2** (Instructional Elements), **L3** (Learning Outcomes), and **L4** (Learning Adaptation). The Game dimensions are : **G1** (Game Mechanics), **G2** (Game Environment), **G3** (Results), and **G4** (Game Adaptation) (Pistono et al., 2024). The two domains are structured and subdivided similarly as **L1** corresponds to **G1**, **L2**

to **G2**, **L3** to **G3**, and **L4** to **G4**, reflecting the authors' view that effective serious game design requires learning and game considerations to be developed in parallel rather than sequentially.

1.3.2 Evaluation of Serious Games and MEEGA+

The question of how to evaluate serious games is itself a significant methodological challenge. Games are complex artifacts whose effects on players unfold over time, are mediated by individual differences, and manifest at multiple levels (from immediate engagement during play to longer-term changes in knowledge, skills, and behavior).

Among the evaluation instruments developed specifically for educational games, the MEEGA+ (Model for the Evaluation of Educational Games) (Petri et al., 2016) model deserves particular mention in the context of programming education. MEEGA+ is a standardized questionnaire-based instrument designed to assess the quality of educational games across two primary dimensions : player experience (including focused attention, fun, challenge, ... etc) and usability (learnability, operability, aesthetics and accessibility) (Petri et al., 2016). Its standardized item structure allows comparative evaluation across different games and different populations, addressing a methodological weakness identified by multiple systematic literature reviews : the proliferation of ad hoc evaluation instruments that make cross-study comparisons an arduous task. Figure 1.2 provides the general decomposition of the MEEGA+ model.

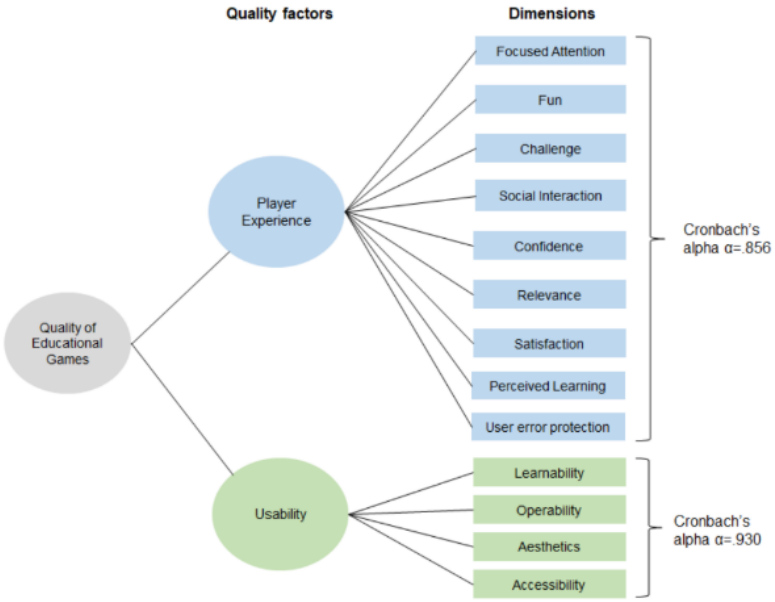


FIGURE 1.2 – Decomposition of the MEEGA+ model (Petri et al., 2016).

MEEGA+ has been applied in numerous studies evaluating games designed to teach programming concepts, data structures, software engineering, and related topics (Gundersen & Lampropoulos, 2025 ; Vahldick & Nolli, 2026). Its inclusion of both motivational and cognitive dimensions reflects the consensus view in the serious games literature that engagement and learning are not separable targets but mutually reinforcing outcomes that any effective serious game must produce together.

1.4 Serious Games for Advanced Notions

1.4.1 Relics of Chaos

Relics of Chaos is a 2D top-down Role-Playing Game (RPG) developed using RPG Maker MZ to introduce students to the Visitor design pattern (Vahldick & Nolli, 2026). The game takes place across three maps (the city of Miriadin, the Enchanted Dark Forest, and a cave) where the player controls a protagonist named Edivaldo on a quest to retrieve the Dark Bear's Hide. Learning content is woven directly into the game's narrative, with NPCs assigning instructional tasks that must be completed to advance the story.

Game Mechanics and Progression
The core gameplay follows a top-down RPG structure combining exploration, NPC dialogue, and turn-based combat. Progression is gated by instructional tasks : players cannot advance to new areas of the map until they successfully complete the learning challenges at their current location. For example, the protagonist is blocked from entering the Enchanted Forest by city guards until the player completes the learning activities in the City Depot (Vahldick & Nolli, 2026).

Combat encounters are also tied to the instructional content. Before each battle with a guardian enemy, students must correctly answer a multiple-choice question identifying erroneous code fragments (see Figure 1.3). After answering correctly they can enter the turn-based battle, which uses RPG Maker's native battle system with elemental attack types (Vahldick & Nolli, 2026). This creates a loop where conceptual understanding unlocks gameplay rewards, reinforcing the learning objectives in context.

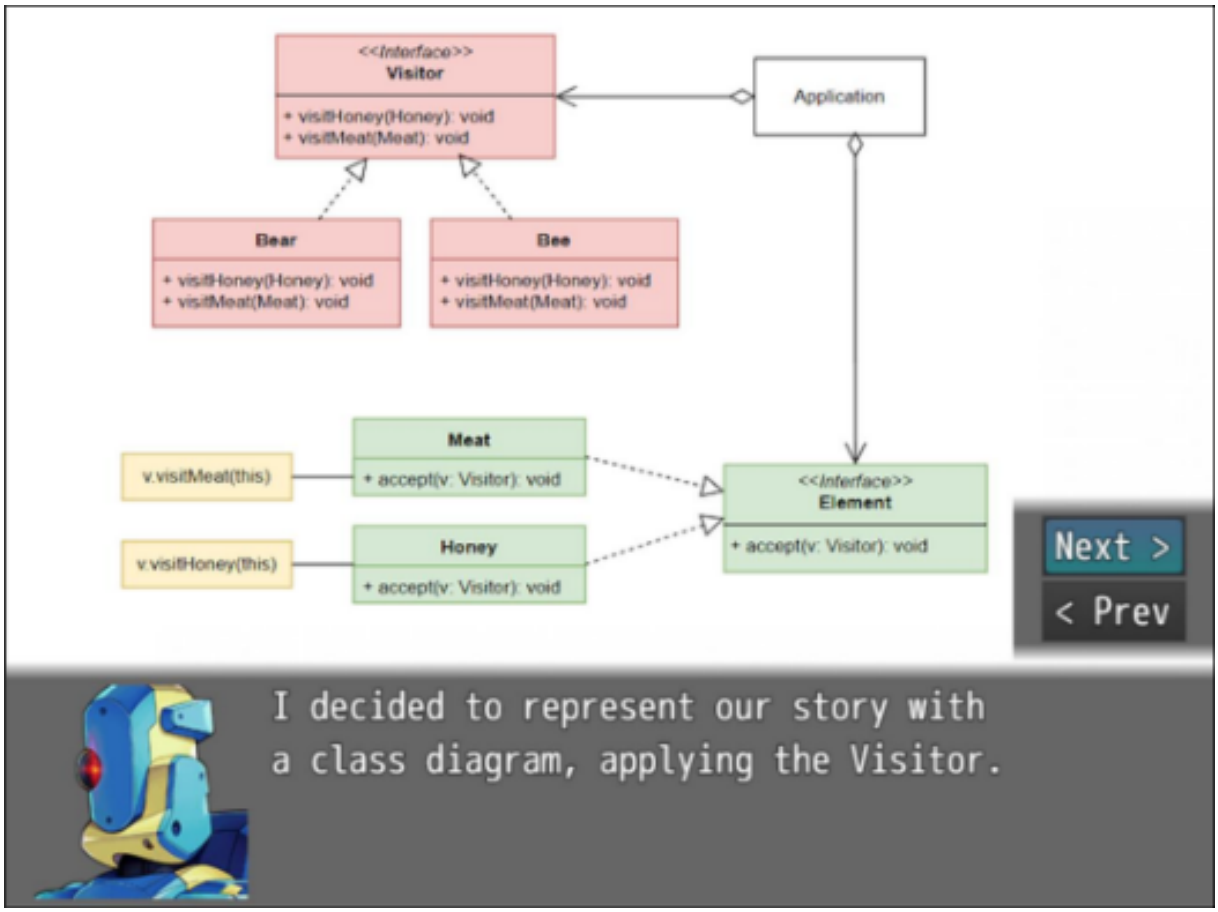


FIGURE 1.3 – Screen capture of Relics of Chaos, one of the questions about the last instructional objective (Vahldick & Nolli, 2026).

Visitor Design Pattern Concepts

The game is structured around four instructional objectives (Vahldick & Nolli, 2026) :

— **Intent and Use :** An NPC robot introduces the purpose of the Visitor pattern through explanation screens featuring UML class diagrams (see Figure 1.4). A hunter-and-animals case study is used as a concrete metaphor — the same bait (visitor) affects different animals (elements) differently — grounding the abstract pattern in an intuitive scenario (Vahldick & Nolli, 2026).

— **Structure and Components :** The same robot NPC presents the three core components of the pattern (visitors, visitables, and the object structure) through navigable diagram screens, followed by multiple-choice comprehension questions (Vahldick & Nolli, 2026).

— **Interaction and Functioning :** A second NPC re-explains the pattern's interaction logic using Java source code rather than class diagrams (Vahldick & Nolli, 2026), reinforcing the same concepts at a more concrete level.

— **Recognition in Code :** At three separate points, students are shown fragments of five classes (Refer back to figure 15) and must identify which two are incorrectly implemented before unlocking the corresponding combat encounter (Vahldick & Nolli, 2026).



FIGURE 1.4 – Screen capture of Relics of Chaos, example of an explanation screen (Vahldick & Nolli, 2026).

The hunter-and-animals metaphor also functions as a gameplay hint for the final boss encounter : knowing that the bear calms down when fed honey and grows aggressive when fed meat gives the player a gameplay advantage, rewarding careful attention to the instructional content (Vahldick & Nolli, 2026).

Educational Value and Limitations The game's principal strength is its integration of instructional content into an engaging narrative. Students noted that the pattern explanations did not break immersion because they were well embedded in the storyline (Vahldick & Nolli, 2026). The pre-test and post-test results (see Figure 1.5) demonstrated statistically significant learning gains ($p = 9.395 \times 10^{-6}$, $d = 1.76$), with the mean score rising from 2.4 to 5.9 out of 10 (Vahldick & Nolli, 2026). The instructor further reported that the class average on a subsequent programming exam improved from 5.0 in previous semesters to 8.0 for the cohort that played the game (Vahldick & Nolli, 2026). Furthermore, evaluation of the results were done through statistical hypothesis validation and the MEEGA+ questionnaire.

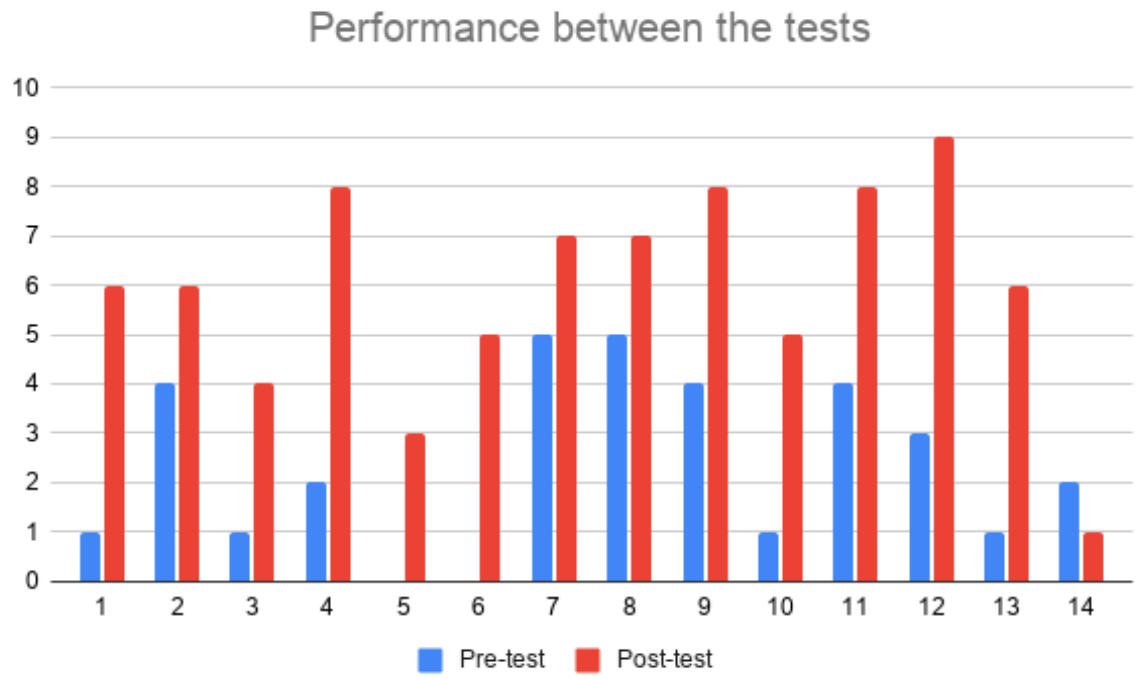


FIGURE 1.5 – Bar chart comparing individual pre-test and post-test scores across all 14 students (Vahldick & Nolli, 2026).

However, several limitations are worth noting :

— **No practical coding tasks :** The game's reliance on RPG Maker MZ constrained the task types available. An intended error-correction task for the application objective had to be reduced to multiple-choice questions due to tooling limitations (Vahldick & Nolli, 2026), which restricts the depth of hands-on engagement with actual code.

— **Battle system usability :** Several students struggled with the turn-based combat mechanics. With only 50 minutes allotted for gameplay, many did not use the available tutorial, leading to frustration and dissatisfaction with the time spent playing (reflected in disagreement responses on MEEGA+ item Q28) (Vahldick & Nolli, 2026).

— **Small sample size :** The validation involved only 14 students from a single course cohort at one institution.

1.4.2 UMLCraft

UMLcraft is a 2D computer game developed in the Unity Game Engine using C# to complement the teaching of Unified Modeling Language (UML) diagrams in software engineering education (Gkouma et al., 2026). Rather than embedding learning content in a narrative, the game takes a direct instructional approach, guiding learners through four cognitive levels (familiarizing, remembering, understanding, and applying) across four core UML diagram types : activity, use case, class, and sequence diagrams. Its target audience is undergraduate and postgraduate students in programs that include UML coursework,

and the game can be played individually inside or outside a classroom with a default maximum session duration of 94 minutes.

Game Mechanics and Progression

The game is structured around two complementary activity types. The first is a point-and-click memory match minigame (See Figure 1.6) in which players pair UML notation cards before a timer expires, targeting the familiarizing and remembering levels. The second is a set of five types of drag-and-drop exercises of increasing difficulty (See Figure 1.7), ranging from answering questions on a pre-built diagram to constructing one from scratch based on a textual description, targeting the understanding and applying levels. Progression in difficulty is built into the exercise sequence rather than gated by correct answers, and a reward system based on hearts (lives) and gems (virtual currency) sustains engagement : players earn gems proportional to their completion speed and can spend them in an in-game store to upgrade timers or expand their heart limit.

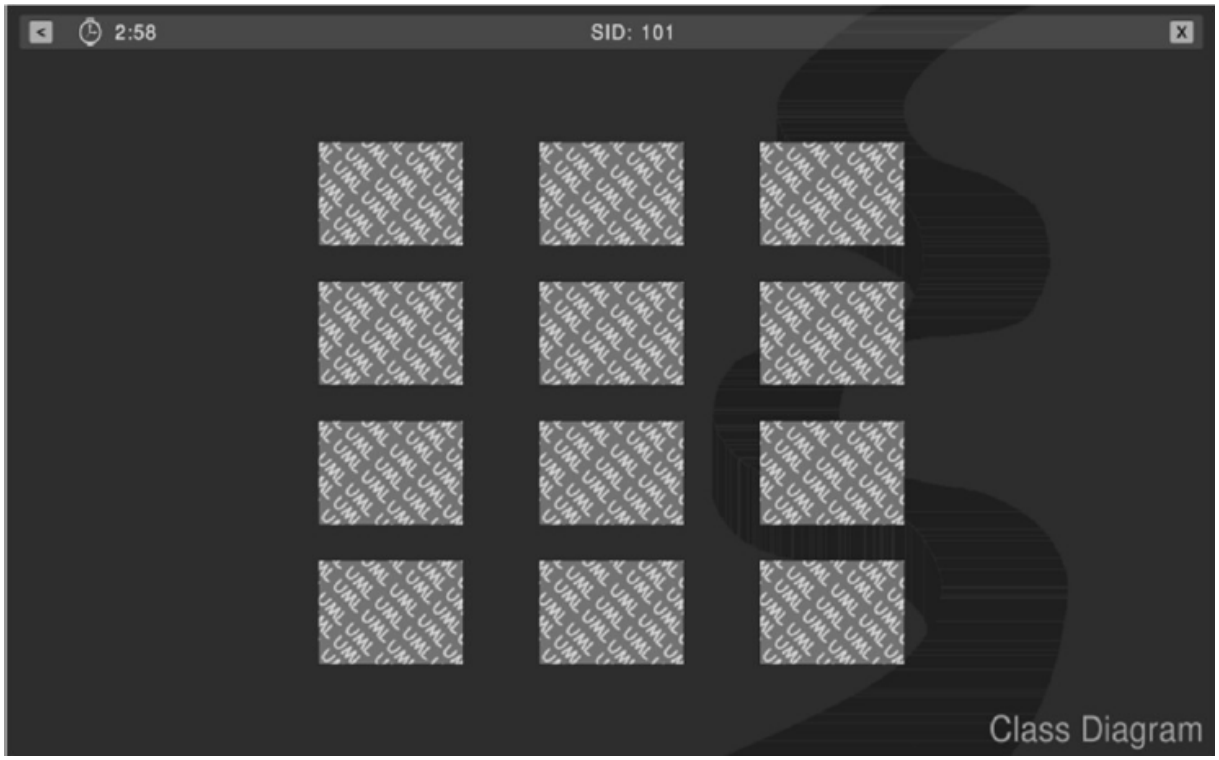


FIGURE 1.6 – Screen capture of UMLCraft, showing a mini-game from class diagram use case (Gkouma et al., 2026).

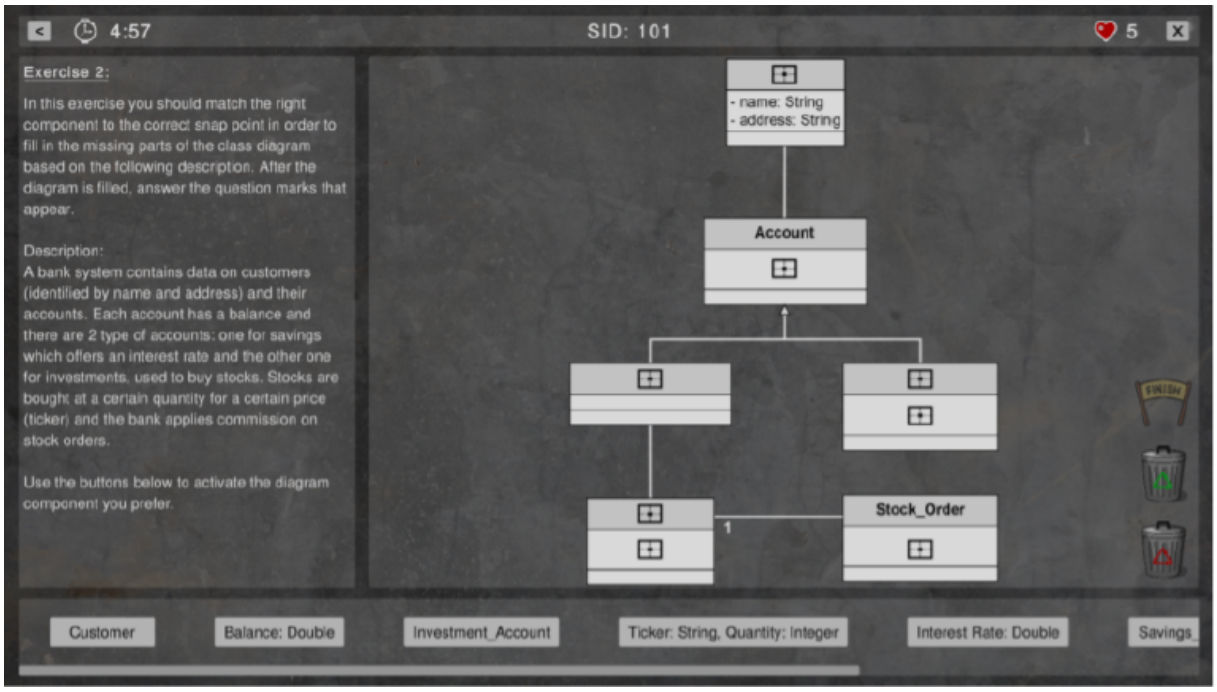


FIGURE 1.7 – Screen capture of UMLCraft, showing an exercise from class diagram use case (Gkouma et al., 2026).

UML Concepts Covered

For each diagram type, the player has access to a theory reference section, instructions, a minigame, and multiple exercises. The four instructional objectives that structure both the game design and the study's evaluation are (Gkouma et al., 2026) :

- **Familiarizing :** naming the different diagram types, their objectives, and the development phases they affect.
- **Remembering :** recalling modeling rules, element types, relationships, and semantics.
- **Understanding :** recognizing and describing elements of a given diagram along with their associations and dependency relationships.
- **Applying :** modeling a UML diagram from scratch given a textual description of a system.

Educational Value and Limitations The game was evaluated with 33 fourth-year undergraduate engineering students, 17 in a test group (Group A) and 16 in a control group (Group B). Across all four cognitive levels, quiz accuracy and task completion time both improved significantly between the pre- and post-play phases, with the strongest gains observed in understanding and applying. Students responded positively in surveys and semi-structured interviews, highlighting the motivating effect of the timer, the reward system, and the theory reference screens. The treatment group also outperformed the control group on the final course examination (mean 5.43 vs. 3.86), though this difference did not reach statistical significance (p ¿ 0.05) (Gkouma et al., 2026).

However, several limitations are worth noting :

- **Limited exercise variety :** The game offers only 1–3 minigames and 3–4 exercises per diagram type. Several students completed all rounds quickly without achieving

mastery, and many were still unable to answer correctly even after a second playthrough.

— **Cognitive load in complex exercises :** The final exercises per diagram type presented substantially higher complexity, leading to performance drops suggesting that difficulty scaling needs to be more granular (ideally decomposed into smaller sequential episodes).

— **Late integration into the course :** The evaluation was conducted at the end of the semester, after students had already attended lectures and completed assignments. The authors note that embedding the game progressively throughout the course would likely improve its impact.

— **Scope and sample :** Only 17 students played the game, and only 4 of UML's 14 diagram types are covered, limiting both statistical strength and the breadth of the game's applicability.

1.5 Analysis and Discussion

The Visitor pattern implementation in Relics of Chaos effectively combines course content with narrative gameplay. It uses story elements and code inspection tasks within an RPG setting, which helps reduce the intimidation often linked to abstract design patterns. The game also uses gated progression, meaning players must understand key concepts before moving forward, which encourages a sense of achievement. The original study reports strong learning improvements (significant pre-post gains, $p < 0.001$), supporting its effectiveness. However, because it mainly relies on multiple-choice questions and does not include real coding practice, it may not fully prepare learners for real-world programming tasks. Adding small coding or debugging activities inside the game could improve practical understanding.

In contrast, UMLCraft focuses more on structured learning tasks such as memory games and drag-and-drop exercises that gradually build understanding from basic recognition to application. Its reward system (hearts and gems) helps keep learners engaged, and the step-by-step design supports steady difficulty progression. Results show that it improves quiz performance and task completion speed, suggesting that active, repeated practice helps learners understand UML concepts better. However, the limited variety of exercises for each diagram type and its late introduction in the course reduce its long-term effectiveness. This could be improved by adding more types of mini-games, adjusting difficulty automatically, and integrating the tool earlier and more consistently throughout the course.

1.6 Conclusion

This chapter introduced some key terms used in the dissertation and presented two case studies that show different ways of teaching advanced programming concepts.

Relics of Chaos demonstrates a narrative-based learning approach that is engaging and motivating, but it offers limited real coding practice. UMLCraft uses a more structured,

task-based approach that helps reinforce UML concepts effectively, but it lacks variety in exercises and introduces them at less optimal times.

Together, these examples show a balance between immersion and hands-on practice. These insights help guide the design choices for the upcoming Refactal remake.

Chapitre 2

Player Tracking in Games

2.1 Introduction

Understanding how players engage with video games, and what keeps them coming back, is a central challenge in game design. Additionally, the rise of educational technology has spurred the development of computational models that track and predict learners' knowledge states. The following section bridges these two domains : player engagement and retention in single-player offline games, and knowledge tracing models used in intelligent tutoring systems. By examining motivation theory, flow theory, reward systems, and learner modeling, we provide a synthesized perspective on how player (or learner) behavior can be tracked, modeled, and influenced over time.

2.2 Engagement

2.2.1 Motivation and Engagement

Engagement in games is fundamentally driven by motivation. Motivation can be split into two main categories : intrinsic motivation (doing an activity for its inherent satisfaction) and extrinsic motivation (driven by external rewards). In gaming contexts, players who are intrinsically motivated tend to exhibit deeper engagement and longer play sessions. Teng et al. (2012) demonstrated that achievement striving (the desire for achievements) and competence (confidence in one's abilities) are positively related to gaming skill and perceived challenge, which in turn foster flow and ultimately gamer loyalty. Their structural model, validated with 994 online gamers, showed that achievement-related personality traits indirectly influence loyalty through skill, challenge, and flow.

The concept of "snacking mechanics", game design elements that encourage brief but regular play sessions, has emerged as a framework for sustained engagement in serious games. Alexandrovsky et al. (2021) experimentally tested five such mechanics (**Rewards, Novelty, Completion, Waiting,** and **Blocking**) in a mobile math-learning game (see Figure 2.1 for screen captures of the game).



FIGURE 2.1 – Game interface. Left : level and difficulty selection screen. Center : screenshot of a play round. Right : player statistics accessible from the main menu (Alexandrovsky et al., 2021).

Using survival analysis over a three-week field study, they found that **Completion** (achievement systems with missions and goals) and **Blocking** (time-limited access to gameplay) facilitated the strongest sustained engagement. Notably, **Completion** mechanics yielded the longest participation (around 20–25 days) compared to other conditions, supporting the idea that clear goals and a sense of accomplishment are powerful retention drivers.

2.2.2 Flow Theory

Flow theory, originally developed by Csikszentmihalyi (1992), describes the "optimal experience", a state of deep absorption where challenges match skills (i.e abilities), goals are clear, feedback is immediate, and self-consciousness disappears. This state is characterized by intense concentration, a sense of control, and the distortion of time. In video games, flow has been identified as a major predictor of player enjoyment and continued engagement.

Figure 2.2 categorizes an individual's feeling during their experience according to the level of perceived challenge in the task *and* their abilities (skills) in regards to said task. Feeling **apathy** (i.e showing or feeling no interest, enthusiasm, or concern), when the experience doesn't challenge them enough nor require adequate display of their abilities. Meanwhile a challenging experience that makes use of their abilities enables them to enter the **Flow Zone** (Beck, 1992).

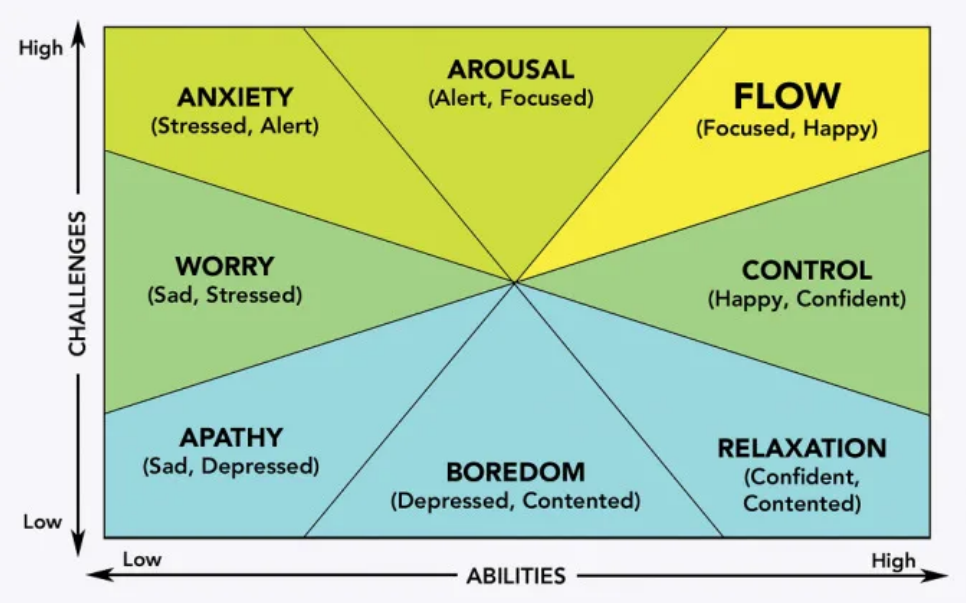


FIGURE 2.2 – Individual's feelings toward an experience, categorized by level of challenge and abilities needed (track2training, 2025).

However, the operationalization of flow (i.e measuring flow, which is not directly measurable) remains contested. Abuhamdeh (2020) conducted a conceptual analysis of flow research over five years and identified 24 distinct ways flow has been operationalized across 42 studies. Three key inconsistencies were highlighted : (1) whether flow is a discrete or continuous construct ; (2) whether flow is inherently enjoyable (autotelic, i.e having an end or purpose in itself) ; and (3) whether flow should be distinguished from its proposed antecedents (clear goals, immediate feedback, challenge-skill balance). It is argued that flow should be conceptualized exclusively as a discrete, highly enjoyable "optimal" state, and that more mundane states of engagement are better captured by existing constructs like "task involvement" and "intrinsic motivation" (Abuhamdeh, 2020).

In gaming contexts, the challenge-skill balance is central to flow. Allart et al. (2017) used survival analysis on telemetry data from two commercial games (Rayman Legends (Ubisoft, 2013) and Tom Clancy's The Division (Ubisoft, 2016)) to study the link between difficulty and player retention. They found that higher difficulty was correlated with higher retention in both games, but with different temporal dynamics. In Rayman Legends (Ubisoft, 2013), players preferred lower difficulty early on and did not like rapid difficulty changes — consistent with self-efficacy theory (Bandura, 1977), which suggests that early failure can harm the belief in one's capacity to succeed. In The Division (Ubisoft, 2016), where failure can be attributed to the avatar's strength rather than pure skill, difficulty variation had no negative impact. These findings confirm that flow and self-efficacy theory operate differently depending on the attribution of failure.

2.2.3 Reward Systems and Progression Loops

Reward systems are among the most powerful retention mechanics in games. They provide players with immediate feedback for their actions, reinforcing desired behaviors

through operant conditioning. In the "snacking" framework tested by Alexandrovsky et al. (2021), **Rewards** (scoring mechanics and power-ups) did not produce the strongest retention compared to **Completion** and **Blocking**, but still outperformed the baseline condition. The authors noted that reward-based motivation may decay over time if rewards are perceived as unpredictable or disconnected from player effort.

Stråat and Verhagen (2018) conducted a longitudinal case study with seven participants playing Tom Clancy's The Division over five weeks, collecting player diaries and survey data. They found that the loot reward system, while initially exciting, frustrated participants when rewards felt disconnected from effort and when the abundance of information about gear and skills became overwhelming. Participants reported that "the rewards are ambiguous" and that it was hard to maintain motivation through repetitive gameplay. This study highlights that reward systems must be carefully calibrated : they need to provide meaningful progression without creating a sense of tedious repetition or information overload.

2.2.4 Player Behaviour and Game Characteristics

Player Retention and Replayability

Player retention, the ability to keep players engaged and playing over time, is critical for the commercial success of games. Stråat and Verhagen (2018) used Hassenzahl's Model of User Experience to analyze how players' attitudes changed over 20 hours of gameplay. All participants showed decreased excitement, reduced willingness to play more, and increased boredom at the 20-hour mark compared to the start. Several "retention breakdown mechanics" were identified :

— Poor manipulation (console-oriented UI causing frustration)
— Lack of stimulation (repetitive missions and enemy types)
— Weak identification (lack of avatar customization and community feeling)
— Evocation failure (ludonarrative dissonance between realistic setting and unrealistic gameplay, i.e conflict between the game's narrative told through the non-interactive elements and the narrative told through the gameplay).

Importantly, players who changed their playstyle (e.g., engaging in player-versus-player combat) maintained interest longer, suggesting that flexibility in how the game is experienced can mitigate boredom.

Replayability is closely tied to retention. Alexandrovsky et al. (2021) found that Completion mechanics, including daily login bonuses, one-off missions, and chains of successive tasks, provided players with choices on how to progress, satisfying needs for competence, autonomy, and relatedness. The Zeigarnik effect (the pull to complete unfinished tasks) (Kontinou, Georgiou, Giamarellos, & Bitha, 2025) was identified as a psychological mechanism driving replayability. Blocking mechanics (time-limited access) also fostered retention by preventing overexposure and creating anticipation for the next session.

Player Autonomy and Non-Linearity

Player autonomy is the freedom to choose how to engage with game content. It's a key factor in sustaining engagement. Stråat and Verhagen (2018) observed that participants who adopted different playstyles (e.g., shifting from story missions to player-versus-player combat) maintained their interest longer than those who followed a linear path. The game's non-linear structure allowed players to select missions in their preferred order, access different areas, and engage in cooperative or competitive play. However, the study also found that too much choice (particularly in the form of overwhelming skill trees and gear options) could backfire, causing players to disengage due to decision paralysis.

Allart et al. (2017) similarly found that in The Division (Ubisoft, 2016), players could modulate difficulty by choosing which missions to attempt and by improving their avatar's equipment. This autonomy reduced the negative motivational impact of failure, as players could attribute failure to external factors (like avatar strength) rather than internal ones (like lack of skill). In contrast, Rayman Legends (Ubisoft, 2013) offered less autonomy, players could choose between available levels but could not change a level's difficulty. Making failure more attributable to personal skill and thus more motivationally harmful, especially early in the game.

2.3 Knowledge Tracing Models

2.3.1 Learning Analytics

Learning analytics refers to the measurement, collection, analysis, and reporting of data about learners and their contexts. In educational technology, knowledge tracing (KT) is a core learning analytics task that aims to observe, represent, and quantify a student's knowledge state (e.g., the mastery level of skills underlying teaching materials) (Abdelrahman et al., 2023). KT models use sequences of student interactions (question-answer pairs) to predict future performance and diagnose knowledge gaps. This enables adaptive instruction, personalized recommendations, and early intervention for students that are trailing behind in understanding.

As Abdelrahman et al. (2023) note in their comprehensive survey, KT has evolved from simple probabilistic models to sophisticated deep learning architectures. The field has been driven by the spread of online learning platforms such as Massive Open Online Courses (MOOCs), intelligent tutoring systems, and educational games, which generate large-scale interaction data. KT is increasingly applied beyond education, including in machine teaching scenarios where a "teacher" model adaptively selects training data for a "student" model.

2.3.2 Learner Modeling

Learner modeling is the process of building computational representations of a student's knowledge, skills, and learning progress. These models typically include :

— aspects about students (prior knowledge, learning capacity)
— aspects about learning materials (difficulty, number of practices)
— aspects about the learning environment (temporal context, channel).

Traditional models like Bayesian Knowledge Tracing (BKT) represent knowledge as binary variables (learned/unlearned), while more recent models use continuous latent representations.

Meng et al. (2019) proposed Cross-Skill Bayesian Knowledge Tracing (CS-BKT), which extends standard BKT by introducing a relationship matrix between knowledge components. The key insight is that knowledge is not independent. Mastering one skill can facilitate learning another. Using data from the KDD Cup 2010, they showed that CS-BKT achieves higher prediction accuracy, AUC (The area under the curve (AUC) represents the probability that the model, if given a randomly chosen positive and negative example, will rank the positive higher than the negative), and lower RMSE compared to standard BKT. The learned relationship matrix provides interpretable insights : for example, in their study of fraction arithmetic, the influence of multiplication on division (0.78) was much stronger than the influence of the concept of fractions on addition (0.10), suggesting differential transfer effects between skills.

2.3.3 Knowledge Tracing Concepts

Bayesian Knowledge Tracing

BKT, introduced by Corbett and Anderson in 1994, is a hidden Markov model that tracks a student's binary knowledge state (learned vs. unlearned) for each skill (Abdelrahman et al., 2023). The model has four parameters per skill (see Figure 2.3 for a visualization) :

— Initial knowledge probability $P(L_0)$
— Learning transition probability P(T)
— Guess probability P(G)
— and Slip probability P(S)

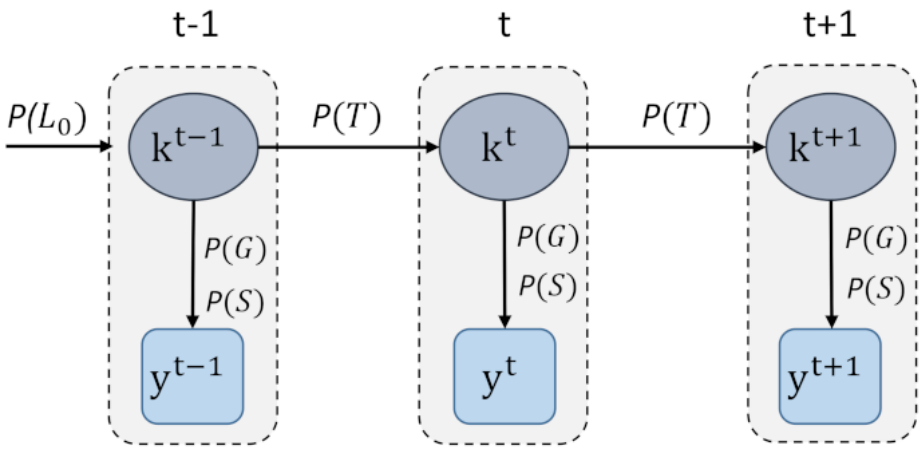


FIGURE 2.3 – The BKT model architecture, where latent variables for skills at step t are denoted as kt and their correspondingly observed variables for answers at the time step t are denoted as yt (Abdelrahman et al., 2023).

Despite its simplicity, BKT has been widely adopted and extended. Individualized variants incorporate student-specific parameters for prior knowledge or learning rate. Dynamic BKT models use Bayesian networks to capture dependencies between multiple skills. BKT's main limitations are its assumption of skill independence and its inability to handle items requiring multiple skills, but extensions like CS-BKT (Meng et al., 2019) address some of these issues.

Deep Knowledge Tracing

Deep Knowledge Tracing (DKT), introduced by Piech et al. in 2015, uses recurrent neural networks (RNNs/LSTMs) to model student learning as a sequence prediction problem (Abdelrahman et al., 2023). DKT takes as input a sequence of (question, answer) tuples and outputs the probability of correctly answering the next question.

DKT outperformed traditional BKT on several benchmark datasets, but has limitations :

— It represents knowledge state as a single hidden vector
— It cannot explicitly model relationships between multiple knowledge components
— and it treats all questions as equally related.

Extensions include memory-augmented models (DKVMN, SKVMN) that use key-value memory to track mastery levels of individual skills, attention-based models (SAKT, AKT, SAINT) that weigh the importance of past interactions, and graph-based models (GKT, GIKT) that capture dependencies between knowledge components (Abdelrahman et al., 2023).

Knowledge Tracing Machines

Knowledge Tracing Machines (KTMs) (Vie & Kashima, 2019), provide a unifying framework that includes several existing models as special cases. KTMs are based on factorization machines (FMs), a regression/classification model that learns pairwise interactions between features. The authors showed that with appropriate feature encoding, KTMs can become equivalent to existing models, such as :

— Item Response Theory (IRT) : when encoding users and items with one-hot vectors and setting the latent dimension d=0
— Additive Factor Model (AFM) : when encoding skills and attempt counts with d=0
— Performance Factor Analysis (PFA) : when encoding skills, wins, and fails with d=0
— Multidimensional Item Response Theory (MIRT) : when using d 0 with user and item embeddings

KTMs offer several advantages : they handle multiple knowledge components per item, incorporate side information (e.g., school ID, teacher ID, tutor mode), scale to high dimensions (up to d=20), and are computationally efficient (logistic regression training in minutes vs. hours for DKT). On the Assistments dataset, KTM with item, skill, win/fail, and extra features at d=0 achieved AUC of 0.815. Thus outperforming vanilla DKT (0.743) while training much faster (Vie & Kashima, 2019). Importantly, KTMs learn interpretable embeddings that can be visualized to provide feedback to students.

Abdelrahman et al. (2023) note that KTMs represent a bridge between traditional factor analysis models and modern deep learning, offering a testbed for trying new feature combinations. Recent extensions include KTM-DLF, which incorporates forgetting behavior through time-lapse features. The key finding from KTM research is that side information (item difficulty, skill tags, attempt counts) improves predictions more than increasing latent dimensions.

2.4 Discussion and Synthesis

The parallel study of engagement in games and knowledge tracing in education reveals several convergences. Both domains are fundamentally concerned with modeling time-sensitive dynamics : how a player's interest or a student's knowledge evolves over time, and what factors influence this trajectory. The methodological approaches also overlap, survival analysis is used both for player churn prediction (Alexandrovsky et al., 2021 ; Allart et al., 2017) and for understanding student attenuation in e-learning.

Flow theory provides a bridge between the two domains. In games, flow is the mechanism through which challenge, skill, and clear goals produce enjoyable, immersive experiences. In education, the same conditions (clear goals, immediate feedback, optimal challenge) are known to facilitate deep learning. Knowledge tracing models, by tracking which skills a student has mastered, enable adaptive systems to maintain optimal challenge levels, thus supporting flow-like states in learning.

A key insight from the literature is that more is not always better. In games, excessive complexity (too many skills, overwhelming gear options) can cause disengagement (Stråat

& Verhagen, 2018). In knowledge tracing, adding more latent dimensions to models yields diminishing returns compared to leveraging side information (Vie & Kashima, 2019).

The "snacking" framework (Alexandrovsky et al., 2021) and the longitudinal case study approach (Stråat & Verhagen, 2018) highlight the importance of studying engagement over time rather than in single sessions. Most game analytics focus on short-term metrics, but retention breakdowns often only become apparent after many hours of play. Similarly, knowledge tracing models that ignore forgetting (the temporal decay of knowledge) produce unrealistic estimates of student mastery.

One limitation of the current literature is the scarcity of studies that directly connect game design elements to knowledge tracing. Educational games that incorporate KT models could dynamically adjust difficulty, introduce new mechanics, or recommend content based on the player's demonstrated knowledge state. The integration of player modeling (engagement, motivation, player type) with learner modeling (knowledge state, learning rate) represents a promising frontier.

2.5 Conclusion

Player engagement and retention in games are driven by a complex interplay of motivation, flow, reward structures, player autonomy, and game characteristics. Flow theory remains the most influential framework for understanding optimal player experience, though empirically measuring 'flow' remains contested. Studies using longitudinal methods and survival analysis have identified specific retention breakdown mechanics (e.g., poor UI, repetitive content, weak identification) and effective retention strategies (e.g., completion mechanics, time-blocking, non-linear progression).

Knowledge tracing has evolved from simple Bayesian models to sophisticated deep learning architectures and unifying frameworks like Knowledge Tracing Machines. KTMs offer a principled way to incorporate side information and recover classic educational models as special cases. The field continues to advance with attention mechanisms, graph neural networks, and forgetting-aware models that better capture the temporal dynamics of learning.

The convergence of game analytics and learning analytics offers exciting opportunities : adaptive educational games that optimize both engagement and learning by tracking knowledge states and adjusting challenge levels in real time. Future research should explore the integration of player engagement models with knowledge tracing models, study the longitudinal effects of game design on learning outcomes, and develop standardized benchmarks for evaluating engagement-aware tutoring systems.

Chapitre 3

Games for Programming as Engaging Learning Tools

3.1 Introduction

In this chapter, we discuss some of the most well-known programming games. While not necessarily being serious games in the traditional academic sense, they have proved indispensable in teaching players to program within a gaming context. Most of them being commercial products (but all share a dedicated community of users), they provide entertainment and engagement first and foremost, an aspect key to the project at hand. Analysis of the strengths and limitations of these tools is key in determining the relevance, positioning, and design parameters of our project.

Game-based learning (in our case, games that can teach players) offer many advantages when it comes to learning programming concepts. These games provide enjoyment, hands-on practice, and increased engagement in the learning process. They also offer immediate feedback and personalization, allowing the specific needs of each player/potential learner to be addressed. As a result, learning becomes more entertaining, more interactive, and more tailored to each individual.

3.2 Existing Games for Programming

Not all programming-related games are designed as educational tools. In fact, most of the promising works were not originally designed as serious games, yet they have garnered a considerable deal of attention when it comes to academic settings (see Zachademics (Zachtronics, 2015b) for example).

These games illustrate how gameplay mechanics can naturally guide players toward algorithmic thinking, debugging, and problem decomposition. In this section, we present five representative titles that highlight different approaches to learning programming through play.

3.2.1 Robocode

Robocode is an engaging and educational programming game that allows you to code, test, and improve robot battle tanks. The game's primary aim is to teach programming skills in a fun and interactive environment (Robocode, 2001).

First released in July 2001, Robocode (see the game's logo in Figure 3.1) is still actively maintained with new releases bringing improvements and bugfixes (Robocode, 2001). As of the writing of this document, version 1.10.0 was released on the 4th of June 2025, bringing Java 24 support to the game.



FIGURE 3.1 – Robocode logo (Robocode, 2001).

The game challenges players to design and implement the artificial intelligence of virtual battle tanks using the Java programming language. Players write the code that governs movement, targeting, and decision-making, then observe their robots competing in automated battles against others (see Figure 3.2).



FIGURE 3.2 – Screen capture of an instance of Robocode gameplay (Robocode, 2001).

Robocode is suitable for all levels, from beginners taking their first steps in coding to experienced developers looking for a fun challenge (Robocode, 2001). The game has a loyal community ; the RoboWiki platform offers extensive documentation and tutorials, going over the best robots made by players, community-led online competitions, and challenges.

Starting on the 23rd of February 2022, an alpha version of Robocode Tank Royale was released. Robocode Tank Royale is a new, completely modernized version of the original game (Robocode Tank Royale, 2022). It is intended to be a better and improved version

of the original game, capable of running Robocode on any programming language (utilizing WebSockets for communications) and running battles via the Internet. Note that any programming language can be used as long as it can establish WebSocket connections (Robocode Tank Royale, 2022).

Robocode is an apt example of an ongoing effort to gamify the learning of not only programming syntax but also intelligent agent behavior and physics vector calculations.

3.2.2 TIS-100

TIS-100 is a puzzle game developed by Zachtronics that challenges players to program using a fictional assembly language. The game's primary aim is to teach low-level programming concepts and computer architecture principles through engaging, parallel puzzle-solving mechanics.

Released in July 2015 on Steam, TIS-100 has maintained a dedicated following among programmers and puzzle enthusiasts. The game continues to be available on multiple platforms and has become a reference in programming education for its unique approach to teaching assembly-like programming.

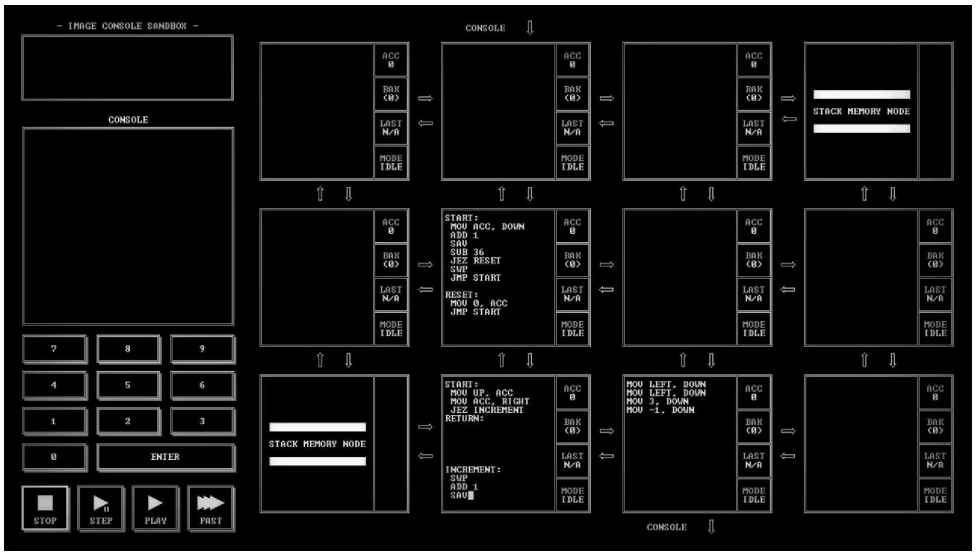


FIGURE 3.3 – TIS-100 game interface (Zachtronics, 2015a).

The game (see Figure 3.3) challenges players to repair corrupted code segments in a fictional computer system. Players work within a grid of interconnected execution nodes, each capable of running simple assembly-like instructions. Using a limited instruction set of only 15 commands, players must manipulate registers, coordinate data transfer between adjacent nodes, and optimize their solutions for speed and instruction efficiency.

TIS-100 is suitable for intermediate to advanced programmers looking to deepen their understanding of computer architecture (Zachtronics, 2015a). The game features a minimalist aesthetic that mimics 1980s technical documentation, complete with a comprehensive in-game manual. Players face strict constraints : each node can hold a maximum of 15 lines of code, and data must be carefully passed between nodes to solve increasingly

complex parallel puzzles.

The game has found extensive use in academic settings as a tool for teaching parallel processing, optimization techniques, and algorithmic thinking at the hardware level. Its puzzle-based structure promotes systematic problem decomposition and encourages players to reason about performance trade-offs—skills directly applicable to real-world systems programming.

TIS-100 is made available for free to public schools and school-like non-profit organizations through the Zachademics initiative (Zachtronics, 2015b).

TIS-100 represents a successful example of how game mechanics can make traditionally challenging concepts like assembly language accessible and engaging, particularly for learners interested in understanding how computers work at a fundamental level. The game's puzzle-based approach has been recognized as an innovative method for teaching low-level programming concepts (Vahldick, Mendes, & Marcelino, 2014).

3.2.3 Screeps

Screeps (Screeps, LLC, 2016) is a massively multiplayer online (MMO) strategy sandbox game where players program the behavior of their units through JavaScript code. The game's primary aim is to provide a persistent programming environment where players can practice real-world software development skills in a competitive, online multiplayer setting.

Launched in November 2016 by Screeps LLC (Screeps, LLC, 2016), the game continues to be hosted on official servers and has an active player base. Screeps is unique in that the game world operates continuously (player scripts run 24/7 on the server), even when players are offline, making it a true persistent programming environment.

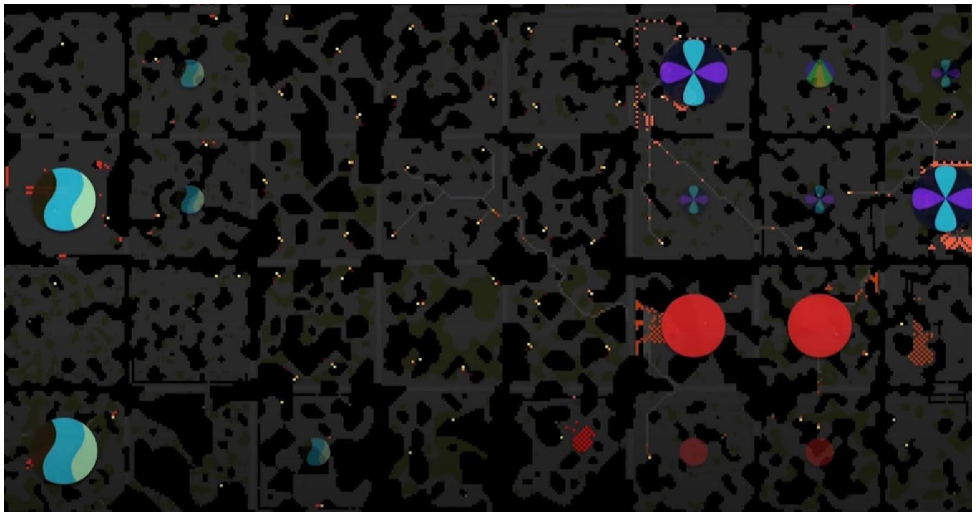


FIGURE 3.4 – Screeps game world showing multiple player colonies (Screeps, LLC, 2016).

The game (see Figure 3.4) challenges players to write JavaScript (or TypeScript) code that controls autonomous units called "creeps." Players must program AI systems capable

of resource harvesting, base construction, territory defense, and colonial expansion. All gameplay interactions occur through code (there is no manual mouse-driven player control), which transforms programming into a strategic, long-term architectural activity.

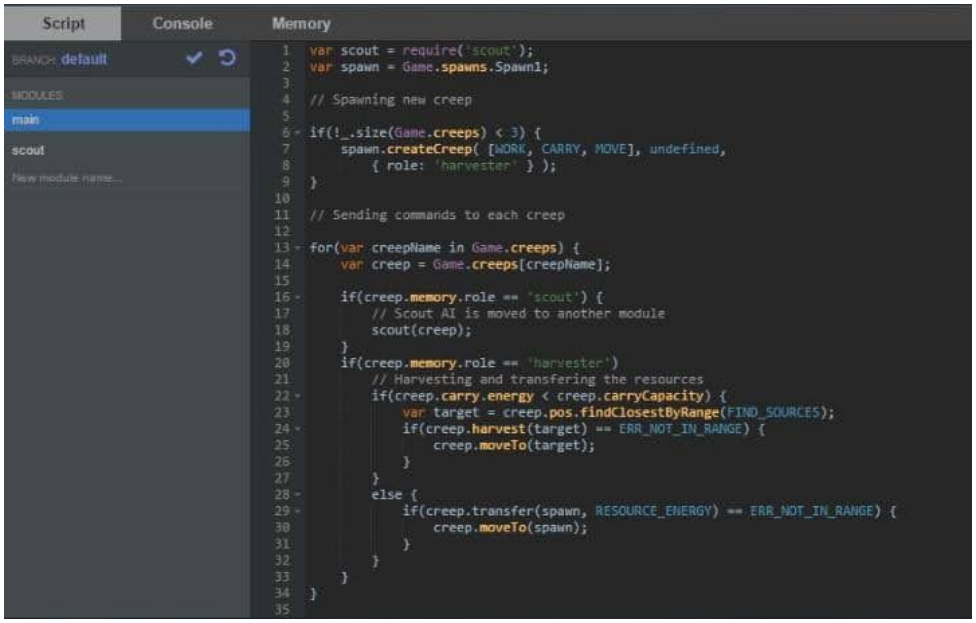


FIGURE 3.5 – Screeps code editor and API documentation (Screeps, LLC, 2016).

Screeps is suitable for intermediate to advanced programmers interested in systems design and software architecture (Screeps, LLC, 2016). The game provides a comprehensive API integrated into a browser-based development environment (see Figure 3.5). Players must manage CPU limits, memory constraints, and asynchronous operations while building modular, maintainable codebases. The persistent multiplayer nature encourages defensive programming practices and forces players to handle edge cases and unexpected scenarios.

The game has an active community with extensive documentation, shared code libraries, and regular competitions. Players collaborate and share strategies through forums, Discord servers, and open-source repositories, creating a learning environment that mirrors professional software development practices.

Screeps is a valuable tool for teaching software engineering principles in an applied context. Unlike traditional programming games that focus on syntax or basic loops, Screeps emphasizes software architecture, code quality, and long-term maintenance. These are skills that are directly transferable to real-world software development. The persistent multiplayer environment provides unique opportunities for learning collaborative programming practices (Vihavainen, Paksula, & Luukkainen, 2011).

3.2.4 SHENZHEN I/O

SHENZHEN I/O (Zachtronics, 2016) is an engineering puzzle game developed by Zachtronics that combines circuit design with low-level programming. The game's primary

aim is to teach players about embedded systems development through hands-on problem-solving that requires both hardware and software expertise.

Released in November 2016, SHENZHEN I/O is part of Zachtronics' acclaimed series of engineering games (Zachtronics, 2016). The game remains popular among programmers and engineering students for its authentic representation of embedded systems design challenges.

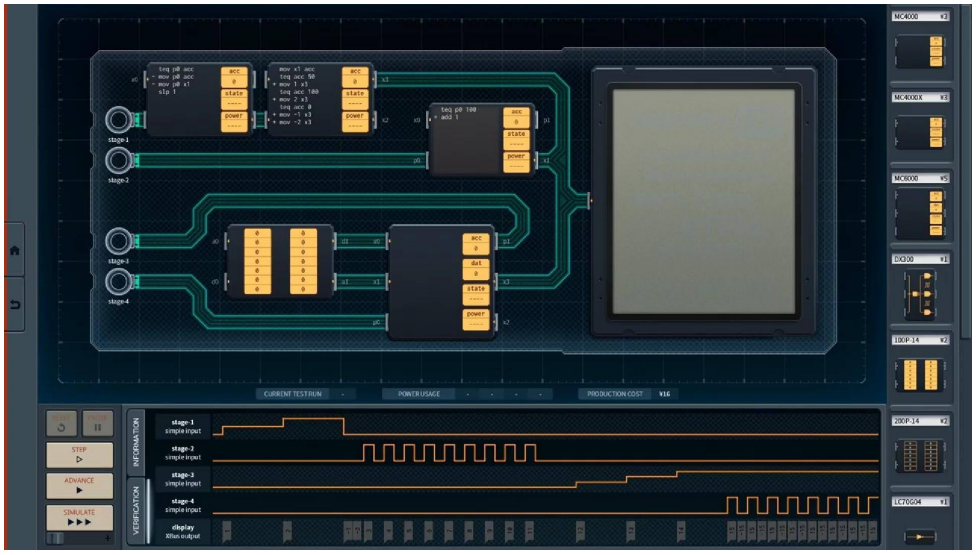


FIGURE 3.6 – SHENZHEN I/O circuit design interface (Zachtronics, 2016).

The game (see Figure 3.6) challenges players to design electronic devices by combining circuit board layout with microcontroller programming. Players take on the role of an engineer at a Shenzhen electronics company, receiving client requests for various devices. Each puzzle requires selecting and arranging hardware components (such as microcontrollers, memory chips, and logic gates) then writing assembly-like code to control their behavior.

SHENZHEN I/O is suitable for players with prior programming experience interested in hardware-software integration (Zachtronics, 2016). The game introduces concepts such as signal processing, state machines, timing constraints, and resource optimization. Players must balance competing constraints : component costs are limited by a budget, code space is restricted (typically 9 to 14 lines of code per microcontroller), and execution timing must meet specifications. This encourages elegant, minimalist solutions that mirror real embedded systems work.

The game includes a comprehensive PDF manual styled as authentic technical documentation, covering all components, instruction sets, and design principles. This manual serves as both an in-game reference and a learning resource that reinforces the game's educational goals.

This game is likewise available for free to public schools and school-like non-profit organizations through the Zachademics program (Zachtronics, 2015b).

SHENZHEN I/O is an effective tool for introducing embedded systems concepts and hardware-software co-design. The game's emphasis on optimization, trade-offs, and constraint satisfaction provides valuable lessons in engineering decision-making that

extend beyond programming to broader systems thinking.

3.2.5 The Farmer Was Replaced

The Farmer Was Replaced is a Python-based programming puzzle game developed and published independently (Koster, 2024b), with its primary aim being to teach algorithmic thinking and programming optimization through farm automation gameplay. While not originally designed as an academic serious game, the developers claim that it requires no prerequisite programming knowledge to play and that it is suitable for learning Python (Koster, 2024b).

Released in 2024 on Steam, the game has quickly built a highly positive reception and an active community of players who share solutions and optimization strategies (Koster, 2024a). It remains actively maintained and continues to attract both hobbyist programmers and learners looking for accessible Python learning tools, having reached a peak of 7,436 active players, and comfortably maintaining over 200 active players at all times (SteamDB, 2024).

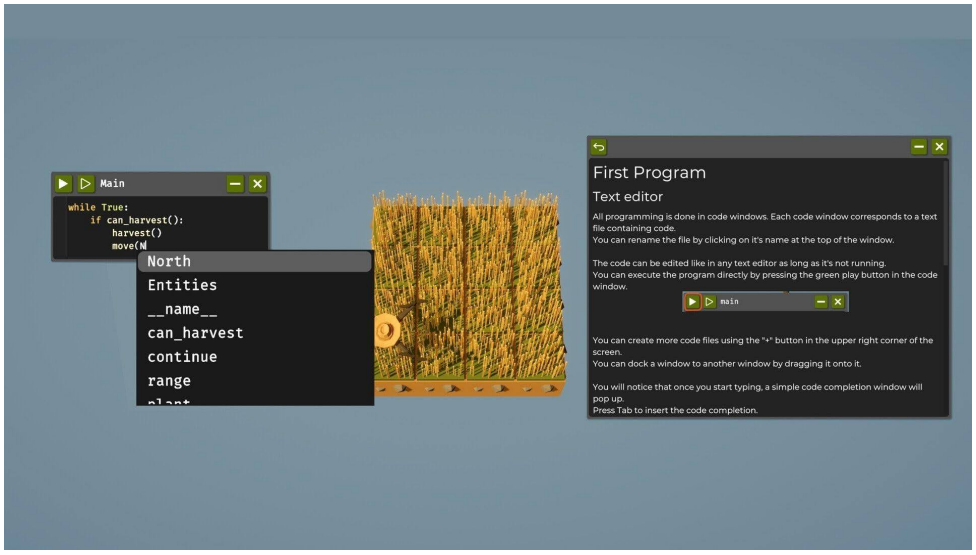


FIGURE 3.7 – The Farmer Was Replaced gameplay showing the start of the game tutorial (Koster, 2024b).

The game (see Figure 3.7) challenges players to write Python code controlling an autonomous drone tasked with managing a farm. Players automate tasks such as planting, watering, and harvesting crops across a grid-based environment. Beginning with simple sequential operations like harvesting hay, the game progressively introduces more demanding challenges requiring pathfinding algorithms, resource management systems, and parallel task execution (see Figure 3.8).

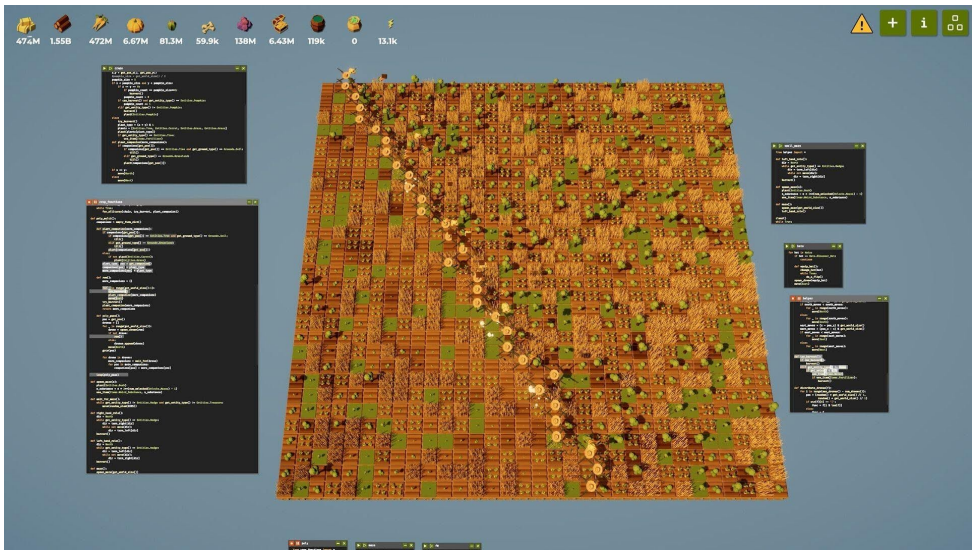


FIGURE 3.8 – The Farmer Was Replaced gameplay showing the code editor and farm grid (Koster, 2024b).

The game is suitable for players with basic familiarity with Python, as it assumes foundational knowledge rather than teaching syntax from scratch (Koster, 2024b). Its technology progression system unlocks new crops, drone capabilities, and programming constructs over time, encouraging iterative refinement of solutions (see Figure 3.9). Players are rewarded for optimizing their code, creating a natural incentive to reason about algorithmic efficiency and performance trade-offs.



FIGURE 3.9 – Technology progression tree showing unlockable features (Koster, 2024b).

While the game does not explicitly teach object-oriented programming theory, it naturally exposes players to several OOP-adjacent concepts through practical application. The drone object encapsulates its state and behaviors behind well-defined methods, encouraging players to build custom abstractions such as `harvestField()` or `plantOptimalCrop()`. Advanced levels reward modular, reusable code structures—a principle central to both procedural and object-oriented design.

The game's commercial success on Steam and its active solution-sharing community (Flekay, 2026) suggest strong intrinsic motivation among its player base. Its immediate visual feedback, open-ended challenges, and scalable difficulty make it a compelling informal learning tool, though the absence of explicit instruction or formal assessment mechanisms distinguishes it from research-backed educational software.

3.3 Comparison of the Games

The comparison of the selected programming games highlights both their diversity and the range of pedagogical strategies they employ. Although all of them allow players to develop programming-related skills, their design philosophies and educational reach differ significantly. This diversity is visible when examining their languages, gameplay mechanics, levels of abstraction, and conceptual coverage.

All selected games rely on textual programming languages but vary in complexity and target skill level (see Table I). TIS-100 and SHENZHEN I/O emphasize low-level, assembly-like programming, offering strict constraints that encourage precision and optimization. Screeps, by contrast, immerses players in a real-world JavaScript environment, enabling more expressive programming and long-term software architecture design. Robocode and The Farmer Was Replaced occupy a different space by teaching higher-level reasoning through object-oriented structures (Robocode indirectly through Java APIs, The Farmer Was Replaced directly through the use of Python). The presence of 3D graphics in The Farmer Was Replaced further differentiates it from the rest.

Table I – Classification of games according to programming language, genre, platform and type of graphics

Game	**Language**	**Genre**	**Platform**	**Graphics**
Robocode	Java	Puzzle, Simulation	Desktop	2D
TIS-100	Assembly	Puzzle	Desktop	2D
Screeps	JavaScript	MMO	Desktop	2D
SHENZHEN I/O	Assembly	Puzzle, Automation	Desktop	2D
The Farmer Was Replaced	Python	Automation, Idler	Desktop	3D

Table II highlights some key differences between the aforementioned games. Commercial games (Screeps, SHENZHEN I/O, TIS-100, The Farmer Was Replaced) are highly polished and engaging, but were not originally designed with structured pedagogical progression. Their learning is emergent rather than guided. In contrast, academic serious games implement explicit pedagogical intentions, including targeted learning outcomes, structured chapters, and gradual conceptual progression. However, academic games often fall short in terms of long-term player retention, aesthetic appeal, and overall production value—factors that strongly influence engagement. This is due in part to the motivating nature of some of the commercial games, be they story-driven or competitive. Simply put, the presence of stories in games can motivate players intrinsically to finish them while competition provides extrinsic rewards based on the players' rankings ; both intrinsic and extrinsic motivators serve to maximize playing time and thus potential learning time.

Table II – Comparative table of characteristics per game

Game	Commercial	Singleplayer	Multiplayer	Competitive	Story-driven	Academic
Robocode		Yes		Yes		
TIS-100	Yes	Yes			Yes*	Yes**
Screeps	Yes		Yes	Yes		
SHENZHEN I/O	Yes	Yes			Yes	Yes**
The Farmer Was Replaced	Yes	Yes				

* : Some elements included, not main focus
** : Academic offer exists, was not originally planned

The ACM CS2013 mapping (ACM/IEEE-CS Joint Task Force, 2013) (see Figure 3.10) and the detailed conceptual classification (see Table III) show that each game covers a specific subset of computer science concepts :

— Robocode supports algorithmic reasoning, event-driven programming, and basic OOP interactions, but generally lacks deep structural abstraction.
— TIS-100 excels at low-level operations, logic reasoning, and optimization under constraints.
— Screeps stands out for its coverage of high-level design concepts, including modularity, software architecture, asynchronous logic, and maintainability over time.
— SHENZHEN I/O bridges hardware and software reasoning, promoting systems thinking, debugging, and trade-off analysis.
— The Farmer Was Replaced covers the most computer science concepts among the mentioned games, but mostly through the direct use of Python as the programming language.

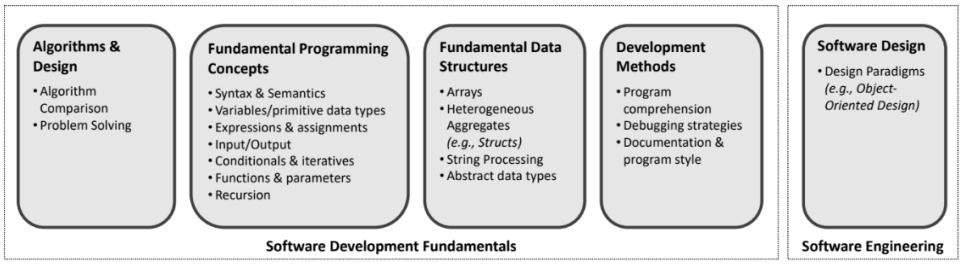


Figure 3.10 – ACM Computer Science Curricula 2013 knowledge areas (ACM/IEEE-CS Joint Task Force, 2013).

TABLE III – Classification of games according to educational content

Game	Alg. Comp.	Problem Solv.	Non-spec. Prog.	Syntax & Sem.	Variables	Expressions
Robocode			Yes	Yes*		
TIS-100		Yes	Yes	Yes	Yes	Yes
Screeps				Yes*	Yes*	Yes*
SHENZHEN I/O		Yes	Yes	Yes	Yes	Yes
The Farmer Was Replaced	Yes*	Yes	Yes	Yes	Yes	Yes

Game	I/O	Conditionals	Functions	Recursion	Arrays	Heterogeneous
Robocode						Yes*
TIS-100	Yes	Yes			Yes	
Screeps		Yes*	Yes*		Yes*	Yes*
SHENZHEN I/O	Yes	Yes	Yes*		Yes	
The Farmer Was Replaced	Yes	Yes	Yes	Yes	Yes	

Game	String Proc.	Abstract Data	Program Comp.	Debugging	Doc. & Style	OO Design
Robocode		Yes*				Yes
TIS-100			Yes	Yes		
Screeps		Yes*				Yes
SHENZHEN I/O	Yes		Yes			
The Farmer Was Replaced		Yes*	Yes	Yes	Yes*	Yes*

* : Some elements included, not main focus

This conceptual comparison reveals a major gap in the existing landscape : very few games, especially popular, high-quality ones, address advanced topics in object-oriented design. Most games stop at procedural or introductory OOP.

3.4 Conclusion

This chapter provided a comprehensive overview of the existing landscape of programming games. The examined titles demonstrate that games can effectively support the development of many programming competencies, from low-level parallel node operations to high-level asynchronous software engineering. However, most widely played commercial titles focus on introductory concepts, procedural thinking, or hardware-constraint satisfaction. Only a few, if any, address the advanced object-oriented principles and design patterns central to modern software development.

In conclusion, the existing landscape confirms the relevance, academic value, and pedagogical importance of a game centered around advanced object-oriented design. The findings of this chapter reinforce the significance of our project and provide a solid conceptual foundation for the development and enhancement efforts of Refactal that follow.

Deuxième partie

Contribution

Chapitre 4

Background Analysis

4.1 Introduction

This chapter provides a detailed technical audit of Refactal, a serious game for teaching advanced object-oriented programming concepts. By reviewing the design choices, gameplay mechanics, and player data collection of prior iterations, we identify critical usability bottlenecks, learning challenges, and tracking limitations. This assessment serves to validate the design requirements for the redesigned version of Refactal, establishing a clear link between player performance data and our subsequent software engineering contributions.

4.2 Presentation of Refactal

Refactal is a 3D educational coding puzzle game (see Figure4.1 for a screen capture of the game) aiming to teach players some advanced principles of object-oriented programming (OOP) and software design (Morsli, 2022). The game is divided into five thematic chapters, each containing several levels. Most levels focus on a specific design pattern or software quality principle, where the player is presented with a puzzle in the form of partially completed code that they must repair and compile to progress through the environment (Morsli, 2022).

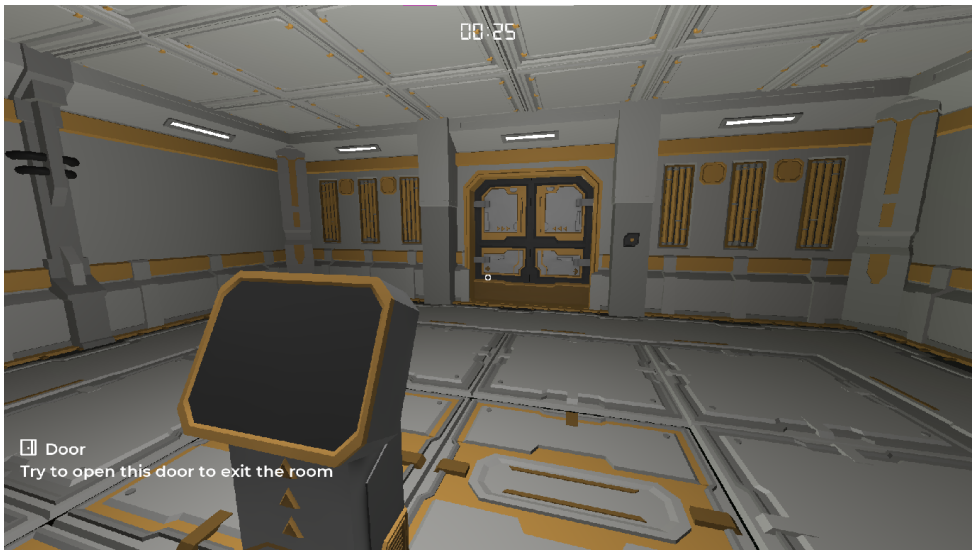


FIGURE 4.1 – Screen Capture of Refactal (Mouheb, 2023).

The first chapter serves as a tutorial, containing four learning levels and a final practice level to test the player's grasp of the baseline mechanics. The second chapter introduces the SOLID design principles. The third chapter addresses the GRASP design patterns. The fourth chapter covers Code Smells, and the fifth chapter covers Gang of Four (GoF) design patterns, ending on a final level that resolves the narrative arc of the game.

4.2.1 Storyline

The narrative of Refactal takes place aboard the space station *Orbite*. The station's computer system has malfunctioned, and remote control systems have failed. The player, acting as a cyber-architect/engineer, is dispatched to travel to the station and reactivate the system manually.

As the player navigates the space station to reach the main control room, they encounter software bugs in the station's source code, caused by the poor programming habits of the developers who built the station. The player must refactor and complete these code segments, applying correct OOP practices and software quality principles to unlock the doors and advance through the rooms. The levels follow an "escape room" structure (Morsli, 2022), where each room contains a computer terminal linked to the objects in that room.

4.2.2 Refactal Versions

Since it's first release in 2022, Refactal has gone through many changes. New levels were added and some additional functionalities were implemented.

The following serves as a general chronological overview of the evolution of Refactal, as of the writing of this document (2026).

Refactal Version 1.0

The initial version of Refactal (see Figure4.2 for visualization) was developed by Reda Morsli during the 2021/2022 academic year (Morsli, 2022). The game featured five chapters :

- **Chapter 1 :** A tutorial chapter with four learning levels and one practice level.
- **Chapter 2 (SOLID Principles) :** Containing the Single Responsibility Principle (SRP) level.
- **Chapter 3 (GRASP Design Patterns) :** Containing the Controller level.
- **Chapter 4 (Code Smells) :** Containing the Code Duplication level.
- **Chapter 5 (GoF Design Patterns) :** Containing the Observer level.

The player then finishes by playing a final level wrapping the storyline of the game.

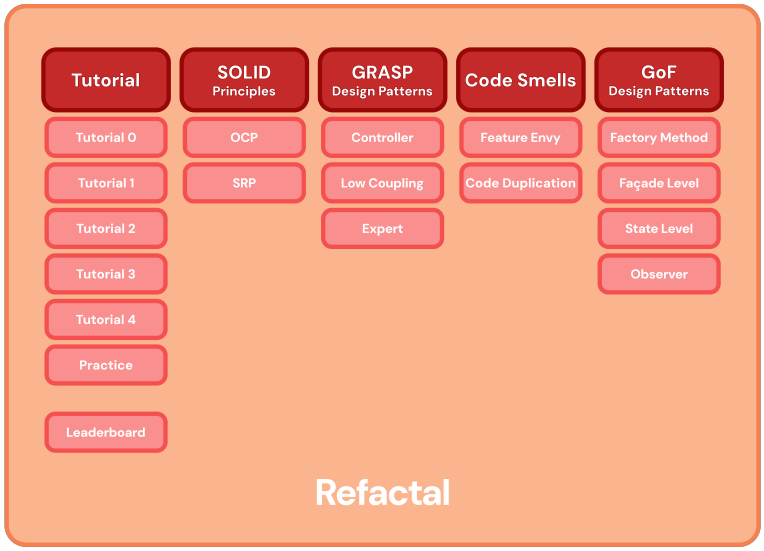


FIGURE 4.2 – General Composition of Refactal version 1.0 (Morsli, 2022).

To create a sense of competition among players, a global scoreboard was implemented to rank them based on their results (see Figure 4.3) (Morsli, 2022). The scoreboard was implemented using the SilentWolf backend services (Morsli, 2022).



FIGURE 4.3 – Ranking of players based on their scores (Morsli, 2022).

Refactal Version 2.0

Developed by Khaoula Mouheb in the 2022/2023 academic year, version 2.0 expanded the game's scope by adding new levels and user tools (Mouheb, 2023). While the first chapter remains unchanged, more levels were added to the other chapters :

— **Chapter 2 (SOLID Principles) :** Added the Open-Closed Principle (OCP) level.
— **Chapter 3 (GRASP Design Patterns) :** Added the Low Coupling and Expert levels.
— **Chapter 4 (Code Smells) :** Added the Feature Envy level.
— **Chapter 5 (GoF Design Patterns) :** Added the Factory Method, Façade, and State levels.

Alongside the addition of the aforementioned levels, a custom level editor called *Refactal Teachers* was developed. This standalone software allowed teachers to design custom levels with unique puzzles, focusing on OOP concepts not present in the base game, and share them with students (Mouheb, 2023).

Additionally, a new user data storage solution was implemented. Utilising the same SilentWolf backend services that the scoreboard uses (Morsli, 2022), player data (such as level entry time, level exit time, lines coded in a level,... etc) was collected and stored (Mouheb, 2023), albeit in a hard to retrieve, near-impossible to analyse way (JSON files only accessible through the SilentWolf website, through manual installation).

The general composition of Refactal version 2.0 is illustrated below (see Figure 4.4), new elements added in this version are pictured in red while pre-existing ones are blue.

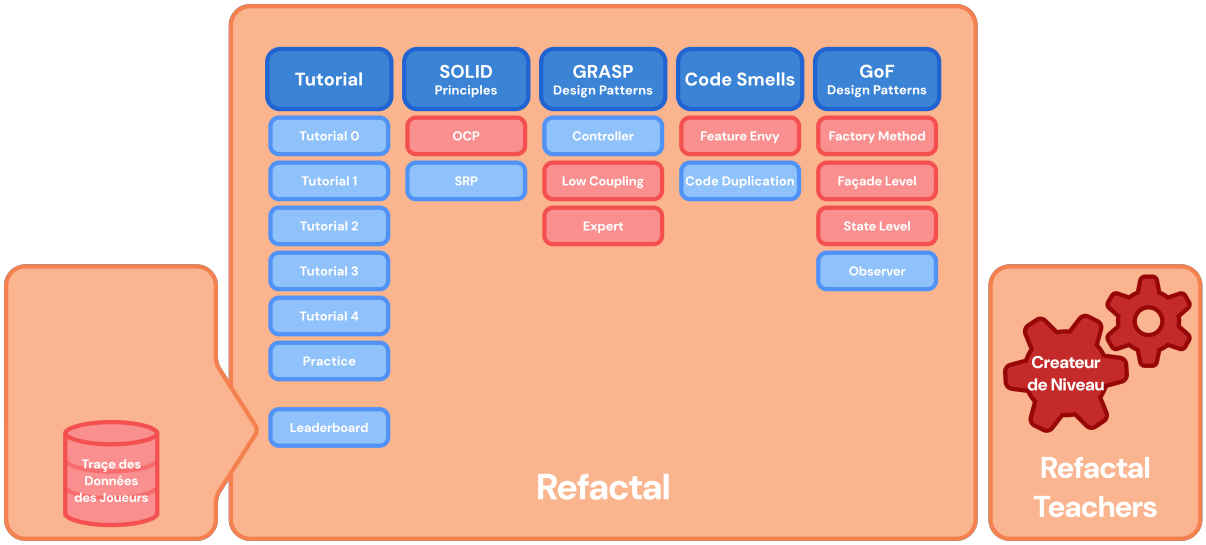


FIGURE 4.4 – General Composition of Refactal version 2.0 (Mouheb, 2023), (New additions in red, already existing parts in blue).

Related Works

During the summer of 2024, two key additions were made to Refactal as part of practical internships. Ayyoub Kasmi and Aymen Makhloufi developed a lazy-loader for the

web version of the game, reducing initial loading times by loading only one level at a time (Makhloufi & Kasmi, 2024). Simultaneously, Abdellah Arrouch and Mahdi Mohammed Tirouche developed an administrative dashboard (Arrouche & Tirouche, 2024). This dashboard allowed administrators to access, process, and visualize collected player data.

The general composition of Refactal after the aforementioned modifications is illustrated below (see Figure 4.5).

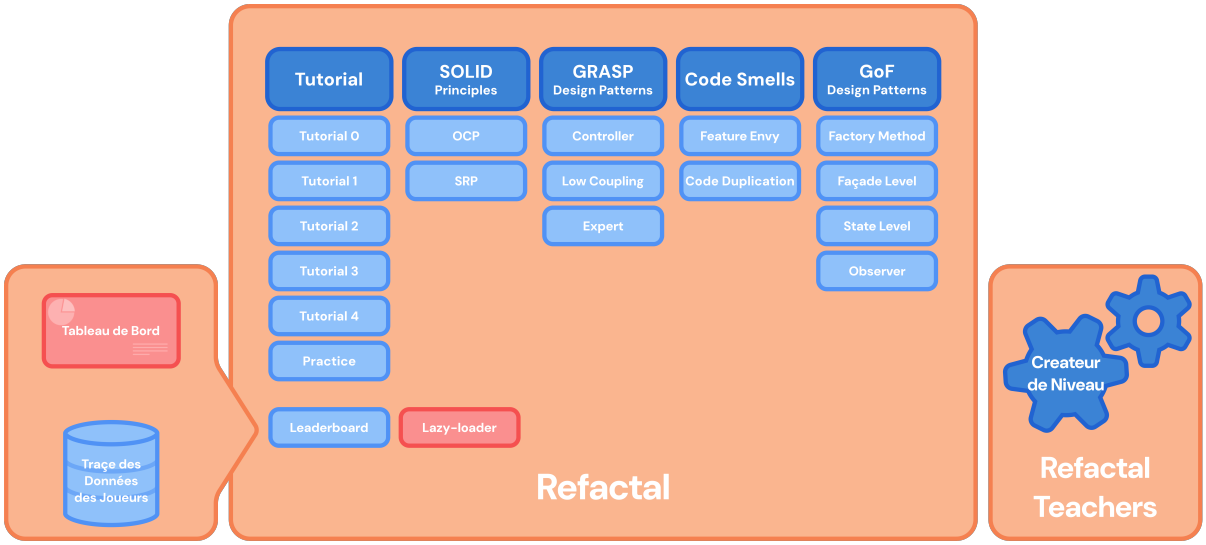


FIGURE 4.5 – General Composition of Refactal version 2.0 with complimentary additions (Makhloufi & Kasmi, 2024; Arrouche & Tirouche, 2024), (New additions in red, already existing parts in blue).

Refactal Version 3.0

The third version of Refactal was developed by Zakaria Hamdane during the 2024/2025 academic year (Hamdane, 2025), and is the most up-to-date version as of the writing of this document (2026) (see Figure 4.6 for the current composition of Refactal). This version integrated an AI-based assistant chatbot designed to support players in solving puzzles, answering questions, and maintaining engagement.

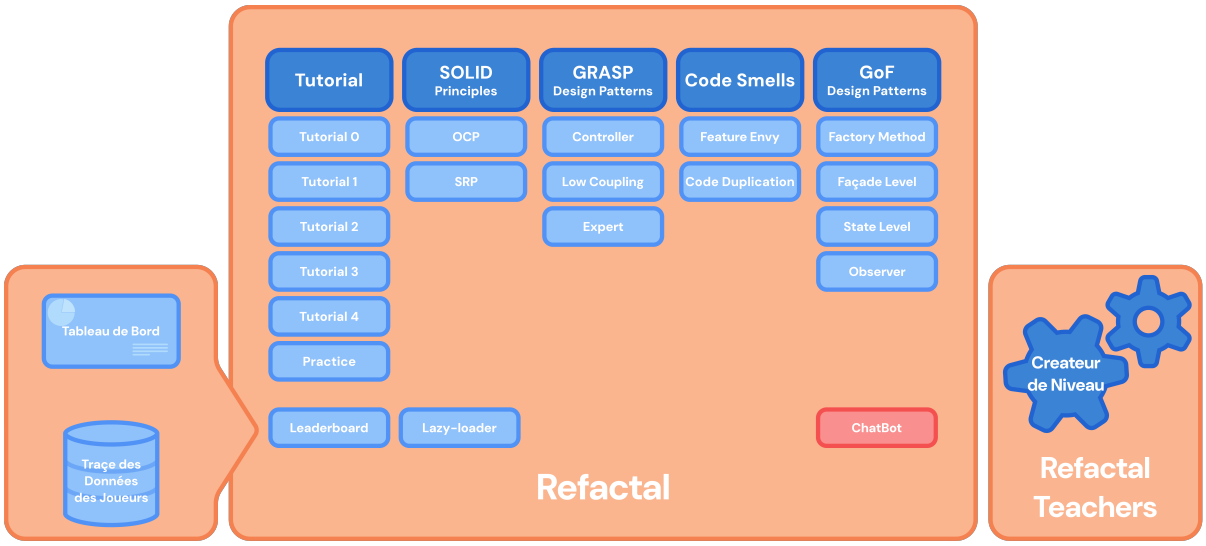


FIGURE 4.6 – General Composition of Refactal version 3.0 (Hamdane, 2025), (New additions in red, already existing parts in blue).

4.3 Educational Objectives of Refactal

The aim of Refactal is to teach users good practices in object-oriented programming (Morsli, 2022). Morsli states that, therefore, users are required to already possess basic programming knowledge, alongside knowing the fundamentals of the OOP paradigm. The user profiles targeted by Refactal are therefore :

— Computer science students who have already learned the fundamentals of algorithms and object-oriented programming.

— Programmers who want to practice or learn good programming habits/practices.

When it comes to the targeted learning concepts (detailed above in 4.2.2, per game version), the objective is first to provide theoretical knowledge related to these practices (Morsli, 2022). Then, the player should be able to practice and assess their understanding through an engaging game scenario.

4.4 Gameplay Overview

Refactal is structured around 3D puzzle environments using an escape-room format. The player controls a character in the first-person perspective, exploring rooms on the space station. Each room represents a puzzle level locked by a door (see Figure 4.7 for reference). To open the door, the player must locate a terminal computer in the room, which displays the code template associated with the door or other interactive objects (e.g., buttons, displays, or door-locking mechanismes).

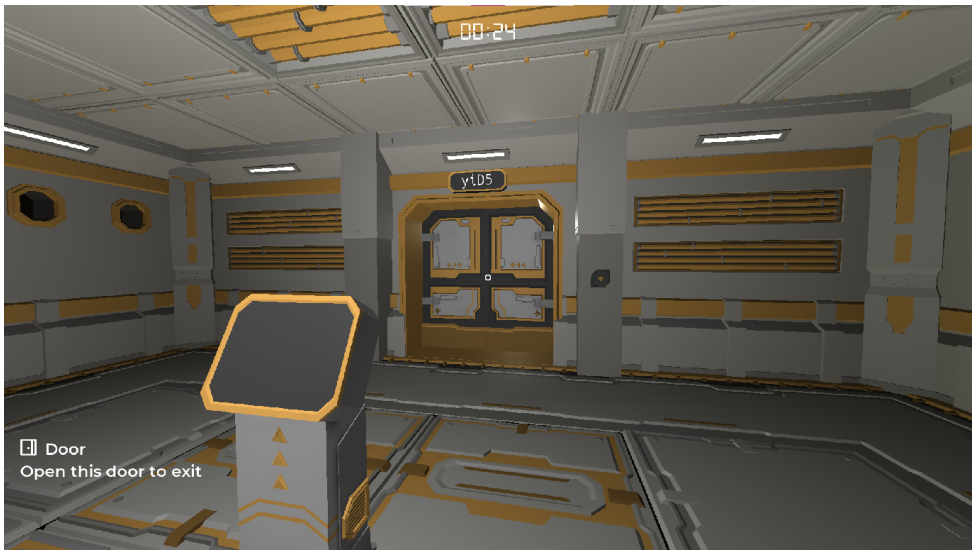


FIGURE 4.7 – Screen Capture of Refactal version 2.0 (Mouheb, 2023), showing the Tutorial 2 level.

The player modifies the code directly in the terminal editor (as Figure 4.8 illustrates) and hits an "Apply" button, applying the changes the player has made to the code. The game runs static compile checks and executes dynamic assertions to test if the player's modifications satisfy the requirements. Once the assertions pass, the door opens, and the player can proceed to the next room.

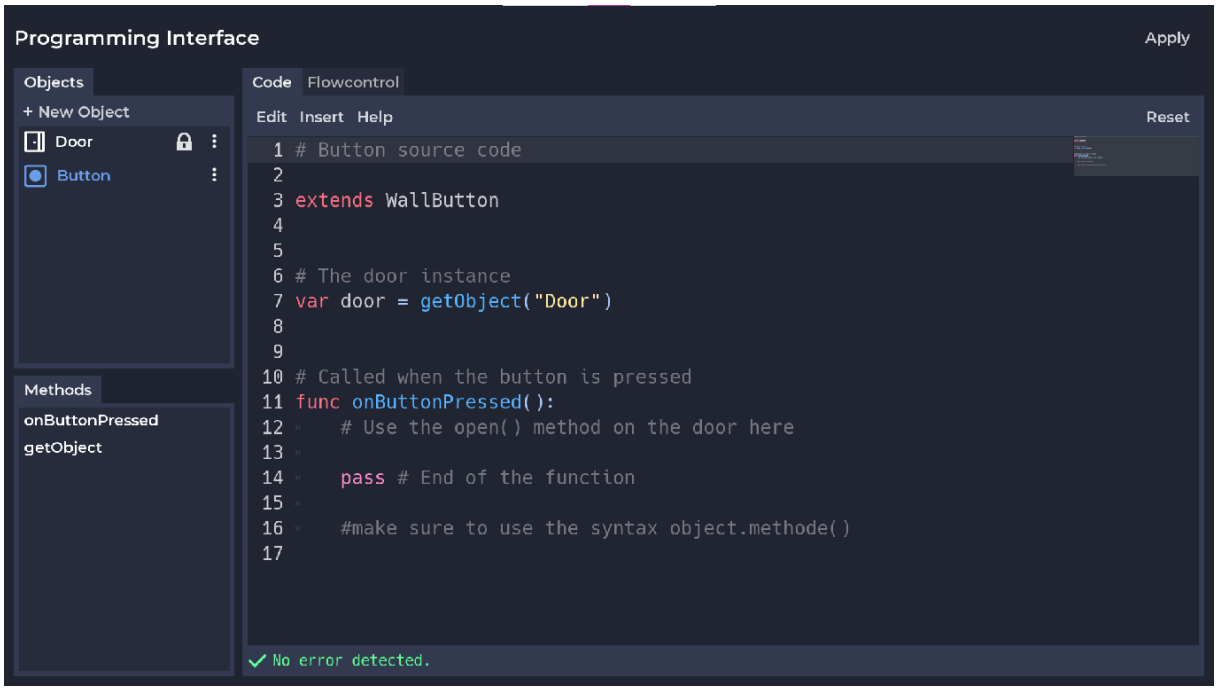


FIGURE 4.8 – Screen Capture of Refactal version 2.0 (Mouheb, 2023), showing the code editor with example partial code.

4.5 Existing Mechanics and Progression Systems

The progression system in Refactal is strictly linear. Players must complete levels in a predefined order, moving from the tutorial chapter through SOLID, GRASP, Code Smells, and GoF patterns.

The core mechanics of the game include :

— **Terminal Interaction :** Interacting with in-game terminal computers opens a code editor panel showing code templates for the objects present in the level.

— **Compilation and Verification :** Clicking "Apply" builds the code in the game engine environment and runs assertions to check logic.

— **Hint Slides :** If players struggle, they can open slides containing hints (see Figure 4.9 for an an example).

— **Score Calculation :** Scores are calculated and sent to the SilentWolf backend.

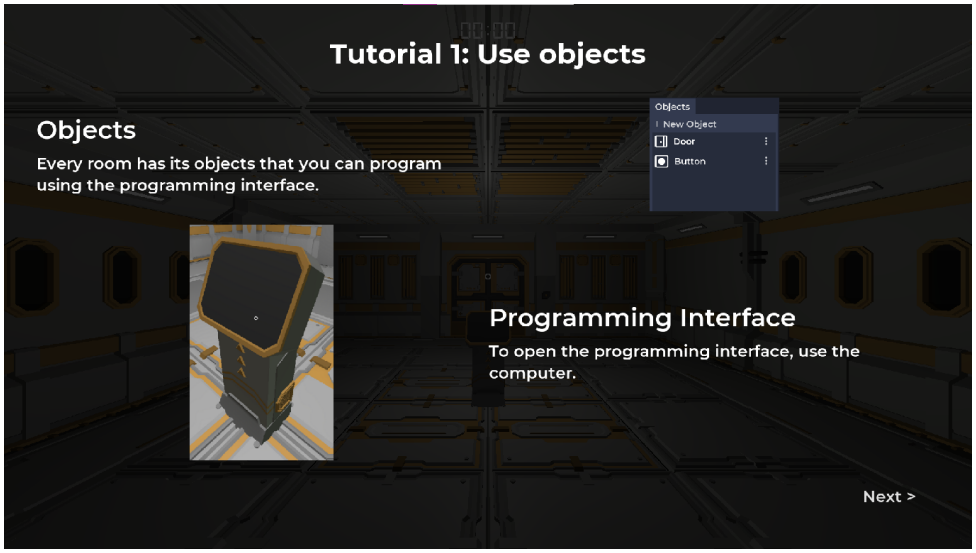


FIGURE 4.9 – Screen Capture of Refactal version 2.0 (Mouheb, 2023), showing one of the hint slides in the Tutorial 1 level.

The metrics used for scoring include the total time taken to complete the level, the number of lines of code written by the player and the cyclomatic complexity of the code (Cyclomatic complexity is a quantitative measure of the number of linearly independent paths through a program's source code (McCabe, 1976)). A bonus (of 100 points) or a penalty (of 50 points) is afforded to the player based on their answer on a quiz question at the end of *some* levels (those levels being the **SRP, Controller, Low Coupling, Expert, Feature Envy, Code Duplication, Factory Method, Façade, State,** and the **Observer** levels). The scoring formula is as follows :

$$\text{Score} = \frac{1000}{\begin{aligned}&1 + \mathit{time_taken_in_seconds} \times \tfrac{0.5}{360} + (\#\mathit{lines_of_code} + 10) \times \tfrac{0.25}{11} + (\mathit{cyclomatic_complexity} - 1) \times \tfrac{0.25}{10} \\ &+ \mathit{correct_quiz_answer} \times 100 \\ &+ \mathit{false_quiz_answer} \times (-50)\end{aligned}} \tag{4.1}$$

$$\mathit{correct_quiz_answer} \in \{0, 1\}, \quad \mathit{false_quiz_answer} \in \{0, 1\}$$

4.6 Dashboard and Player Data Analysis

The administrative dashboard designed by Arrouche and Tirouche (Arrouche & Tirouche, 2024) represents the first attempt to visualize and query player data. The data pipeline for the dashboard can be summarized as follows :

- **Player Data Tracking through SilentWolf :** The game client tracks player actions (level entry/exit, hint usage, quiz responses) and sends them to SilentWolf to be saved.
- **Extracting Player Data from SilentWolf :** The data is then manually extracted from SilentWolf in a JSON format, as one file containing all player data.
- **Transforming and LoadingPlayer Data :** The extracted data is transformed, filling in missing events (see Figure 4.10 for an example) and loaded in a PostgreSQL database.
- **BI and Visualization :** Metabase is used to query the PostgreSQL database and build dashboards. Metabase is deployed in a Docker container.

start level	Tutorial 2	12/11/2023_23:3:37
Discover mode entered	Tutorial 2	12/11/2023_23:3:45
Start coding		12/11/2023_23:4:5
Aplly pressed	Exit Coding	12/11/2023_23:6:36
Level passed	Score: 690.051754	12/11/2023_23:6:50
start level	Tutorial 3	???
Start coding		12/11/2023_23:7:18
Aplly pressed	Exit Coding	12/11/2023_23:8:50
Level passed	Score: ???	???
start level	Practice	12/11/2023_23:9:12
Start coding		12/11/2023_23:11:50
Aplly pressed	Exit Coding	12/11/2023_23:12:32
Level passed	Score: 718.132855	12/11/2023_23:12:51
Go to main menu	returned to main menu from Practice	12/11/2023_23:13:1

FIGURE 4.10 – A sample of collected player data (Arrouche & Tirouche, 2024), missing rows are indicated in yellow.

The dashboard is organized into four main tabs :

— **General Tab :** showing player counts over time and quiz performance per level, among other metrics.
— **Levels Tab :** showing detailed per-level statistics such as time spent, score distribution, hint usage, and restart counts.
— **Players Tab :** displaying per-player breakdown of scores, engagement, time played, and quiz results.
— **Engagement and KPI Tabs :** showing new vs. active player trends, a custom-made engagement metric, and drop-off rates across levels and chapters (see Figure X for reference), notably showing that for version 2.0, out of 118 players, only 10 finished the game, with most dropping off in the early levels of Tutorial and SOLID Principles chapters.

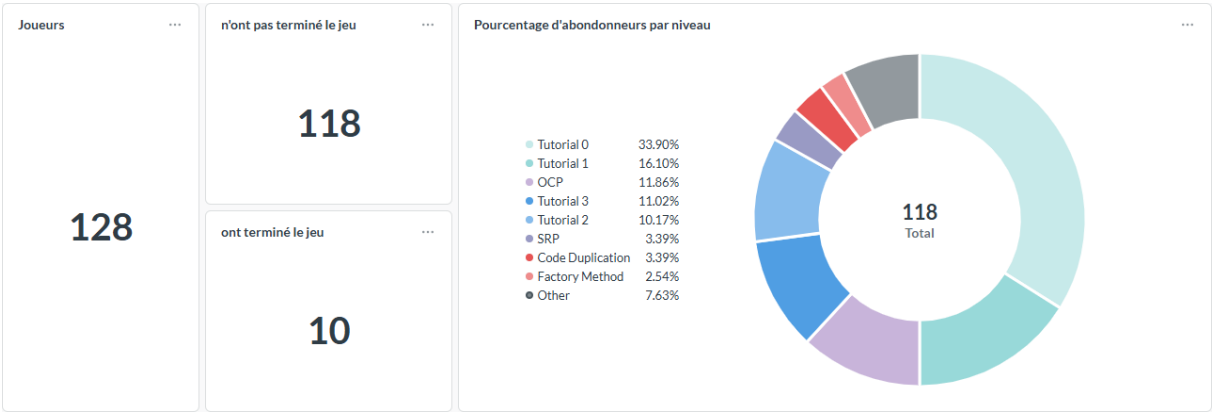


FIGURE 4.11 – A screen capture of the administrative dashboard, showing examples of charts in the Engagement tab (Arrouche & Tirouche, 2024).

4.7 Analysis

Analysis of the data collected and visualized in the administrative dashboard (Arrouche & Tirouche, 2024), combined with student feedback (see Appendix A), highlights several anomalies and usability issues in the previous versions of Refactal.

4.7.1 Analysis of Player Retention

Player data reveals severe dropout rates :

— **Initial Player Drop-off :** 34% of players quit during the very first tutorial level. Over 50% of players abandon the game before finishing the first three levels (see Figure 4.12).

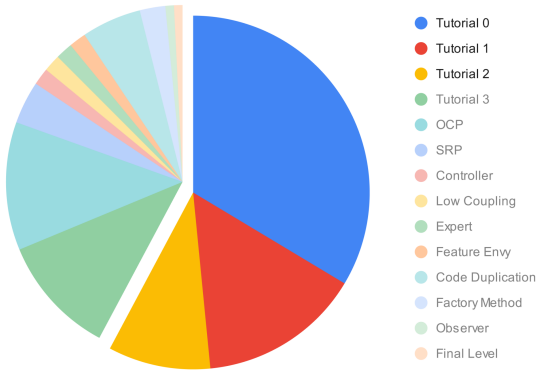
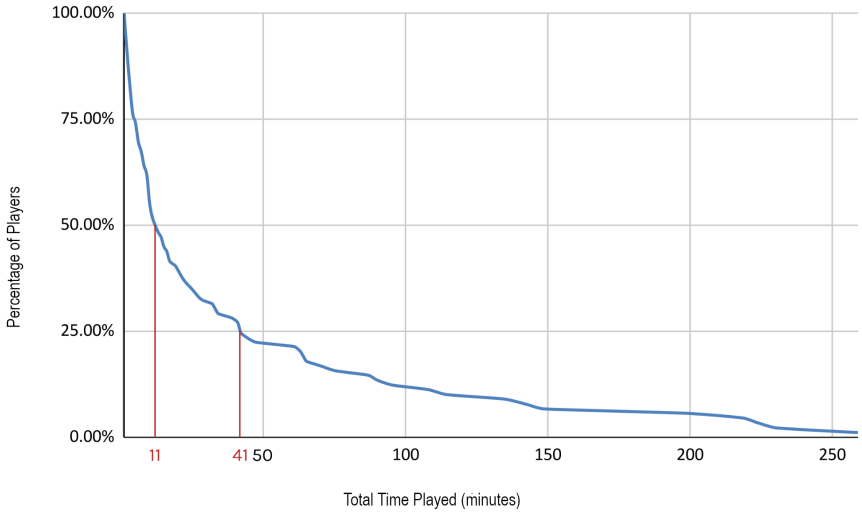
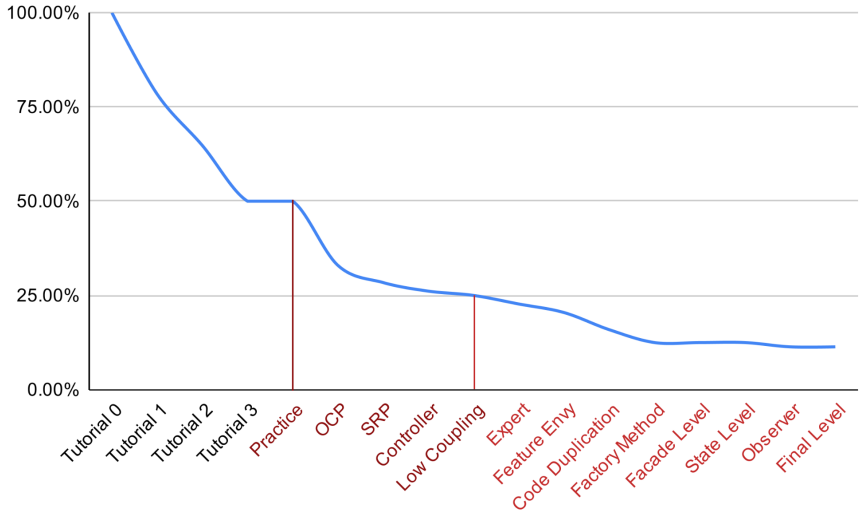


FIGURE 4.12 – Percentage pie chart of abandoning players by level (Arrouche & Tirouche, 2024).

— **Time-based Player Drop-off :** 50% of players quit within the first 11 minutes of gameplay, and only 25% remain active after 41 minutes (see Figure 4.13a).
— **Chapter-based Player Drop-off :** By the end of Chapter 2 (SOLID Principles, containing **OCP** and **SRP** levels), 83% of players have quit (see Figure 4.13b), usually within the first 33 minutes of starting the game.



(a) Retention graph of Players (%) by Total Time Played (Minutes)



(b) Retention graph of Players (%) by level.

FIGURE 4.13 – Retention graphs of players by total time played and by level (Arrouche & Tirouche, 2024).

4.7.2 Analysis of Learning Difficulties

Completion times and error rates indicate a poorly calibrated difficulty curve :

— Average completion times increase arbitrarely per level (see Appendix B, Figure B.1), indicating the abscence of an adjusted difficulty curve. Levels should get harder as the player becomes better (to remain engaged), but not *too* hard (that they become frustrated).

— Several levels(**OCP, Controller, Code Duplication,** and **State**) show exceptionally long completion times (see Appendix B, Figure B.1) and could cause high player frustration.

— The **Controller** level shows a large variance in completion times (see Appendix B, Figure B.2), indicating a wide disparity in how players grasp this pattern.
— Some quiz questions show high error rates, such as a 55% error rate in the Expert level quiz (see Appendix B, Figure B.3), often due to ambiguous or confusing questions.

4.7.3 Analysis of Engagement Issues

Subjective feedback from players (see Appendix A) points to engagement bottlenecks :

— The game lacks active, visual, and narrative rewards to motivate progression. After the initial novelty wears off, coding tasks feel repetitive.
— The default user interface is plain, lacking clear guidance, tutorial hints, or aesthetic details, causing beginners to feel lost.
— Play sessions occur mainly upon request, during classes, after email reminders or when motivated by external factors (see Appendix C), indicating a lack of spontaneous engagement.

4.7.4 Identification of Design Limitations

Previous versions of Refactal suffer from major technological and pedagogical limitations :

— **Superficial Metrics :** Code evaluation is dynamic and limited to lines of code and completion times. There is no automated analysis of code quality (e.g., cohesion, coupling, or compliance with the design principles taught).
— **Static Assistance :** The hints are static, pre-written slides that do not adapt to the player's behavior, errors, or coding attempts. The AI assistant in version 3.0 (Hamdane, 2025) only serves to ammend a deeply rooted problem.
— **Linear Progression :** The rigid linear progression forces players to complete every puzzle in order, causing them to abandon the entire game when stuck on a single difficult level.

4.8 Comparative Positioning with Existing Games for Programming

Refactal occupies a unique niche compared to existing games for programming. Those games, especially the commercial ones, typically focus on :

— **Basic Syntax and Computational Thinking :** teaching introductory coding syntax, loops, and conditions to children or beginners.
— **Low-Level and Assembly Programming :** Games like TIS-100 and Shenzhen I/O focus on low-level instructions, registers, and hardware-level constraints. A choice the developers made as low-level programming interests them.

— **Scripting and Agent Automation :** Games like Robocode and Screeps focus on coding artificial intelligence and controlling autonomous units (tanks or creeps).

None of these commercial games (and only a few academic ones) target software architecture, **SOLID principles, design patterns (GoF, GRASP),** or **code smells**. Refactal is the one of few games designed to teach high-level software engineering design quality (see the previously discussed Relics of Chaos and UMLCraft as other examples). However, commercial games excel in visual design, narrative integration, smooth onboarding, and adaptive difficulty. To succeed, Refactal must combine its advanced software engineering curriculum with the polished UX, narrative pacing, and progression design of commercial games.

4.9 Discussion and Synthesis

The background audit of Refactal highlights a clear gap : while the educational curriculum (**SOLID Principles, GRASP Design Patterns, GoF Design Patterns,... etc**) is highly valuable for software engineering students, the game's execution suffers from severe onboarding bottlenecks, rigid linear progression, lack of visual rewards, and data collection issues.

From a retention standpoint, the data reveals a critical onboarding failure : over 34% of players abandon the game at the very first tutorial level, and 83% have quit by the end of the second Chapter. This is worsened by a poorly calibrated difficulty curve, where levels such as **OCP, Controller, Code Duplication, and State** show disproportionately long completion times and might indicate players' high frustration. Alongside the rigid linear progression, players stuck on a single level have no alternative path forward and are forced to either persist or abandon the game entirely.

From an engagement standpoint, subjective player feedback aligns with what the quantitative data suggests : the game lacks visual rewards, narrative momentum, and adaptive guidance. Play sessions occur primarily upon external prompting rather than spontaneous motivation, pointing to a game that has not succeeded in sustaining intrinsic engagement beyond initial novelty.

Finally, from a data and evaluation standpoint, the existing infrastructure presents significant limitations. Code is only evaluated based on completion time (players learn at different rates), number of lines of code (deleting empty lines inflates a player's score) and cyclomatic complexity (inherent in the provided partial code, so the player can not control it). Hints are static and non-adaptive, and the original data collection pipeline through SilentWolf is difficult to retrieve and nearly impossible to analyse systematically, only partially addressed by the administrative dashboard.

Taken together, these findings confirm that the shortcomings of Refactal are not isolated but deeply interconnected, spanning onboarding design, progression structure, engagement mechanics, and data infrastructure.

4.10 Conclusion

In summary, this background analysis of Refactal versions 1.0, 2.0, and 3.0 demonstrates that a successful educational game requires a balanced integration of pedagogical content, gameplay dynamics, and robust data collections. The severe player drop-off rates and learning bottlenecks identified in previous tests validate the need for our proposed architectural redesign, which we detail in the following chapters.

Chapitre 5

System Design

5.1 Introduction

Designing an educational serious game that successfully bridges the gap between complex software engineering paradigms and engaging gameplay mechanics requires a systematic and well thought out design methodology. Unlike traditional entertainment games where the sole objective is player amusement, or non-gamified educational software where pedagogy is often reduced to multiple-choice quizzes, a software engineering serious game must treat code as a direct, physical interactive element within the game world.

This chapter presents the comprehensive system design of the redesigned Refactal serious game. It details the redesign objectives, the selection of the game genre, our core design philosophy, and the respect of not only pedagogical principles but also gameplay design principles simultaneously. Furthermore, we outline the system architecture, the blueprints for our player data collection methods, and the user interface layout. The entire system design is structurally grounded in the MDA framework (Hunicke et al., 2004), with inspiration from certain principles of the Framework for Adaptive Serious Games (F4ASG) (Pistono et al., 2024), ensuring that mechanical choices serve an explicit educational purpose.

5.2 Design Objectives and Redesign Strategy

5.2.1 Redesign Objectives

The architectural overhaul of Refactal is driven by three primary design objectives :

— **Educational Objectives :** The primary objective is to teach concepts of advanced object-oriented design. The game must require players to actively design and complete real code, reinforcing deep structural understanding rather than surface-level terminology.

— **Engagement Objectives :** Address the challenges encountered at the start and onboarding bottlenecks of prior iterations, by designing a smooth tutorial sequence

and providing contextual coding help. Also, creating a cohesive, visually appealing environment, accompanied by an immersive narrative that motivates the player to sustain focus across complex challenges, thereby mitigating the attenuation rates observed in Refactal.

— **Technical Objectives :** Design an automated player data collection logic of recording pertinent player behavior, which serves both to adapt in-game help and to feed an administrative visualization dashboard.

5.2.2 Scope

While a redesign of Refactal, some elements of previous versions can still be salvaged, not only to limit the scope of the work but also to highlight the good parts of the previous versions. The redesign scope is defined as follows :

— **Elements Preserved from the Original Game :** The science fiction narrative setting aboard the space station Orbite, the escape-room level progression gating where physical barriers are cleared via completing partial code, and the focus on advanced software architecture concepts.

— **Elements Modified :** Evolving the gameplay perspective from a first-person 3D environment to a 2D environment, replacing static slideshow hints at the start of the level with a programming guide accessible at all times and containing diegetic hints.

— **Newly Introduced Systems :** A custom made coding environment (Interpreter) instead of the game engine provided one used in previous version (Morsli, 2022), a non-blocking asynchronous event-driven player data tracking manager, a reworked user interface.

5.3 Global Design Framework

5.3.1 Core Design Philosophy

Three fundamental design principles guide the remake :

— **Code as a Concrete Physical Element :** Code modifications must give immediate, observable changes in the physical environment (e.g., rotating platforms, unlocking doors).

— **Active Refactoring over Syntax Exercises :** Puzzles present functional but poorly designed code templates (e.g., one object that does everything in a room). Players must refactor the architecture, mirroring real-world software engineering.

— **Exploration-Driven Discovery :** Players learn through code comments, error messages and trial-and-error within a risk-free editor, rather than reading definition slides beforehand.

5.3.2 Pedagogical Design

While the initial aim was to teach as many advanced OOP concepts as possible, a compromise was made early on (anticipating difficulties that will be encountered). Three concepts were chosen for the redesign :

— **Single Responsibility Principle (SRP) :** A segment that covers at least an identical concept covered by the previous version of Refactal was deemed necessary, SRP was chosen mostly as a baseline for comparison.

— **Builder Design Pattern :** Of all the literature found (and other non-academic sources), the Builder Design Pattern was one of the patterns not covered by any previous game, serious (like Refactal or Relics of Chaos) or not. It was chosen as a novel contribution factor, and for a *perceived* ease of implementation.

— **Memento Design Pattern :** Much like the Builder Design Pattern, the Memento Design Pattern was one of the patterns not covered by any previous game, serious (like Refactal or Relics of Chaos) or not. It was also chosen as a novel contribution factor, and for a *perceived* ease of implementation.

5.3.3 Choice of Game Genre and Experience Direction

Two core aesthetic categories from the MDA framework guided the genre selection : **Challenge** and **Discovery**. Challenge demands that the game present meaningful, skill-testing problems where success feels earned, that means coding problems that require genuine understanding rather than guesswork. Discovery asks that the world reward curiosity and exploration, drawing players forward through the desire to uncover what lies beyond the next room, making the game intrinsically motivating rather than extrinsically. Together, these aesthetics pointed clearly toward a puzzle-centric, non-linear exploration-based platformer (the non-linear exploration-based platformer genre is colloquially known as a Metroidvania (Wikipedia, s. d.)).

This type of genre also aligns well with some F4ASG aspects, learning aspects such as opportunities to redo tasks, and gaming aspects such as choices, levels (rooms), rewards. As non-linear exploration-based games encourage going back to previous locations (redoing tasks) and provide ample paths for the player to choose which to take (providing choices to the player).

The game was chosen to be 2D (instead of the previously used 3D environments), as that allows us personally more freedom for visual expression, and allows players to observe more of the context of a coding problem at once. Player feedback (see Appendix A) also points to the 3D Refactal being hardware intensive (although that might be an optimization-level issue) and rather dull and repetitive looks-wise.

The player controls a character navigating the rooms of the space station Orbite, preserving continuity with the previous version's setting (Morsli, 2022). Certain experience feelings we want to be evoked are :

— **Discovery through Spatial Metaphors for Software Layers :** The 2D map translates abstract code components into physical space. Locked doors, moving platforms, and terminals stand in for software boundaries and interfaces that must be

accessed or modified through coding.

— **Challenge through Natural Gating and Progression :** The room-based structure acts as a physical gate reinforcing the Challenge aesthetic : to advance, the player must resolve each room's coding problem.

— **Replayability through Non-Linear Exploration :** Unlike linear genres, the Metroidvania structure allows players to revisit earlier areas with newly acquired abilities or understanding, creating opportunities for self-directed exploration and repeat engagement.

— **Ease-of-use through Reduced Mechanical Complexity :** Moving from a first-person 3D environment to a 2D side-scrolling view removes the burden of spatial navigation (camera control and locating objects in three dimensions) that would otherwise compete with the player's attention during problem-solving.

5.4 Gameplay and Progression Design

5.4.1 Exploration and Ability-Based Progression

Rather than forcing a purely linear path, the space station is divided into distinct thematic sectors. The player can explore corridors, discover programming guide pages containing information, and interact with malfunctioning devices. Progress to subsequent sectors is gated by the hardest coding problems that act as roadblocks.

5.4.2 Puzzle Design

Every coding problem (coding puzzle) encounter follows a three-phase cycle :

— **Diagnosis :** The player inspects the physical layout and each device's code snippet (programmable object's code snippet) present in the room, which highlights architectural violations (e.g., "SRP not respected : This Button is doing *too many* things").

— **Execution/Refactoring :** The player, having read comments potentially present in the code snippets, information available through the programming guide and using their prior knowledge, can then edit the code structure to fix its underlying issues.

— **Validation :** The game evaluates the code. Success unlocks the pathway ; failure outputs diagnostic compilation errors.

In the context of a puzzle room, the player is expected to diagnose the room's code, Refactor it and validate their solution before either attempting to solve the problem again or going to the next room (see Figure 5.1 for a UML state diagram representation).

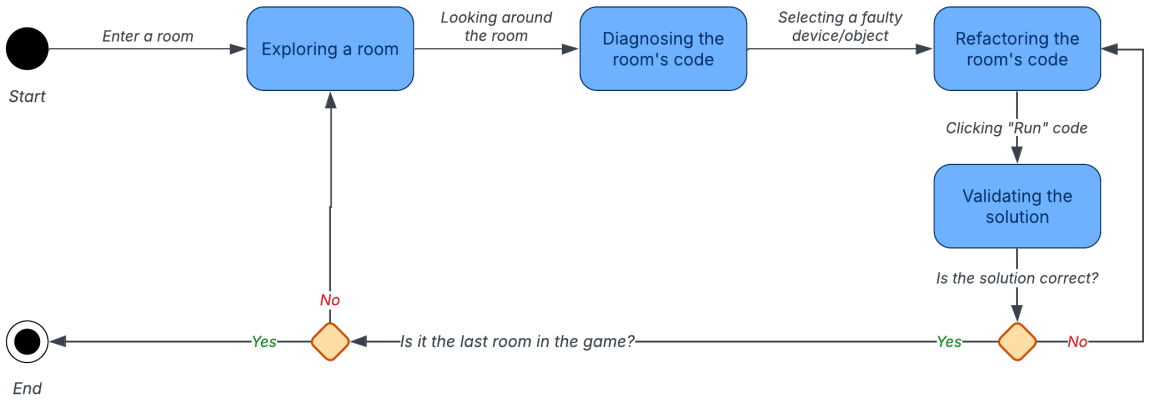


FIGURE 5.1 – UML state diagram representing the puzzle design and room based progression.

5.4.3 Boss Design and Pattern Representation

In video-games, bosses are a type of enemy, generally far stronger than others previously encountered by the players up to that point in a game. The concept of boss can be generalized to non-entities as well.

Each sector culminates in an encounter with a System Boss of sorts. These are either actual bosses, or puzzles significantly harder than the ones that came before them. The player must assess the situation at hand, facing some potential constraints unique to each boss. The aim of a boss encounter in our case is turning architectural design concepts into active gameplay mechanics, and having the player interact with them by either *intentionally* not respecting the advanced OOP concept the boss exemplifies, or by applying it where it is not applied at all.

5.4.4 Narrative Design

The player character's spaceship crashes into Orbite, a seemingly silent and empty gigantic space ship. The player must then investigate the cause of the crash by getting in contact with Orbite's Captain. The player is accompanied by a helper robot, whose dialogue provides conceptual hints.

5.4.5 Room Design Principles

Rooms are styled according to their pedagogical theme (All rooms treating the same concept must look visually cohesive). In each section, the rooms start easy in difficulty, increasing as the player traverses them. A room must be easily navigated by the player ; while the game *is* a platformer, the challenge should lie only in the coding problems, not in any moving actions that the player is expected to do (Jumping from one platform to another should be easy, no time sensitive motor actions, not punishing platforming mistakes,...etc).

5.4.6 Game Loop Design

The main game loop represents a continuous cycle of interaction. The player moves through the physical space, encounters a coding puzzle that blocks progress, accesses the scripting workspace to diagnose and write structural refactoring edits, compiles the code, and either debugs failures or progresses to the next room upon success.

As to not induce monotony, the main game loop is broken by either story elements (dialogues), boss fights or previously unmentioned mini-games that remain within the context of advanced OOP concepts. Figure 5.2 illustrates the entire game loop.

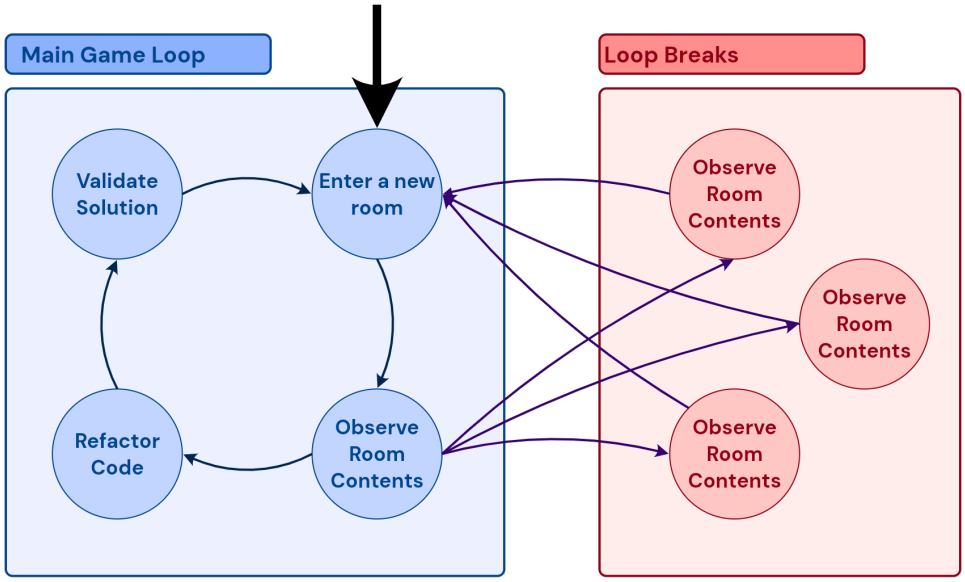


FIGURE 5.2 – A state machine representing the game loop, with states grouped into two distinct parts : Main Game Loop and Loop Breaks.

While the mini-games can take any form, as long as it differs significantly from the main game loop's form, we opted for one mini-game style in particular : a timed quiz.

With previous versions of Refactal, responder feedback has been lackluster. The last recorded test (found in (Arrouche & Tirouche, 2024)) had 118 participants, of which only 10 reached the end. Of the initial 118, only 20 participants filled out the pre-test forums, and only 10 filled out the post-test. A way to capitalize on the players who won't fill out the forms is by gamifying the pre-test and post-test and including them inside of the game, not as a replacement, but as extra data points to be leveraged.

While the easiest format for the quiz is choice-based questions (Such as multiple choice questions, cloze questions, ...etc), research has shown their effectiveness in teaching and testing knowledge acquisition in certain fields such as nurse education and text comprehension (Brady, 2005 ; Oc & Hassen, 2025). To the best of our knowledge, it has not been tested with advanced OOP concepts as a topic.

Before proceeding with the quiz mini-game, an experiment was conducted, where a collection of OOP terms were grouped into three categories : easy, medium and hard. Each category contained at least two groups (shown in Figure 5.3 below). A Find the Odd One Out style questions were made, two groups are chosen randomly from one of

the categories of difficulty, three words are taken from the first group and one from the second group. A participant must find the odd term out of the bunch.

Easy		Medium					Hard		
Basic OOP Concepts	Advanced OOP Concepts	GoF Patterns	GRASP Principles	Code Smell	Anti-Patterns	SOLID Principles	Structural Patterns	Conceptual Patterns	Behavioural Patterns
Class	Code Smells	Creational Patterns	Creator	Code Duplication	God Object	Single Responsibility	Factory Method	Adapter	Interpreter
Object	Design Patterns	Structual Patterns	Information Expert	Feature Envy	Magic Number	Open-Closed	Abstract Factory	Bridge	Template Method
Instance	SOLID Principles	Behavioural Patterns	Low Coupling	Large Class	Poltergeist	Liskov Substitution	Builder	Composite	Chain of Responsibility
Polymorphisme	GRASP Principles		High Cohesion	Shotgun surgery	Big Ball of Mud	Interface Segregation	Prototype	Decorator	Command
Inheritance	Anti-Patterns		Controller		Gas Factory	Dependency Inversion	Singelton	Façade	Iterator
Encapsulation			Pure Fabrication		Spaghetti Code			Proxy	Mediator
Redefinition			Indirection					Flyweight	Memento
Override			Polymorphisme						Observer
									State
									Strategy
									Visitor

FIGURE 5.3 – The grouping of OOP terms per category of difficulty.

Starting with the easy category, the difficulty increases with each five questions, ending in question fifteen. Each question is timed, with ten seconds to answer. This experiment was conducted against six individuals, separately and individually. They possessed varying prior knowledge in OOP. The results of the experiment are as shown in Table I :

TABLE I – Results of the quiz experiment.

Tester	**Ratio of correctly answered Easy Questions**	**Ratio of correctly answered Medium Questions**	**Ratio of correctly answered Hard Questions**	**Correct Answers in Total**
1	5/5	4/5	2/5	11/15
2	3/5	1/5	1/5	5/15
3	4/5	2/5	1/5	7/15
4	2/5	1/5	1/5	4/15
5	2/5	1/5	1/5	4/15
6	5/5	4/5	2/5	12/15

From Tables I and II, we can see a clear pattern emerge. Individuals #4 and #5 got the lowest scores in the quiz, and their actual level of OOP concepts understanding reflects that they know enough to answer some of the easiest questions while lacking any advanced knowledge. Individuals #2 and #3 got scores hovering around the average (7.17), showing either complete understanding of OOP basics, or a mixed understanding of OOP advanced concepts. Individuals #1 and #6 showed the highest scores, implying moderate knowledge, but complained about the time limit of questions being too short.

Table II – Expected vs. actual level of OOP understanding based on the quiz experiment.

Tester	**Expected level of OOP concepts understanding [Based on quiz results]**	**Actual level of OOP concepts understanding**
1	Knows OOP basics and advanced concepts but does not distinguish between GoF Patterns.	Knows OOP basics (had an OOP course in 2nd year of higher education), exposed to advanced OOP concepts but does not memorize all GoF patterns by type.
2	Some OOP basics knowledge, doesn't know advanced concepts, answered based on luck/intuition.	Knows OOP basics (had an OOP course in 2nd year of higher education).
3	Knows OOP basics and *some* advanced concepts but doesn't know GoF Patterns.	Knows OOP basics (had an OOP course in 2nd year of higher education), knows some advanced OOP concepts due to software development experience.
4	Little to no OOP knowledge, answered based on luck/intuition.	Knows OOP basics (had an OOP course in 2nd year of higher education).
5	Little to no OOP knowledge, answered based on luck/intuition.	Knows OOP basics (had an OOP course in 2nd year of higher education).
6	Knows OOP basics and *some* advanced concepts.	Knows OOP basics (had an OOP course in 2nd year of higher education), exposed to advanced OOP concepts through a different course's exercise material.

The experiment conducted, while small in nature, shows promise in the quiz as a minigame. In light of the results, it was decided to go forward with it in the implementation phase.

5.5 System and Technical Architecture

5.5.1 System Architecture Overview

The architecture of the redesigned Refactal is organised as Figure 5.4 illustrates. The Player interacts with the Game Client, which sends Player data over to a Database. The Administrative Dashboards regularly fetches data from the Database, to be visualized.



FIGURE 5.4 – General system architecture of the redesigned Refactal

5.5.2 Core System Components

As most game engines prefer component-based design, we were guided to opt for a hierarchical component representation of our system. At the highest level, our system is composed of three main components :

1. **The Game** : The most complicated of the three components, and where most of the implementation efforts will be concentrated. It is composed of seven subcomponents :
   - **GameController** : The central coordinator that manages game state transitions, level loading, object initialization, and bindings between local game objects and the execution sandbox.
   - **Rooms** : Individual environmental levels containing the physical 2D collision layouts, interactive terminals, gates, hazards, and mechanical systems.
   - **Interpreter** : The scripting execution engine that tokenizes, parses, validates, and runs player-submitted refactoring code in a sandboxed, isolated virtual memory space.
   - **UserInterface** : The workspace displaying the active 2D game world on the left and the slide-out IDE (with editor panels, compile consoles, and assertion checklists) on the right.
   - **ProgressSaver** : The client-side persistence controller responsible for saving and restoring player progression, level completions, and session scores.
   - **EventBus** : A decoupled global event dispatcher enabling asynchronous communication between subsystems (e.g., terminal triggers, dialog completions, and doors unlocking).
   - **DatabaseManager** : The analytics client responsible for formatting metrics payloads, managing database connections, and syncing offline events.
2. **Database** : The storage solution for the game, it will store player data for dashboard visualization.
3. **Dashboard** : Visualizes the players' data, showing learning and engagement indicators, among other measures for evaluating the game.

The general component-based composition of the redesigned Refactal is illustrated in Figure 5.5 below.

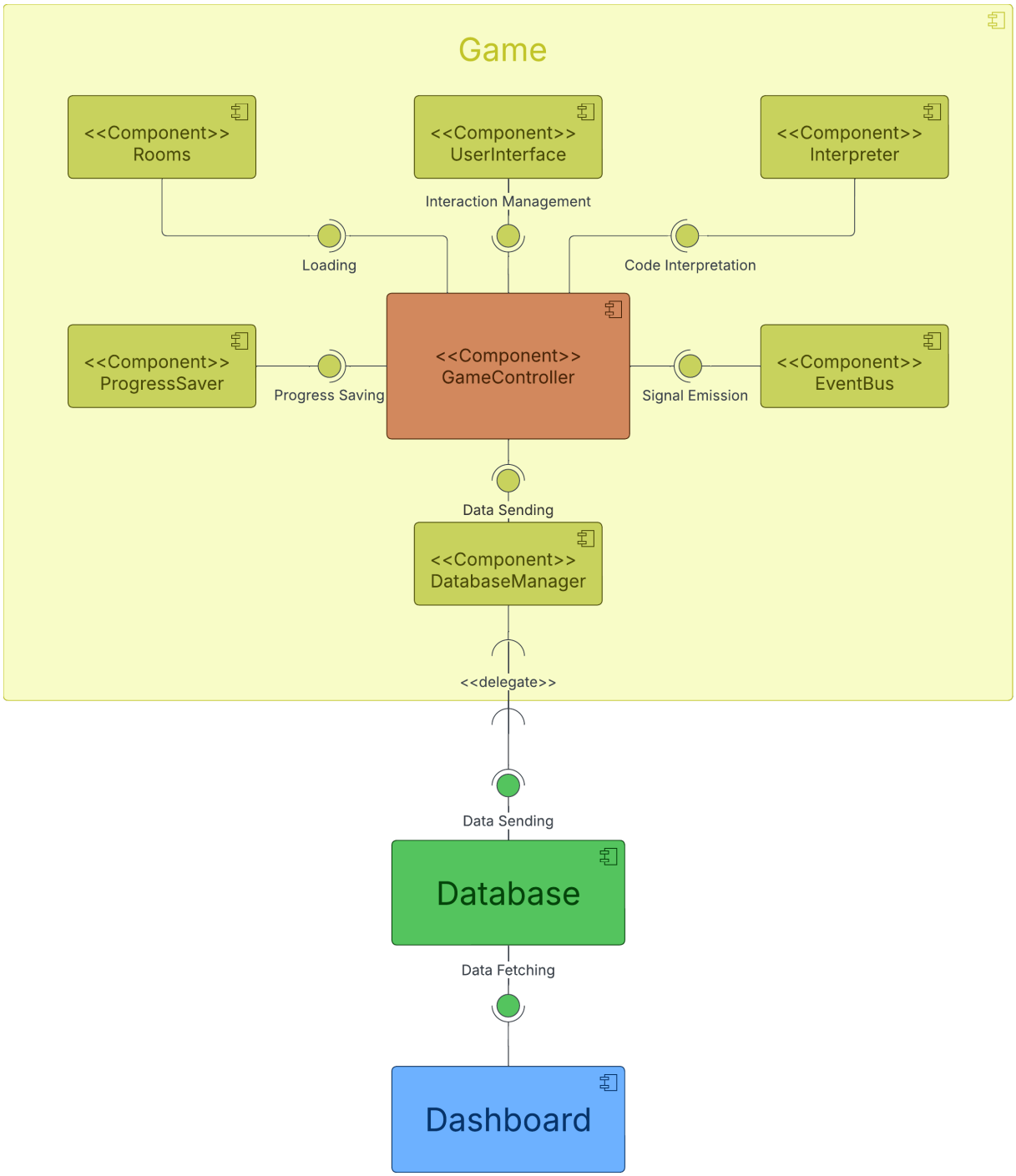


FIGURE 5.5 – General system components diagram of the redesigned Refactal

5.5.3 Data Flow and System Interactions

A typical execution sequence follows can be described by the conceptual data flow below (also see the sequence diagram in Figure 5.6 for an illustration), with all interactions between the subcomponents of the **Game** component passing through the **GameController** (unless stated otherwise) :

1. The player clicks “Run Code” in the editor workspace (within the **UserInterface** component).

2. The editor UI script forwards the plain-text code to the **Interpreter**.
3. The **Interpreter** takes care of the code, where upon correct solving of the code issue, an event is emitted to the **EventBus**, bypassing the **GameController**.
4. The **GameController** captures this event, updating player progression states using the **ProgressSaver**, and invoking doors or platforms to trigger physical animations in the scene.
5. Simultaneously, the **DatabaseManager** captures editor analytics, queues a transmission payload, and posts it asynchronously to the remote **Database**.

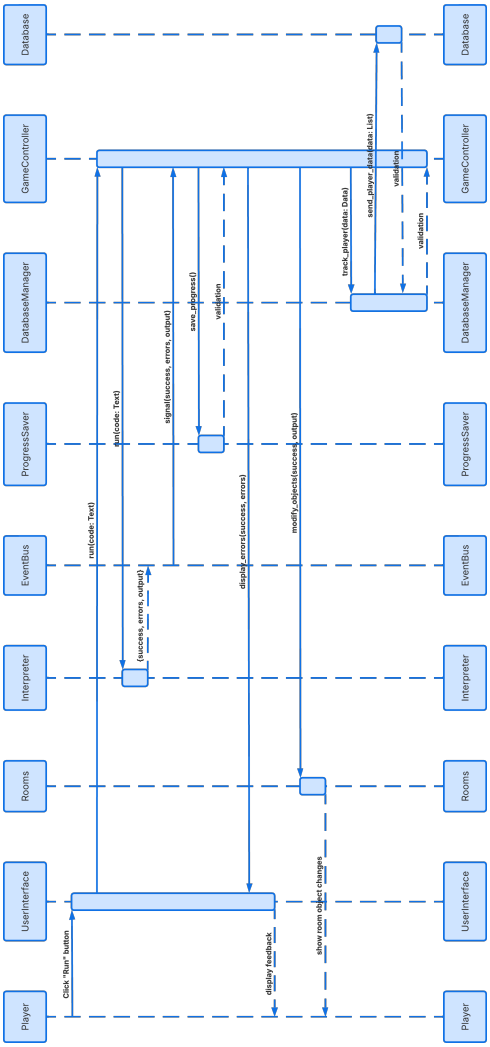


FIGURE 5.6 – Sequence Diagram, code execution flow from player input through the interpreter to world response.

5.5.4 Player Tracking and Analytics Design

The tracking of player behavior must happen without impacting runtime performance, and simultaneously assuring the data is received by the database despite any potential connectivity issues on the part of the player.

To enable learning analytics, an event-driven philosophy was adopted. Through the **EventBus** launching signals for particular actions the player takes, the **DatabaseMana-**

ger captures said signals, with the essential data associated with each signal given by the **GameController**. The **DatabaseManager** can then log the data, to be sent when a reliable connection is established with the **Database**. This system tracks three categories of events :

— **Player identity.** On first launch, the game creates a player record containing the username and game version (for assessing each game version separately). The returned ID is encrypted and stored locally on the player's machine, providing a persistent identifier across sessions without requiring login or registration.

— **Room events** : Every room entry and exit fires an asynchronous POST request to the analytics backend. Each event record identifies the player, the room, the chapter, the current game version, and the type of transition (entry/exit). A third event type records when the player resets their code to the template, providing a proxy measure of puzzle difficulty. The combination of entry and exit timestamps allows computation of time spent per puzzle room.

— **Quiz session data** : Each quiz session records a session-level entry and a per-question record for all 15 questions, containing the question number, question type, difficulty tier, player answer, correctness boolean, and time remaining on the timer at the moment of answer. The per-question difficulty-tier and time remaining fields are specifically designed to support knowledge tracing analysis (Corbett & Anderson, 1997), enabling modelling of each student's learning trajectory across the three advanced OOP concepts' chapters.

5.6 User Experience Design

5.6.1 User Interface Design

Due to previous criticism of the programming experience being wholly separate from the gaming experience (see Background Analysis), the layout was redesigned to prevent detaching the learning from the playing, by splitting the UI (User Interface) into a 2D playing environment part and a coding environment part :

— **Left Sector (envisioned to be around 60% of the screen) :** The 2D physical world showing character physics, environmental layouts, and running platforms.

— **Right Sector (envisioned to be around 40% of the screen) :** The slide-out console. When the player accesses a programmable device, the console slides in smoothly.

Other UI elements include :

— **The Guide Book :** A physical book within the game's fiction, accessible at any time. Its pages unlock progressively as the player advances through rooms, ensuring reference material is available precisely when it is relevant. When the guide is open, the code editor is hidden to prevent visual overlap.

— **The Pause Menu :** A lightweight overlay providing three actions : resume the session, consult the (moving) controls manual, and return to the main menu.

— **The Quiz UI** : A full-screen interface that replaces the game world during assessment sessions. It presents a question counter, a score display, a countdown timer, and the current question.
— **The start menu :** The application entry point.

5.6.2 Interaction and Feedback Design

Feedback is immediate and multi-sensory :

— **Aesthetic Themes :** Successful compilations should play animations on all the concerned devices.
— **Error Visualization :** Syntax or structural errors should display a pop-up window, showing the exact cause of the error.
— **Programming Guide :** The programming guide should include, alongside the needed pedagogical definitions and syntax help, contextual hints to puzzles.

5.7 Conclusion

This chapter detailed the system design of the redesigned Refactal serious game. Grounded conceptually in the DDE and F4ASG frameworks, the proposed redesign maps advanced pedagogical concepts (the Single Responsibility Principle, the Builder pattern, and the Memento pattern) to physical 2D puzzle-platformer challenges. By transforming the environment into a side-scrolling exploration layout, planning event-driven asynchronous player data tracking, and styling a balanced split-screen user interface, this design directly addresses the engagement and usability limitations identified in historical cohorts. These conceptual system design blueprints establish the theoretical foundation for the technical implementation and the empirical evaluation discussed in the subsequent chapters.

Chapitre 6

Implementation

6.1 Introduction

In this chapter, We will describe the full technical implementation of Refactal : Reimagined (the redesigned Refactal game which will henceforth be referred to as RE :RE), a 2D side-scrolling puzzle-platformer developed in Godot 4.6 that utilizes live code execution as its main gameplay mechanic. The player is inhabiting an abandoned space station and interacts with its systems doors, stairs, lights, robots, and other devices and objects by writing real programs in a GDScript-like language, which is tokenised, parsed, validated and executed at runtime by a fully embedded interpreter built within the game engine itself.

The implementation work described in this chapter translates the design objectives laid out in the previous System Design into concrete, functioning software. Every system described here from the player controller and the embedded interpreter pipeline to the Supabase analytics backend and the progressive guide unlock mechanism was built, tested, and integrated within the Godot 4 scene tree.

The chapter is organised to mirror the architecture of the game itself : from the development environment and project structure, through the individual subsystems in order of dependency. Finally, we highlight the principal game development challenges encountered during the project and the engineering solutions implemented to resolve them.

6.2 Technical Stack & Development Environment

6.2.1 Engine Configuration & GDScript

RE :RE is built entirely in **Godot Engine 4.6**. Godot was selected primarily because its scene-tree architecture and signal bus system map perfectly onto the decoupled, modular room design the game requires (where each room is an independent scene and communication relies on signals rather than hard-coded direct references). Being an open-

source engine, it doesn't suffer from certain drawbacks that commercialized engines do (e.g : Enforcing taxes on developer per user game installation).

All game logic is implemented in **GDScript**, Godot's native object-oriented language. It was chosen because its Python-like syntax is highly approachable for software engineering students, allowing them to easily read and manipulate the game's source code. To avoid manual memory management and prevent memory leaks, the custom interpreter pipeline (Lexer, Parser, Validator, and Executor) is built extending Godot's RefCounted base class, meaning these objects are automatically freed when no longer referenced.

6.2.2 Cloud Integration & Local Persistence

For player analytics and data tracking, the game utilizes **Supabase**, a cloud-hosted PostgreSQL backend-as-a-service. Godot's open-source nature and built-in HTTPRequest node allows the implementation of a Supabase REST API, provided as a plug-in by the community of Godot game developers. All database communication is handled via asynchronous HTTP POST requests from the SupabaseDB autoloaded singleton.

For local state persistence (such as the player's current room, spatial coordinates, and session flags), the game relies on JSON Serialization. This data is serialized to a local save file (user ://autosave.json), allowing seamless resumption of gameplay across multiple sessions.

6.2.3 Development Tools

Alongside the Godot game engine, other development tools were used and are listed below :

— **Visual Studio Code :** commonly referred to as VS Code, a free integrated development environment (IDE) developed by Microsoft.

— **Git :** a distributed version control software system, manages versions of source code and data.

— **Github Desktop :** a free and open source application that helps in working with files hosted on GitHub or other Git hosting services. In this project's case, it was used for easily managing git commit merging conflicts.

— **Supabase :** An open-source Backend-as-a-Service (BaaS) platform that provides developers with ready-to-use backend infrastructure for building web and mobile applications. It offers services such as a PostgreSQL database, user authentication, real-time data synchronization, file storage, serverless edge functions, and APIs automatically generated from the database. It was used in this project for its attractive free-plan offer.

6.2.4 Additional Software and Technologies Used

Over the course of the project, certain tools (technologies, softwares and websites) were used :

Communication Tools

— **Gmail :** A cloud-based email service developed by Google, used for professional communication, exchanging project updates, sharing files, and coordinating between team members.

— **Gmail Chat :** An instant messaging and collaboration platform by Google used for real-time communication, discussion, and coordination.

File Sharing And Concurrent Usage

— **Google Drive :** A cloud storage and file-sharing platform that allows users to store, organize, access, and share project files securely across devices.

— **Google Docs :** A web-based document editor that supports collaborative writing and real-time editing by multiple users simultaneously.

— **Google Sheets :** A cloud-based spreadsheet application used for organizing data, performing calculations, tracking progress, and collaborative data management.

— **Google Forms :** An online form and survey creation tool used for collecting responses, conducting questionnaires, and gathering structured data.

Development Help

— **Claude :** An AI assistant developed by Anthropic used to support development activities such as generating code suggestions, assisting with debugging, explaining concepts, and improving documentation.

Graphics And Figure Creation, Editing And Generation

— **Nano Banana :** An image generation and editing capability associated with Google's generative AI ecosystem, used to create illustrations, visual assets, and enhance graphical content for project materials.

— **Lucidchart :** A web-based diagramming and visualization tool used to design flowcharts, UML diagrams, system architectures, and process illustrations.

— **Gimp :** GNU Image Manipulation Program, an open-source image editing software used for creating, modifying, retouching, and exporting graphical assets.

Dashboarding and Deployment

— **Metabase :** An open-source business intelligence and dashboarding platform used to visualize data, build interactive dashboards, and monitor project metrics.

— **Docker :** A containerization platform that packages applications and their dependencies into isolated environments called containers, simplifying deployment and ensuring consistent execution across systems. Used to deploy a Metabase instance in the case of this project.

6.3 Scene Organization and Systems

6.3.1 Node Hierarchy Within a Room

Every room scene follows a consistent hierarchy (seen in Figure 6.1) that facilitate the creation of each chapter and it goes as follows :

— **Root Node2D :** The first node created in the room and its attached to the room controller script, inheriting from the shared Room base class.
— **Middleground :** Manages 2D collision geometry using custom pixel-art grid tiles defining the structural geometry of each room.
— **Objects Node2D :** Groups all interactable gameplay elements : the player (player.tscn), programmable objects such as doors (door.tscn), buttons, stairs, robots and terminals, alongside their associated proximity areas and collision shapes.
— **UI CanvasLayer :** Contains room-specific interface elements such as dialogue containers, phase hint labels, and proximity prompt labels. This layer is instantiated with the room and freed with it on transition, ensuring room-specific UI never persists beyond its room.
— **Entries Node2D :** Used to mark the player spawn point when entering from a specific direction.
— **Light2D nodes and background layers :** Provide the character and atmospheric lighting and parallax backgrounds used in each room's visual environment.

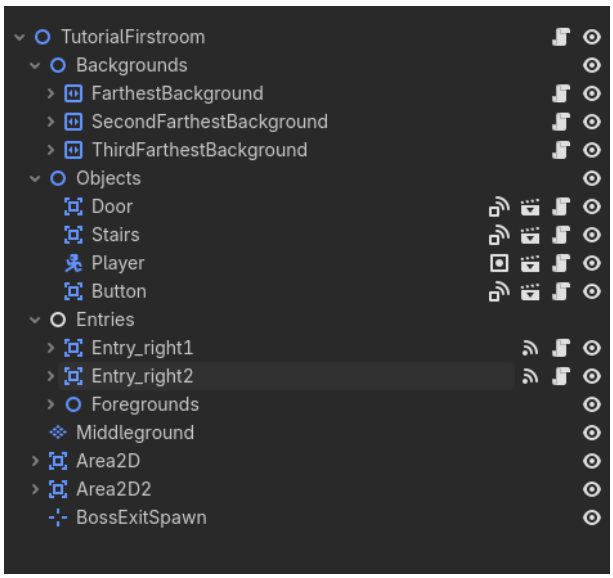


FIGURE 6.1 – Generic Node Hierarchy within a Room.

6.3.2 The Signal System

Two separate signal buses handle different categories of inter-system communication, enforcing a clean separation of concerns.

GlobalSignal carries interaction-layer events that are spatial, per-frame, and directly tied to what the player is hovering or clicking and Its signals include :

— Programming_object_mouse_entered
— Programming_object_mouse_exited
— Programming_object_clicked
— Disable_programming

— Enable_programming

EventBus carries game-flow events with infrequent, session-level, and related to quiz state and difficulty. Its signals include :

— Quiz_question_end
— Quiz_difficulty_up
— Quiz_end
— ...

This separation ensures that the high-frequency interaction signals never interfere with the low-frequency state-management signals, and that neither bus is burdened with responsibilities outside its domain.

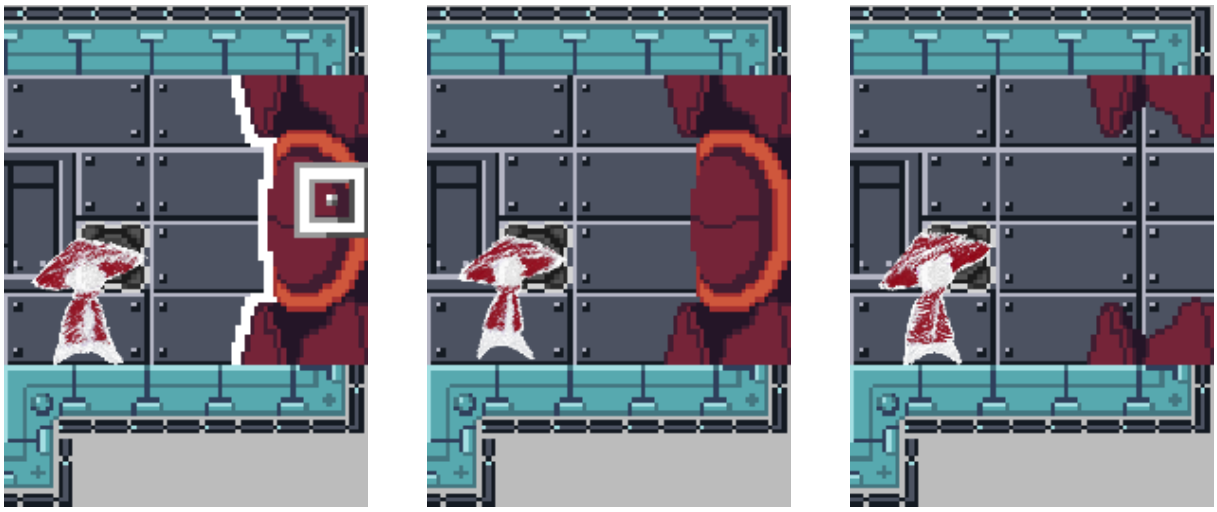
6.4 Gameplay Mechanics Implementation

6.4.1 Player Movement and Physics

The player character can move through the game world using the left/right and jump movements. The player's movement consists only of horizontal ones and a fixed height jump. There is no sprinting, no wall jumping and no skilled movement element, expressly designed as such. The game is not trying to challenge the player's reflexes. The player should be spending their mental effort on reading and writing code, not on navigating an obstacle course. If the player is struggling with the movement, their attention is taken away from reading and writing code. Movement should be learnable in the first few moments.

6.4.2 Interactive and Programming Objects

All programming devices inherit from the same parent programming object, in which are defined the clicking, modifying and execution interactions that the player can have with the object. When the player cursor enters an object's interaction area, a white outline appears (growing bigger and smaller), indicating that it is selectable by mouse click. On click, the object will load its partial code template to the code editor.



(a) Hovered state. *(b)* Normal (closed) state. *(c)* Open state.

FIGURE 6.2 – Difference between the door object states.

There are two types of programming objects in the game :

— **Non-programmable objects** appear in red and cannot be modified by the player. If clicked on, they will open the editor in a locked state showing the object's name and its code but not allowing the player to make any changes.

— **Programmable objects** appear in grey (see Figure 6.3 for an illustration) and will open the editor in editable mode.

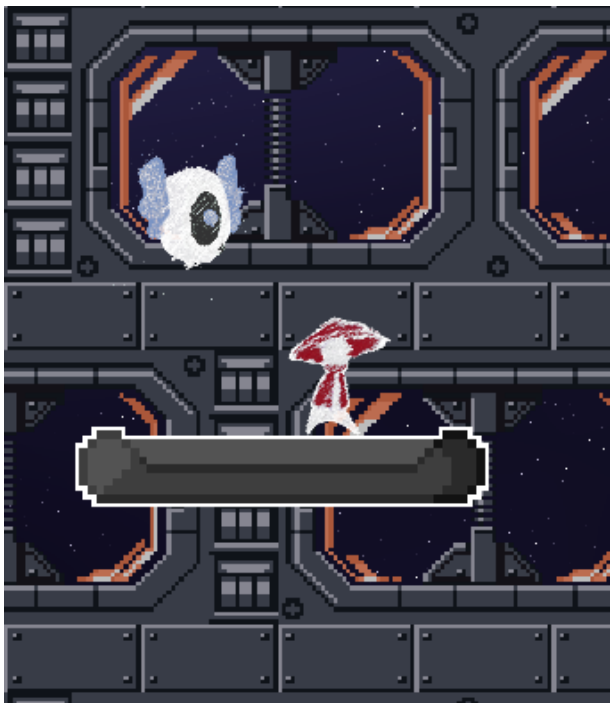


FIGURE 6.3 – A programmable platform (grey programming object) with an actively shrinking outline.

6.4.3 Room Transitions and Progression

A room transitions when all puzzle triggers evaluate as fulfilled and the exit trigger activates. There is no popup or score screen when you complete a room – the world simply reacts : An exit door opens up, or the walk-through trigger at the edge of the room activates. The player walks through it and a transition animation occurs.

In that transition, the game will autosave the current state to disk, and record a log to the analytics backend that the player exited the room. When a new room is loaded, the previous state is restored, the room entry is logged, alongside the position of the player at the specified spawn point for the entry direction.

6.5 Graphics Design

6.5.1 Art Direction

Being one of the main critiques of the previous version of Refactal, the art direction for RE :RE has overgone many iterations (see Figure 6.4) and attempts at finding an appealing look and art style.

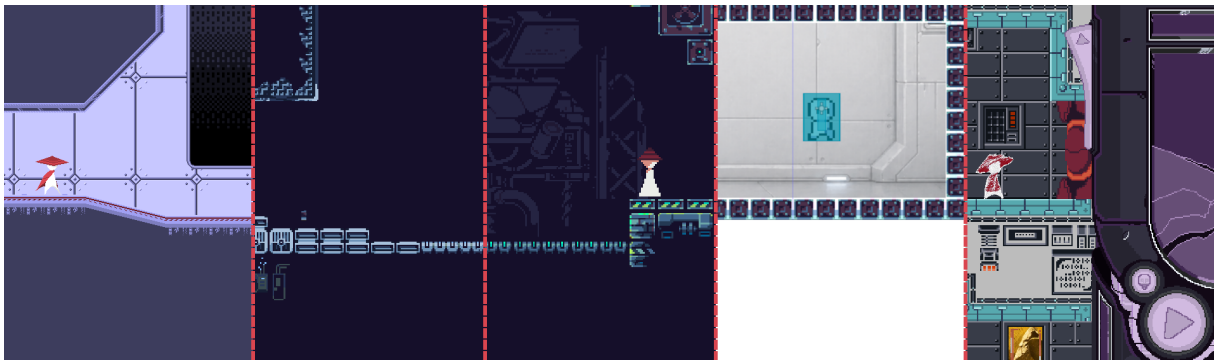


FIGURE 6.4 – The evolution of the graphics of RE :RE over time.

The game's visual style was governed by an industrial science fiction pixel art style, some elements of the design are :

— **Pixel art only :** pixels were chosen for a number of factors. It is a new refresh from the previous versions' empty and simplistic 3D models, it is easier and faster to iterate over multiple designs for an object, and animating them is comparatively easier than 3D animations or hand drawn ones.

— **Limited colour palette :** keeping the visuals as clean as possible, so as to not overwhelm players with colours. But not limited enough that all rooms look the same, similar rooms remain cohesive while different areas remain distinct.

— **Legibility First :** players must not have a hard time distinguishing between foreground, middle ground and background elements, nor have trouble differentiating between interactable and non-interactable objects.

6.5.2 Master Colour Palette

The master colour palette is derived from the existing in-game sprites and defines roles for every colour used in the game world. Table I shows a sample of some primary colours and their roles.

TABLE I – Master colour palette and primary roles.

Role	Hex	Usage
Void black	#0A0A0F	Deepest background, empty space
Panel dark	#111118	Background panels, wall fill
Panel mid	#1C1C28	Mid-tone structural panels
Steel mid	#3D5068	Platform tops, wall mid-tone
Cyan primary	#00C8C8	Glowing accents, door frames, screens
Cyan bright	#80FFFF	Hot highlight on cyan elements
Screen green	#00FF88	Terminal readouts, solved states
Red warning	#CC2222	Active warnings, damage states
Off-white	#C8D4DC	Text, label plates, fine detail lines

6.5.3 Tile System and Grid

All environment art is grid-based, designed on a 32×32 pixel level (see Figure 6.5 for a sample of the environment art). We work with 64×64 sized tiles in-engine within TileMapLayer nodes. Tiles are imported with nearest-neighbour filtering (adjustable in Godot's import settings), to maintain crisp pixel edges when scaling in the engine.

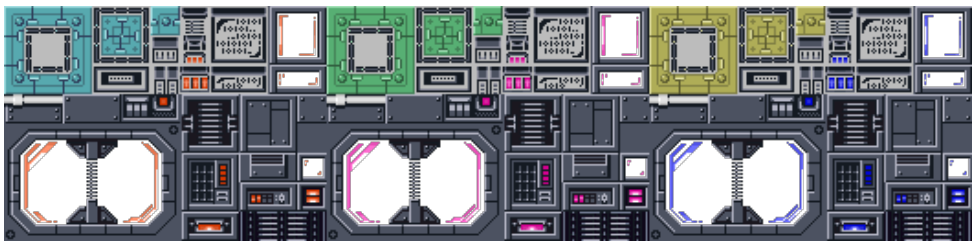


FIGURE 6.5 – Part of the tile grid and texture layout.

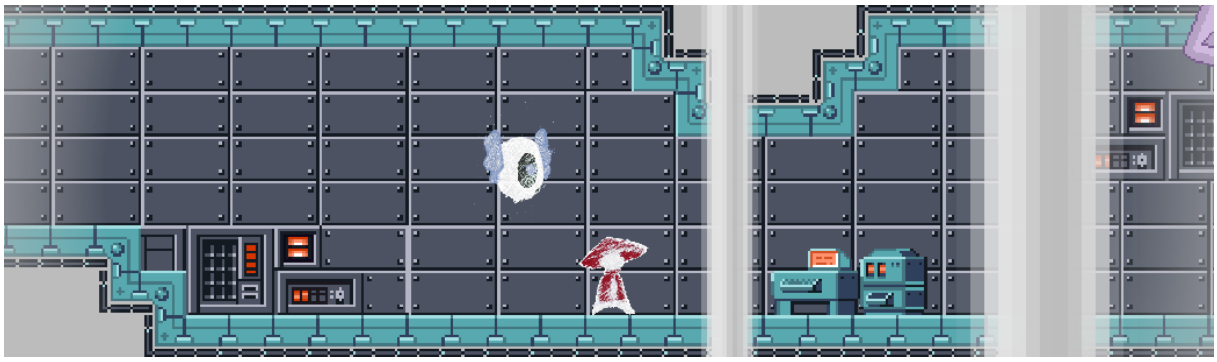
6.5.4 Parallax Background Layers

The game uses a three-layer parallax scrolling system for depth :

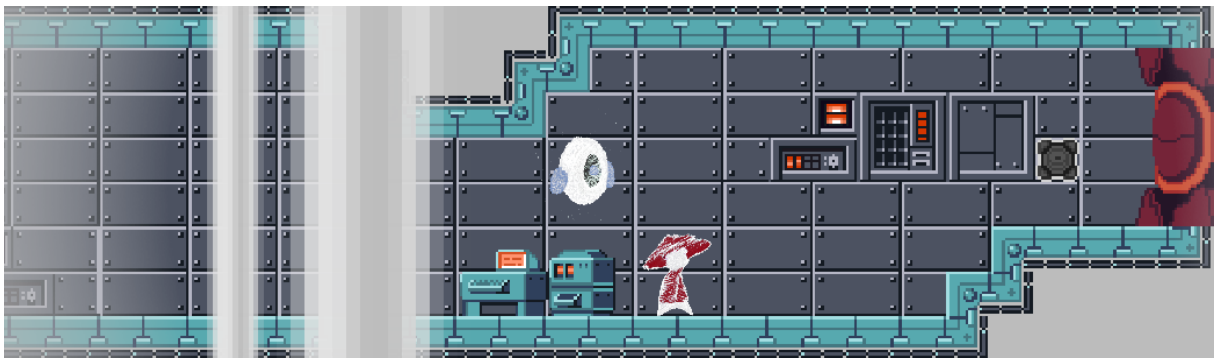
— **Layer 1 (farthest back) :** near-black space exterior or deep interior void ; minimal detail. Scrolls slowly compared to the player's movement.

— **Layer 2 (middle) :** large structural elements, tiles, interactable and non-interactable objects, windows and some colour highlights. Scrolls slightly faster than the player's movement.

— **Layer 3 (nearest) :** pipes, vents, cables and other foreground elements. Scrolls quickly compared to the player's movement. Notice how in Figure 6.6a, when the player character is near the left side of the room, the two vertical pipes in the background are far apart, but as they approach the right side of the room (seen in Figure 6.6b), the pipes move closer to each other, implying that they are closer to the viewing player than the character and the background elements.



(a) Player near the left side of the room, with foreground elements covering parts of the scene.



(b) Player near the right side of the room, the foreground elements having moved.

FIGURE 6.6 – Foreground and background parallax separation.

Each layer is its own Parallax2D instance containing a scrolling script. The scrolling script applies an offset to the background coordinates per frame based on the camera movement. This achieves the parallax effect without needing multiple full-screen sized sprites.

6.5.5 Player Character and Object Sprites

Much like the art direction, the player character sprite has changed a lot over the course of development. You can see in Figure 6.7 a snapshot of the player character's design at different points.

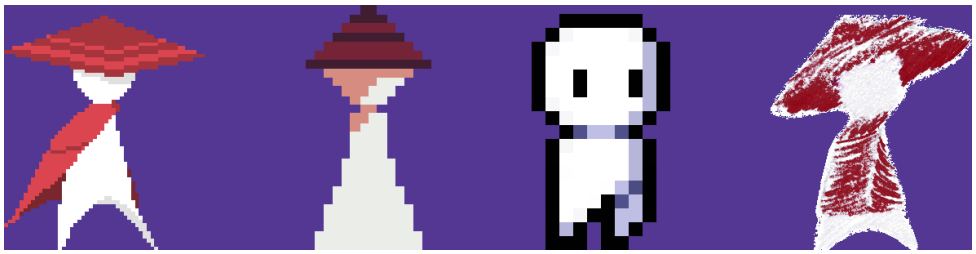
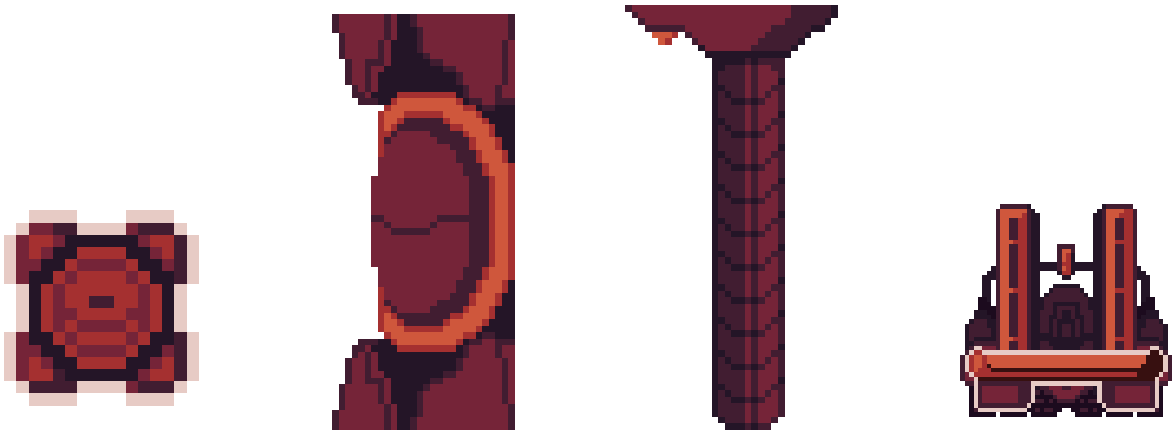


FIGURE 6.7 – The evolution of RE :RE's player character's design.

The player character sprite is a small humanoid figure, an animated sprite which loops one of three animations : idle animation, walk animation (4-frame cycle), or a jump animation.

On the other hand, the objects' sprites didn't evolve much over the course of the game, as remaining simple and identifiable was at the forefront of our objectives when making them. Figure 6.8 shows examples of the programming objects found in the game (Doors, Buttons, Sliding Walls, Forklifts, ... etc).



(a) Button. *(b)* Door. *(c)* Sliding wall. *(d)* Forklift.

FIGURE 6.8 – Some programming objects from RE :RE.

6.6 Puzzle System and Room Implementation

6.6.1 The Code Execution System

The primary technical hurdle RE :RE faces is executing user-authored code at runtime within a compiled game. Instead of relying on Godot's built-in **GDScript.new()** runtime loader (which would allow arbitrary GDScript execution with full engine access and zero sandboxing), RE :RE bundles its own **custom interpreter** pipeline, implemented entirely in GDScript. This provides RE :RE complete control over what code players can express, what errors they are given, and what they are allowed to do.

The Four-Stage Pipeline

— **Lexer** : takes the raw text string the user has inputted and tokenizes it (splits it into typed tokens). Tokens are words that have meaning to the language : keywords,

object names, method names, brackets, operators. If invalid characters or character combinations are present, the Lexer reports a syntax error at the exact location and returns.

— **Parser** : processes the token sequence and generates an Abstract Syntax Tree (which is a nested representation of the program that shows how all the function calls and control flow commands are ordered). If tokens don't make up a grammatically correct program, Parser reports a structural error and quits.

— **Validator** : validates every object name and method name in the tree against the registry of objects currently available in the room. If the player tries to mention an object that isn't there, or calls a method that the object doesn't provide, the Validator signals a validation error with a message naming the problem.

— **Executor** : traverses the validated tree, calling the appropriate methods on registered game objects. The world reacts : a door slides open, stairs unfold, ...etc. If something goes wrong during execution (a method is given bad arguments) the Executor signals an error and aborts.

Note that each stage aborts at the first error it encounters. It sends the error back up to the code editor which presents it to the player. Because of this early-abort behavior, the player never sees nonspecific error messages about internal syntactic or semantic problems. They always get the most specific, most actionable message possible : the error at the root of the chain of problems.

6.6.2 The Object Registry

A central design decision in the interpreter architecture is the separation between the interpreter and the game objects themselves. The interpreter maintains no references to Godot scene nodes. Instead, every programmable object in a room is wrapped by a registry entry that makes available a whitelisted set of callable methods along with their expected arguments. When the Executor wants to call a method, it queries the registry for the wrapper, then calls through.

Two results follow from this. First, it becomes trivial to limit what the player is allowed to do per puzzle : in the SRP chapter, Button A might be allowed only to call **Stairs.extend()**, while Button B is only allowed to call **Door.open()**. This limitation can be expressed by giving each button its own registry. Second, it ensures that the interpreter cannot know any state outside of its own registry and logic. It cannot access the scene tree, it cannot read global variables, it cannot accidentally cause side effects outside of its allowed methods.

6.6.3 Advanced OOP Concept-Specific Validation

Each chapter's advanced OOP concept-themed room adds an additional layer of validation on top of the interpreter correctness checks. In the SRP chapter, once the interpreter successfully validates and runs the player's code, the room script checks that each button only calls its respective methods. When buttons call methods that they shouldn't have, an SRP violation popup appears, mentioning the name of the button and the offen-

ding method called. In the Builder chapter, the room checks that the player called each construction method in the appropriate order. In the Memento chapter, the room checks that the save and restore operations are used properly.

Since this additional validation logic is kept separate from the interpreter logic, each room script is responsible for determining advanced OOP concept-level semantic correctness while the interpreter can simply focus on generic language-level correctness. This allows the interpreter to be reused completely identically between all the chapters while each room implements its own set of validation rules.

6.6.4 Room Structure

The rooms are all self-contained Godot scenes. A typical room scene will contain the tilemap for the geometry of the environment, any interactive/programmable objects within the room, the player start location, trigger area for room entry, trigger area for the exit, and a room controller script. The room controller script stores the puzzle number for that room, and contains the logic unique to that room : what objects need to be in what state for the puzzle to be considered complete, and what should happen when the player solves the puzzle.

The rooms are all subclasses of a common base room class that handles interfacing with the GameController, handling entrance/exit from the room, analytics logging, and applying any saved player state. Since all of the generic room mechanics are handled by the base class, each individual room's script only has to handle adding its puzzle logic on top. This greatly simplifies each room's script.

Tutorial Rooms

The tutorial consists of six rooms that each teach one new game mechanic. The first room teaches movement (by having the player walk to the exit). The second provides a Universal Coding Scanner (UCS) item, and collecting the UCS in that room unlocks the code editor for the first time. The remaining tutorial rooms cover simple method calls (see Figure 6.9), the Reset button, and the guide book. The boss room at the end of tutorial (see Figure 6.10) puts together all of these mechanics into one puzzle that involves programming within a time limit.

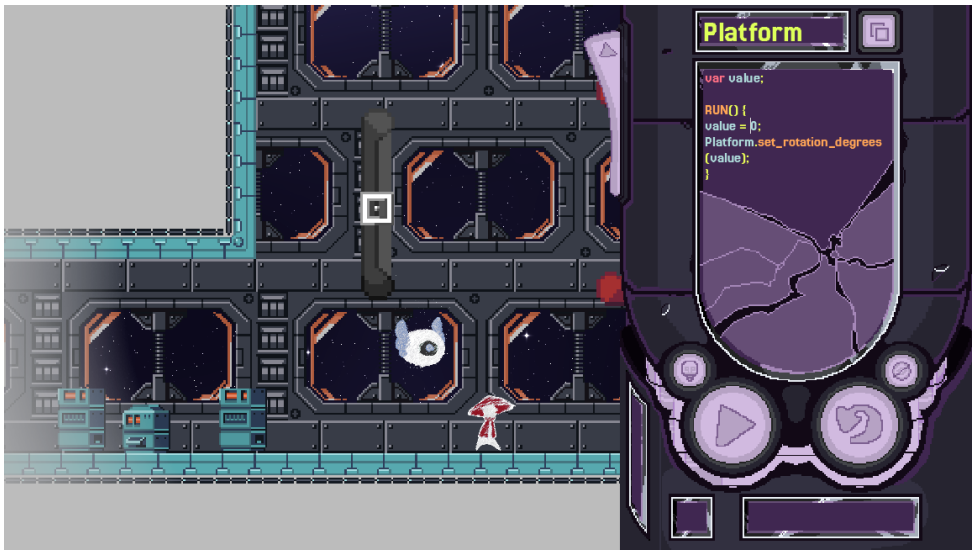


FIGURE 6.9 – Screen capture of RE :RE, showing a tutorial room and gameplay interaction.

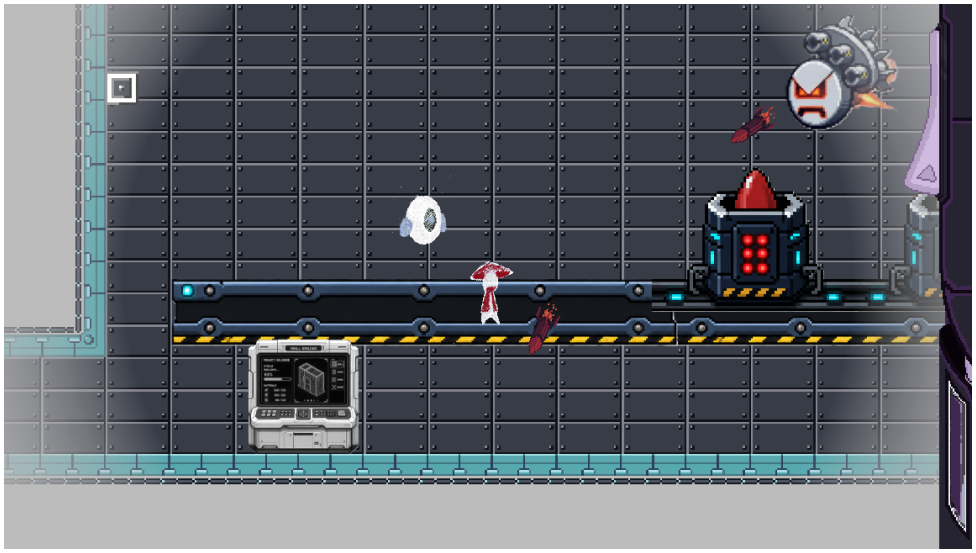


FIGURE 6.10 – Screen capture of RE :RE, showing the boss room.

Advanced OOP Concept Rooms

Each of SRP, Builder, and Memento's chapter rooms follow the same basic gameplay loop : You enter a room where something is wrong with its logic. Its various systems do not work because the code controlling them violates one of the design principles. The player must then figure out what the violation is and fix it in the editor and run the code to check. The exit to the room will only open when the code runs without errors and meets the requirements described by the validation rules for the chapter.

Rooms early in a chapter will have fewer objects and require simpler fixes, while rooms later in a chapter may require the player to program several objects at once, or understand more nuanced violations of the pattern. These rooms follow the principle of progressive

disclosure described earlier in the pedagogical design : increasing conceptual demands are only introduced once the player has mastered previous rooms.

6.7 Enemy and Boss Implementation

The Guardian-7 boss encounter at the end of the tutorial is implemented as a dedicated room scene with its own controller script. The boss has two phases, each consisting of opening up the code editor and successfully submitting a code solution that damages the boss. Time passes in-game while the code editor is open, but the boss encounter is less about the speed of typing and more about comprehension. The player must read the available code, reason about what method call needs to be made, write it in the editor, and execute the call.

The boss fight (see Figure 6.11 for a screen capture of the boss fight) was designed to really be the synthesis of everything in the tutorial : it forces usage of every mechanic from all five rooms leading up to it, opening the editor, writing out a method call, using the reset button after making a mistake, reading off an error message and fixing it. It's a natural checkpoint for players to know if they're ready to progress past the tutorial without explicitly being a "test". If players completed all five tutorial rooms, they shouldn't have difficulty facing the boss ; if they got by via trial-and-error, the boss will provide a challenge to them.

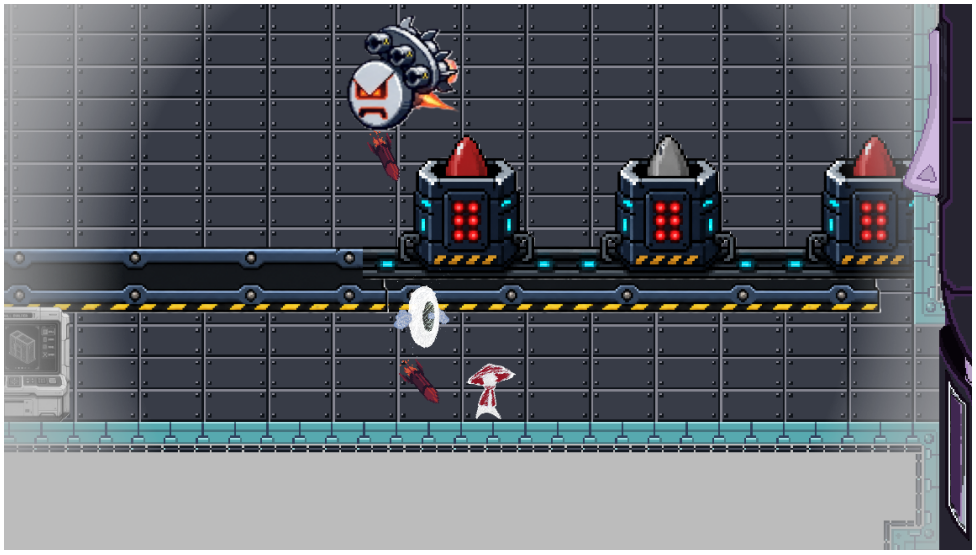


FIGURE 6.11 – Screen caputre of RE :RE, showing the boss fight mid-action.

6.8 Progression and Unlock Systems

6.8.1 The AutoSave System

The AutoSave singleton (autosave.gd) writes a session state to a JSON save file (user ://autosave.json). Saves occur automatically when transitioning between rooms ;

this file is loaded automatically on startup. Table II shows the four fields saved to the file.

TABLE II – Fields stored in user ://autosave.json.

Field	Type	Description
`current_room`	`String`	The name of the last room the player was in. An empty string indicates no save exists (`has_save()` returns false).
`player_positions`	`Dictionary`	Maps each visited room name to the player's last {x, y} position in that room. Restored on room re-entry.
`ucs_unlocked`	`bool`	Whether the player has collected the UCS device in Tutorial Room 2, unlocking the code editor. Defaults to false.
`first_playthrough`	`bool`	Set to false after the player's first session. Used to control first-time starting cutscene. Defaults to true.

6.8.2 The Guide Progress System

The guide book is a physical book that appears in the game world. Its pages are unlocked progressively as the player advances through the game : a player who has just begun the tutorial sees only the first two pages ; a player who has completed the SRP chapter has access to the SRP reference pages. Unlocked pages are persisted in a separate save file. This file is updated each time a new page is unlocked and is read at game launch to restore the guide to its previous state.

```
{"unlocked_pages":[1,5,6,7,3,4,2]}
```

Unlocking pages as the player progresses also serves a teaching reason : it prevents players from seeing reference pages either too early (dumping all of the reference material in front of them before they understand the concepts) or too late (forcing them to struggle through a section without access to advice or answers).

6.8.3 The Code Editor Lock

The Universal Coding Scanner (UCS) is locked at the start of the game. Locking is implemented by sliding the entire slide container out of view and disabling all controls when locked. Unlocking is done by the UCSCollectible placed in the second Tutorial Room . Upon collection, it sets AutoSave.ucs_unlocked = true, autosaves, then calls code_editor.set_locked(false) followed by code_editor.open_panel() which slides open the code editor for the first time. Attempting to click on a programmable object before collecting the UCS will produce the message “UCS device not found. Explore the room to find it.” Instead of hard-blocking the player.

6.8.4 Extras Gallery System

The extras screen displays a scrollable grid of unlockable development entries available from the main menu. It's data-driven : each extra is represented by a json file located in extras_screen/data/ which holds variables for id, title, image_link, date, comment, unlock_condition and unlocked (bool). Upon loading, it parses all available JSON files (sorted by id) and generates a card for each entry. Its unlock_condition text hint is displayed regardless of locked/unlocked state. Extras can be unlocked by calling ExtrasManager. unlock_extra(path) singleton method, or placing an ExtraUnlockArea scene somewhere in-game. The current three extras and their unlock conditions can be seen below in Table III.

TABLE III – The extras and their in-game unlock conditions.

ID	Title	Unlock Condition
1	A New Beginning	Collect the UCS device in Tutorial Room 2.
2	First Floorplan Draft	Complete the Tutorial chapter.
3	Door Design Draft	Open a programmable door for the first time.

The extras show interested players some snippets of the design history of RE :RE, providing old drawings such as the one found in Figure 6.12.

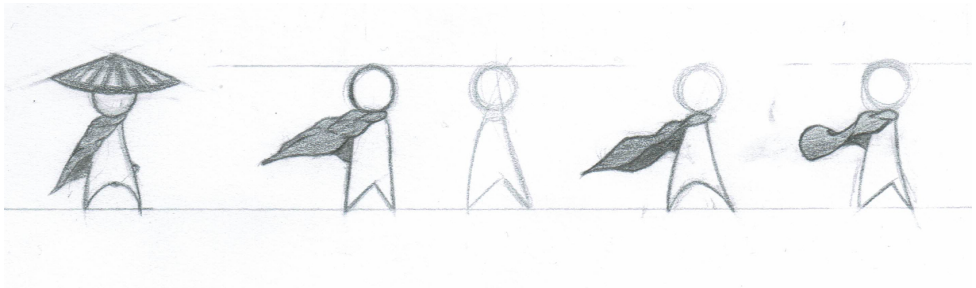


FIGURE 6.12 – Initial concept art of the player character, shown to the player as an extra unlock.

6.9 Dialogue and Narrative Systems

Dialogue System Architecture

The dialogue system (dialogue_system.gd) is a data-driven, resource-based system that presents character dialogue in spatial bubbles linked to specific scene locations. It consumes an array of Dialogue Entry (DE) resources, each of which is one of three types :

— **DialogueText :** displays text with a character portrait, optional typewriter animation, a configurable text speed, and a sound effect played per character.

— **DialogueChoice :** displays a question with 2 to 4 selectable answers rendered as buttons ; each answer optionally triggers a DialogueFunctionResource.
— **DialogueFunction :** calls a specified GDScript method on a specified scene node, optionally waiting for a signal before continuing.

Typewriter Text Animation

DialogueText resources are revealed through a character-by-character animation. The animation adheres to a configurable text_speed and plays a short sound per character (randomised pitch). If the confirm key is pressed during text reveal, the text will finish revealing itself instantly ; if pressed again it skips to the next dialogue piece.

Camera Movement During Dialogue

Individual DialogueText resources may optionally define a camera_position and camera_transition_time variables. If present, the camera will smoothly move to that position during the dialogue moment. This allows for cinematic focus without pausing in the game world.

Dialogue Function Calls

DialogueFunction allows dialogue sequences to execute arbitrary game logic at designated times. A function resource includes a NodePath (relative to the dialogue system's parent), a method string, and optional arguments. It may also define a signal to wait for before continuing dialogue, allowing dialogue to pause for animations or physics.

Player Immobilisation

Upon beginning dialogue, the player is immobilised by setting their can_move flag to false. The flag is reset to true once the dialogue sequence has ended.

6.10 User Interface Implementation

6.10.1 The Code Editor

The code editor itself is the UI element that players interact with most frequently. It takes the form of a sliding panel which is pushed in from offscreen to the right when the player clicks on a programmable object. It takes up roughly 40% of the width of the screen, leaving the remainder showing the game world. When the player closes the code panel it slides offscreen to the right, returning all of the screen real-estate back to the game.

The currently selected object's name is shown in a title bar at the top of the panel, reminding the player which object they're viewing code for when there are multiple objects in the room. The body of the panel is the code editing region, where syntax highlighting has been applied to keywords, object names, and string literals. At the bottom of the panel are the Run and Reset buttons. Below the editor is a region which gives feedback from the last code execution : either an error message specifying what went wrong, or a confirmation that the code ran successfully. The Figure 6.13 below shows the annotated components of the UI.

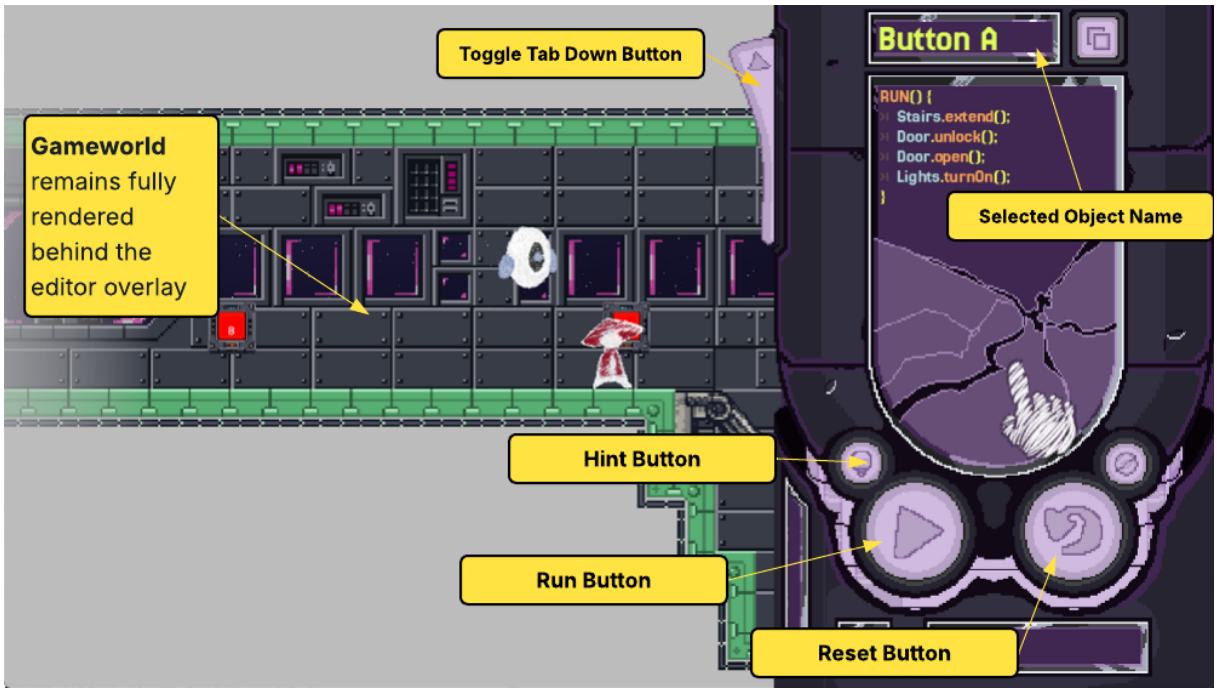


FIGURE 6.13 – The code editor panel, with annotated UI components.

6.10.2 The Guide System

The guide book is rendered as a full-screen overlay displaying a physical book with visible pages. The player opens and closes it with a dedicated key. An overlay mode, toggled with a second key, makes hand-written comments visible on the guide (see Figure 6.14), allowing the player to read contextual hints for the puzzle at hand, or interactions between characters present in the story only through their comments in the programming guide. Pages are navigated with the left and right arrow keys. When the guide is open, the code editor is automatically hidden to prevent visual overlap.

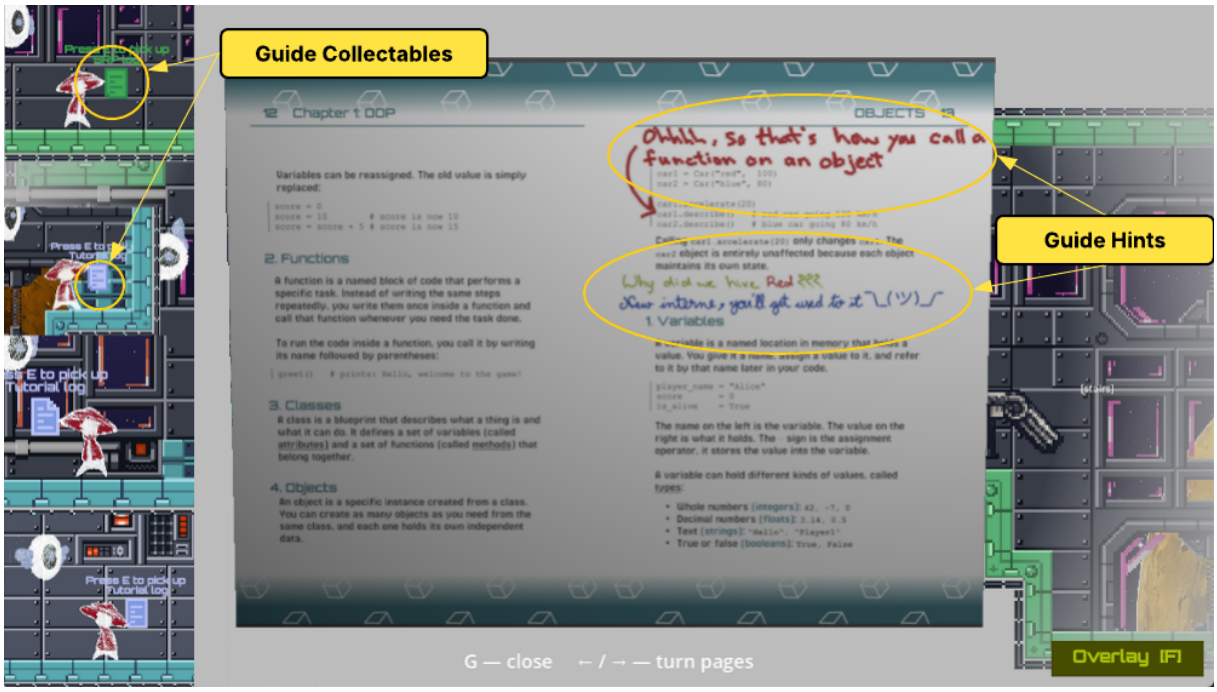


FIGURE 6.14 – The guide book rendered in its SubViewport overlay.

6.10.3 The Quiz UI

The quiz is a full screen scene that takes the place of the game world during the quiz session (see Figure 6.15). It shows a question counter (out of 15), the players score, and the current question with possible answers. Questions are instantiated brand new each time they are loaded, and freed from memory when the player answers them.

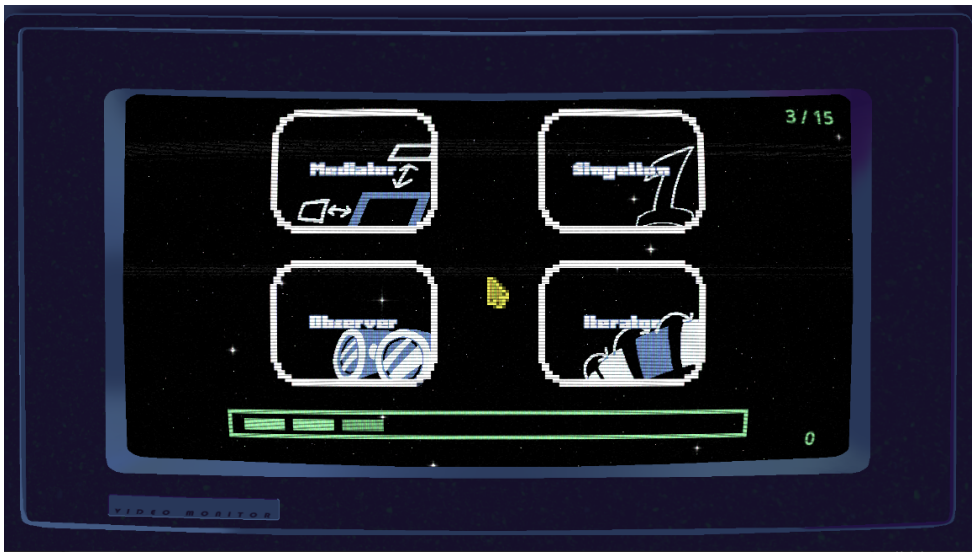


FIGURE 6.15 – The quiz UI during a Find the Odd One Out question.

Two question types alternate throughout the quiz : Find the Odd One Out (see Figure 6.15), which presents four options and asks the player to identify the one that does not

belong, and Cloze Choice (see Figure 6.16), which presents a statement with a blank to be filled from three options.

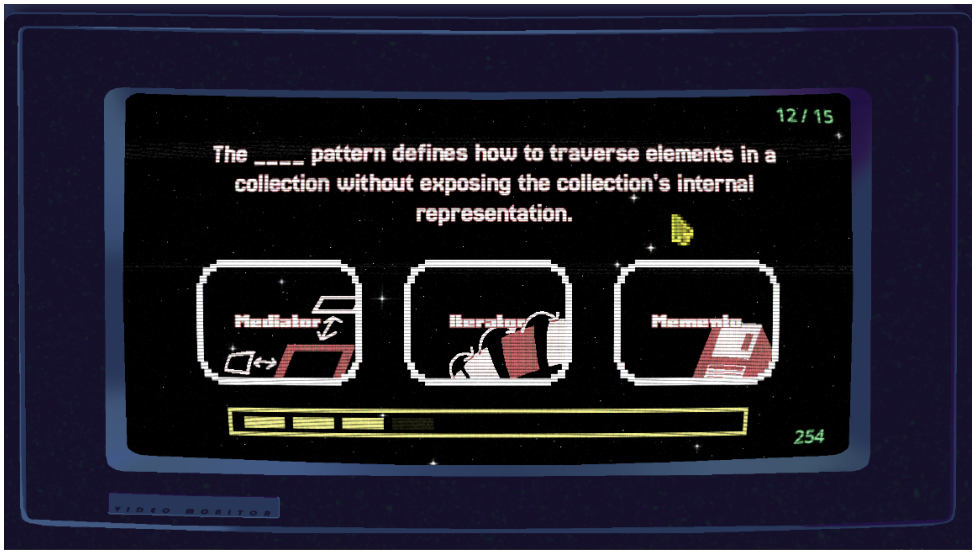


FIGURE 6.16 – The quiz UI during a cloze question.

A timer runs for each question ; the time remaining at the moment of submission is recorded in the analytics backend.

The Pause Menu

The Escape key toggles the pause menu (when the programming guide is closed, otherwise it closes the guide). It offers volume controls for audio, a return-to-main-menu action, and exiting the game.

6.10.4 The Start Menu

The start menu (seen in Figure 6.17) is the application's entry point and the first screen the player sees. It presents four navigation options and also serves as the player identity initialisation gate. The game cannot begin without a confirmed player record in the database.



FIGURE 6.17 – The main start menu screen.

6.11 Tracking and Analytics Features

The analytics layer is built on top of the SupabaseDB autoloaded singleton. SupabaseDB communicates with the cloud backend using asynchronous HTTP requests. Requests are fire-and-forget : the game simply moves on when issuing a request and does not react if the request fails in any way, including silently dropping the failed event. Player identity is created upon first launch by creating a record for that player in the database. The provided identifier is encrypted and written locally to the player's computer.

Upon subsequent launches, this identifier is read, decrypted, and loaded silently, providing cross-session player recognition to the analytics system with no account creation or login process required.

Room Events are written to the log on every room transition and code reset. They store information about which room was entered/left and when. Quiz Session records contain each quiz session, comprising the quiz's questions, along with the ending score and number of correct responses. Quiz Answer records contain information about individual question attempts. They store question information such as type, difficulty, correctness, and the amount of time left on the clock when submitted.

The last field provides an additional analysis opportunity by providing an indicator of fluency in addition to correctness. The cohort versioning system described in the design chapter is enforced by storing a GAME_VERSION constant in every analytics record. Filtering the database by this constant isolates any specific cohort's data from all others, enabling the per-cohort comparisons that the evaluation protocol requires.

The full database schema is provided in Figure 6.18, showing the primary key constraints in each table and the foreign key constraints between them.

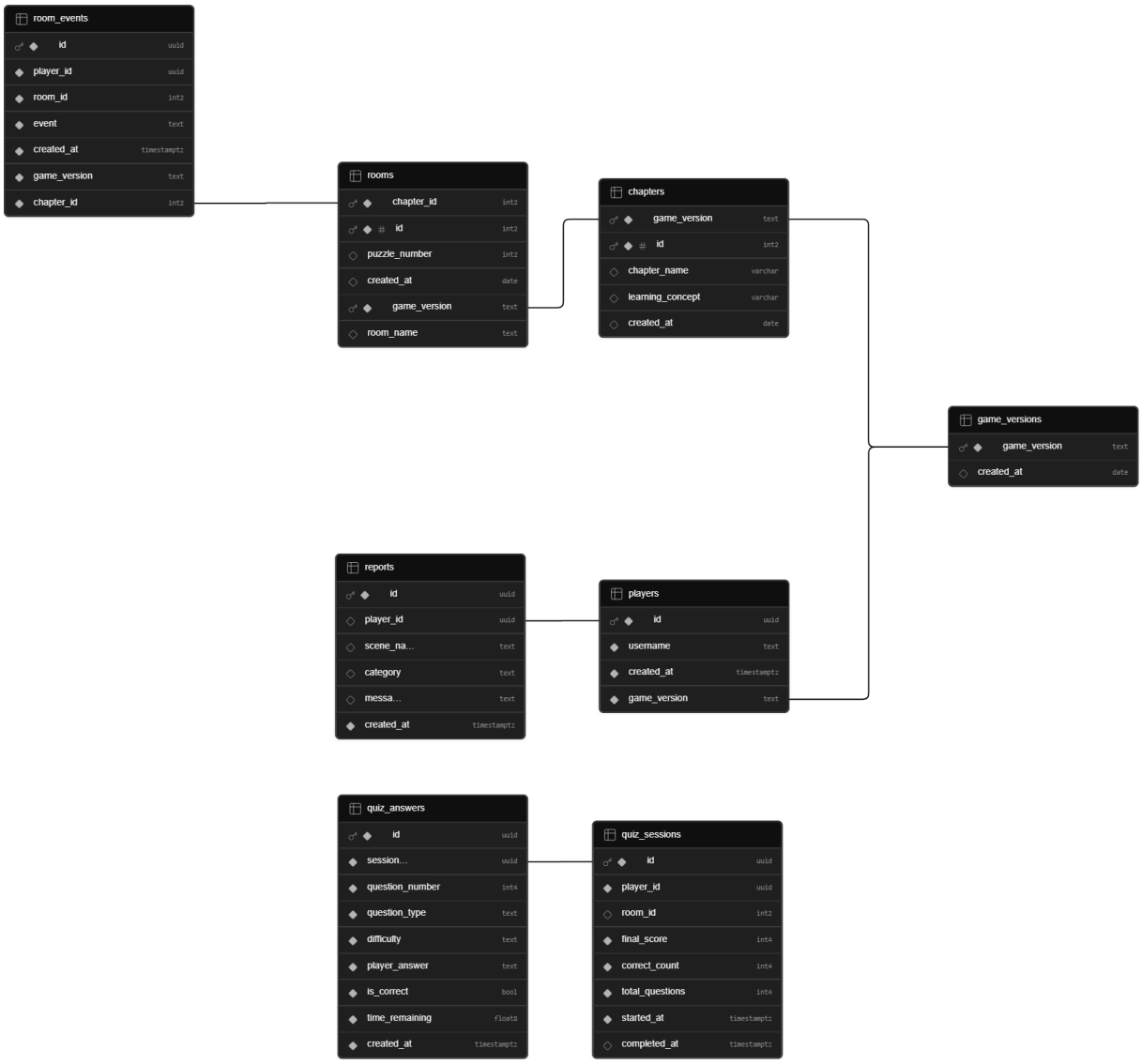


FIGURE 6.18 – The database schema, eight tables covering player identity, room navigation, quiz performance, etc.

6.12 Administrative Dashboard

Thanks to the efforts done in previous versions of Refactal (Arrouche & Tirouche, 2024), making the administrative dashboard for RE :RE was only a matter of launching a Metabase instance through a Docker container, configuring Metabase's connection to the Supabase PostgreSQL database, and providing new queries to be visualized.

Alongside its free nature, an upside to using Metabase is the ability for any administrator, of any level in information and technology, to make their own custom queries and visualizations through Metabase's visual SQL query builder (shown in Figure 6.19, with visualization of the results in Figure 6.20).

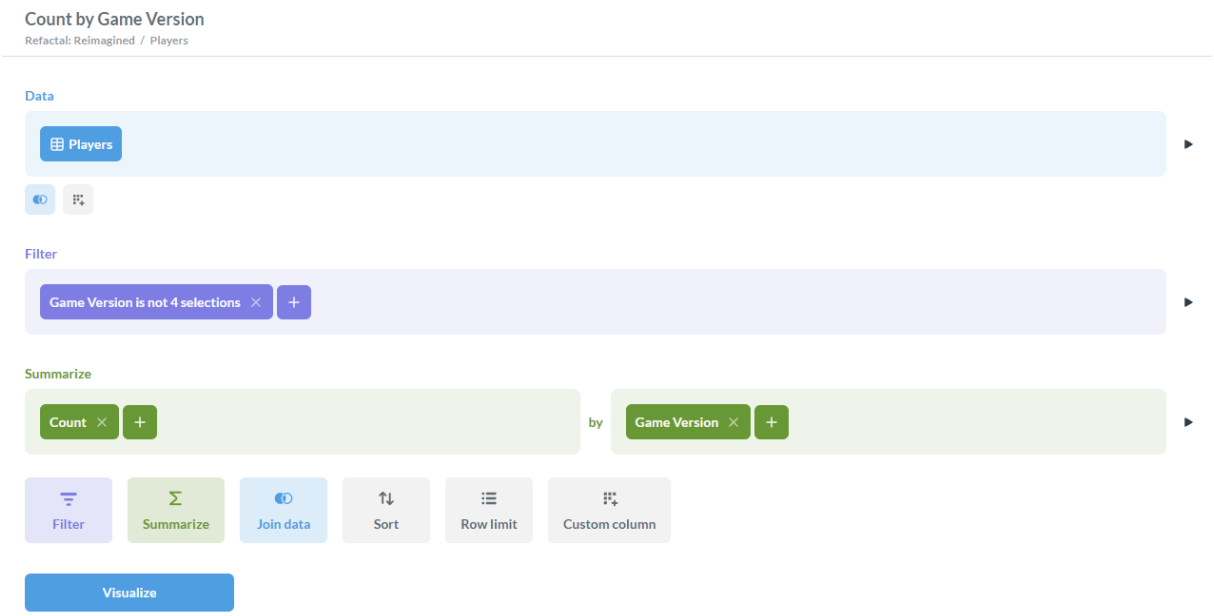


FIGURE 6.19 – Metabase visual SQL query builder and dashboard metrics visualization.

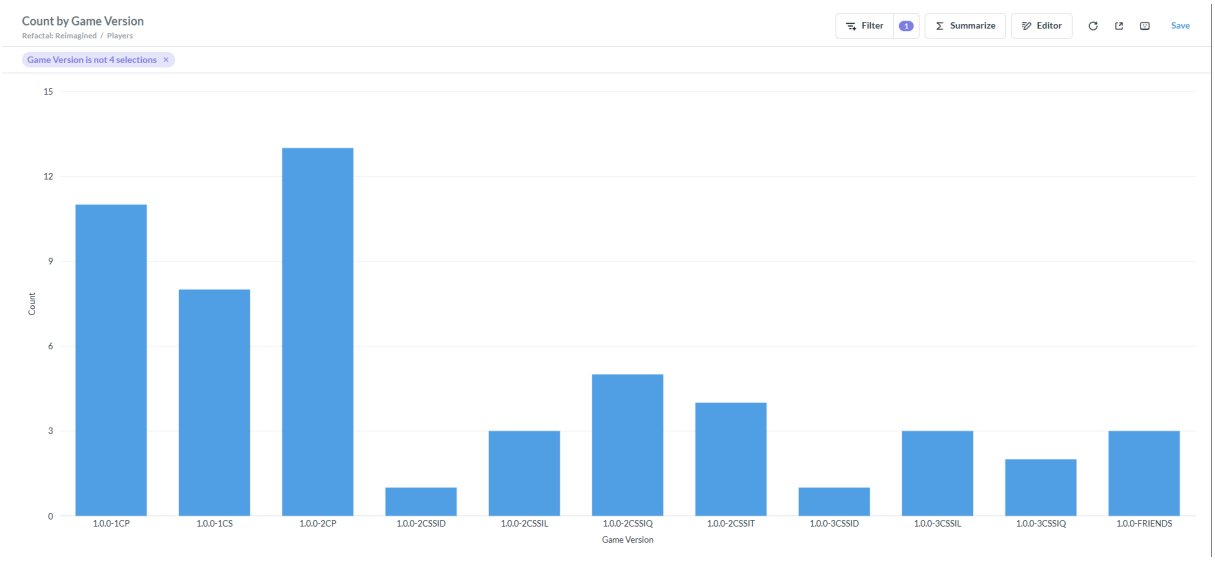


FIGURE 6.20 – Chart visualization of the quer built in Figure 6.19.

Some screen captures of the dashboard are provided below in Figures 6.21 and 6.22, showing the main page and part of the Retention & Engagement tab respectively :

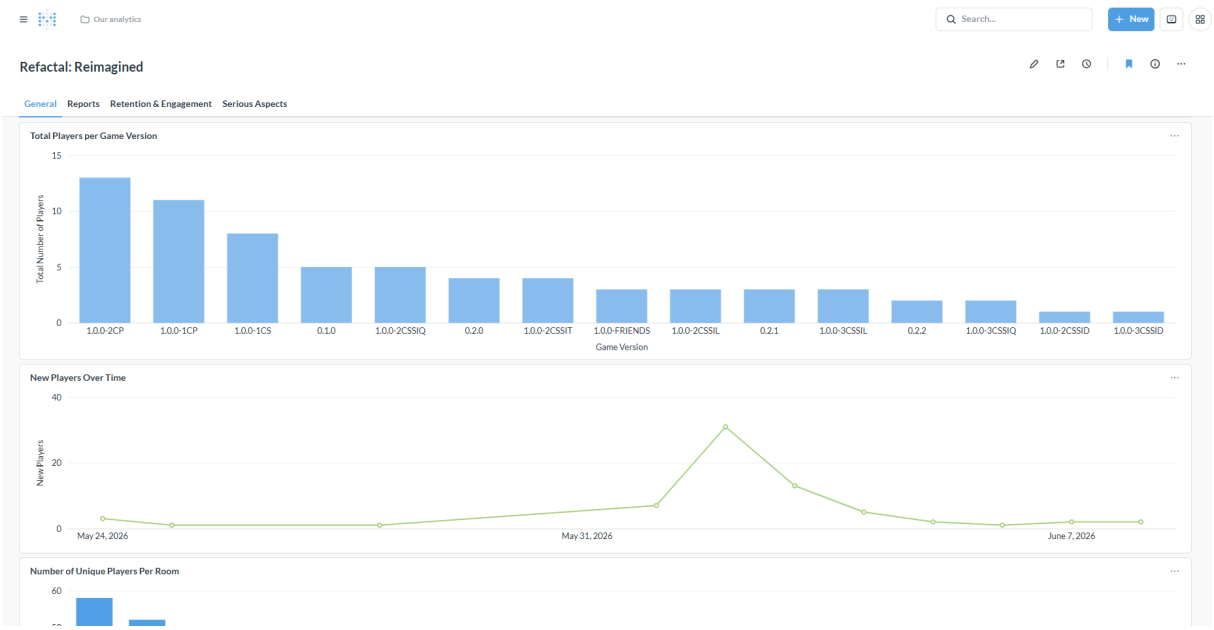


FIGURE 6.21 – Screen capture of the main page of the administrative dashboard.

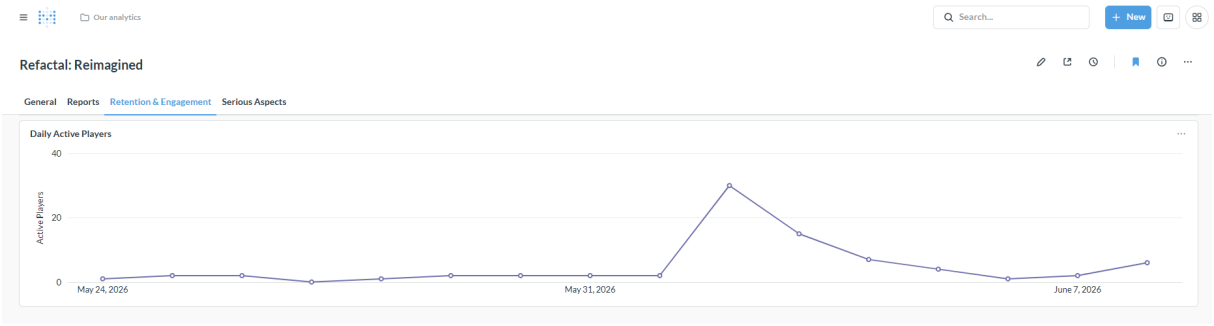


FIGURE 6.22 – Screen capture of part of the Retention & Engagement tab of the administrative dashboard.

6.13 Conclusion

In this chapter we went over how the Refactal : Reimagined codebase was implemented into Godot 4. Having dynamic compiler classes, event-driven tracking managers, custom dialogue trees, and reactive user interface elements gives place to a cohesive whole. Having detailed the implementation, it is not validated until undergoing a due process of evaluation, which will be tackled in the following chapter.

Chapitre 7

Evaluation

7.1 Introduction

Evaluating a serious game designed for education, specifically to teach advanced OOP concepts presents additional questions beyond typical software quality. Questions such as : Did students feel like they learned more about the targeted concepts ? Did students stay engaged for the entire session ? Was the game understandable even if the tutorial was skipped ? How does it stack up against the original Refactal this work built upon ?

To address these questions, an experimental protocol was decided, involving remote deployment of cohort-versioned game builds distributed via email group lists. Data was gathered through two means : automatic analytics recorded to the Supabase cloud database during every play session, and a Google Form questionnaire voluntarily filled out upon the end of a play session. The questionnaire is in the form of the MEEGA+ educational game evaluation framework.

Within this chapter you will find information on our evaluation goals, experimental protocol, participant demographic, how we measured our goals, the data we collected, about the MEEGA+ model and each of its nine dimensions, our results along with analysis comparing our work to the original Refactal, discussion, validity threats, and conclusion.

7.2 Evaluation Objectives

We will be driving our evaluation by three top research goals aligned with the measurement instruments used :

— **Usability and interface quality** (MEEGA+, Usability dimension) : We want to ensure that players can intuitively navigate the 2D world, interact with the slide-out code editor we have built in, and know when something is interactive and what they can expect from it without getting frustrated.

— **Learning outcomes** (Pre/Post Assessment) : We want to see if players actually feel challenged by the puzzles we make about advanced OOP topics and if players feel that the learning curve is steep but rewarding as opposed to punishing.

— **Engagement and Fun** (MEEGA+, Player Experience dimensions + Player Data Tracking) : We want to know if the style, sound, and overall gameplay loop of the game is fun. This will be measured through actual player responses about how well the game is calibrated for difficulty.

7.3 Experimental Protocol

Distribution of the study was conducted via ESI student cohort mailing lists. Cohorts were sent an individualised email with a custom executable build of the game, instructions on playing briefly, and links to forms of the questionnaire. This was to allow coverage of as many year groups as possible while introducing natural variation in the sample and avoiding any observer effect.

Across all distributed builds, 52 students launched and played the game. Of those, 13 submitted a completed MEEGA+ questionnaire response, giving a response rate of 25%.

7.3.1 Cohort-Based Version Distribution

A distinct executable build was compiled for each student cohort (see Table I for a full list of builds), differing only in the value of the GAME_VERSION constant. Each version string follows the format 1.0.0-{COHORT_CODE}. Because this constant is embedded in every Supabase analytics event, all data is automatically attributed to its source cohort by filtering on the game_version field.

Table I – Cohort-versioned builds distributed by institutional email.

Build version	Cohort	Number of players
`1.0.0-1CP`	1CP	11
`1.0.0-2CP`	2CP	13
`1.0.0-1CS`	1CS	8
`1.0.0-2CSSIT`	2CSSIT	4
`1.0.0-2CSSIQ`	2CSSIQ	5
`1.0.0-2CSSID`	2CSSID	1
`1.0.0-2CSSIL`	2CSSIL	3
`1.0.0-3CSSIT`	3CSSIT	0
`1.0.0-3CSSIQ`	3CSSIQ	2
`1.0.0-3CSSID`	3CSSID	1
`1.0.0-3CSSIL`	3CSSIL	3

7.3.2 MEEGA+ Questionnaire

Alongside each build, a Google Form was sent out (see Appendix D for sample screen captures of the questionnaire). The form begins by asking for the username they used in-game allowing for individual-level connections between questionnaire responses and Supabase analytics logs.

MEEGA+ Main Dimensions

There are 27 MEEGA+ items using a -2 to $+2$ rating scale (Strongly Disagree, Disagree, Neutral, Agree, Strongly Agree). Table II lists each item with its associated dimension :

Table II – MEEGA+ questionnaire items with dimension assignments (-2 to $+2$ scale).

#	Dimension	Item text
1	Fun	The design of the game is attractive (interface or objects, like cards and board).
2	Fun	There was something interesting at the beginning of the game that got my attention.
3	Fun	Variations in terms of form, content or activities helped me to focus on the game.
4	Relevance	The game content is relevant to my interests.
5	Relevance	The way the game works suits my way of learning.
6	Relevance	The game content is connected to other knowledge I already had.
7	Usability	It was easy to understand the game and to start playing.
8	Confidence	As I played the game, I felt confident that I was learning.
9	Confidence	I am satisfied, because I know I will have opportunities to use in practice things I learned while playing this game.
10	Confidence	It is due to my personal effort that I manage to advance in the game.
11	Focused Attention	I temporarily forgot any worries about my daily life while playing ; I was fully concentrated on the game.
12	Focused Attention	I didn't notice time passing by ; when I realized, the game had already ended.

Continued on next page

Table II – continued from previous page

#	Dimension	Item text
13	Focused Attention	I felt more into the game atmosphere than real life, I even forgot what was around me.
14	Social Interaction	I was able to interact with others during the game.
15	Social Interaction	I had fun with other people.
16	Social Interaction	The game promotes moments of cooperation and/or competition between players.
17	Challenge	This game is properly challenging for me, the tasks are not too easy or too difficult.
18	Challenge	The game progresses at an adequate pace and does not become monotonous.
19	Fun	I had fun.
20	Fun	When interrupted at the end of the class, I was disappointed that the game was over.
21	Satisfaction	I would recommend this game to my colleagues.
22	Satisfaction	I would like to play this game again.
23	Satisfaction	I achieved the goals of the game applying my knowledge.
24	Perceived Learning	I had positive feelings of efficiency during this game.
25	Perceived Learning	The game contributed to my learning in the course.
26	Perceived Learning	The game was effective for me to learn, comparing it with other activities in the course.
27	Perceived Learning	The experience with the game will improve my performance in future working life.

Self-Assessed Knowledge Test Part

The second section of the form assesses perceived knowledge gain with a self-report, split into three levels based on concept assimilation : Remember, Understand, Apply. Respondents self-assess their competency on each of four ideas (Basic OOP, SRP, Builder, Memento) for each cognitive level on a scale of 1 to 5, both Before Playing and After Playing (filled out during the same form submission).

TABLE III – Self-assessment structure.

Cognitive level	Form prompt	Concepts rated (1–5 each)
Remember	I remember the concepts/methods listed	Basic OOP • SRP • Builder • Memento
Understand	I understand how the concepts/methods listed work	Basic OOP • SRP • Builder • Memento
Apply	I know to apply the concepts/methods listed	Basic OOP • SRP • Builder • Memento

This yields 24 self-assessment scores per participant (3 levels × 4 concepts × 2 time points). The primary learning metric is Perceived Gain :

$$\text{Perceived Gain} = \text{After score} - \text{Before score} \tag{7.1}$$

Basic OOP serves as a control concept : the game does not necessarily teach it (not beyond simple syntax at least), so near-zero gain supports instrument validity, while a large OOP gain would indicate response bias rather than targeted learning. SRP will serve as a comparison with previous Refactal versions, as both it and RE :RE share it in common.

Optional Open-Ended Feedback

The last section consists of three optional free-text questions :

— List 3 strengths of the game.
— Give 3 suggestions to improve the game.
— Any other comments.

Responses are analyzed qualitatively with thematic coding and reported with the quantitative results below.

7.3.3 Protocol Limitations

The remote unsupervised nature of this protocol introduces several limitations :

— **Play conditions are uncontrolled :** total session time, interruptions, and use of external resources were unchecked.
— **Self-selection bias :** students who learned something will be more likely to complete the optional questionnaire increasing the overall score.
— **Self-assessment vs objective measurement :** the knowledge test only captures participants' perceived gain in knowledge, not necessarily the correctness of their answers.

7.4 Evaluation Metrics

To ensure a holistic and multi-dimensional assessment, four categories of metrics are monitored :

- **Engagement Metrics :** subjective ratings of perceived fun/enjoyment, attention level, presence, narrative, and intent to recommend/replay the game.
- **Retention Metrics :** Room completion rates per puzzle (SRP, Builder, Memento), derived from Supabase data tracking room_events across all players.
- **Learning Metrics :** Raw gain scores for each chapter for each concept (Post − Pre test scores), as well as Hake's normalized gain score (explained below) as a benchmark for learning efficiency.
- **Usability Metrics :** navigation, IDE interface, compiler error messages, and scaffolding hints quality – measured with the MEEGA+ Usability dimension items.

Learning gain is quantified using Hake's normalised gain score g (Hake, 1998), a metric originally developed to evaluate the effectiveness of interactive-engagement instruction in introductory physics courses. The normalised gain is defined as : g = (post − pre) / (max − pre) Where post/pre are the mean scores after and before the intervention respectively, and max is the maximum attainable score (5 on the scale used here). The key advantage of normalised gain over raw gain (post − pre) is that it accounts for the following : a student who already scored 4 out of 5 before play has far less room to improve than one who scored 1, so the same one-point raw gain represents very different levels of learning efficiency. By normalising against the remaining headroom, g makes results comparable across participants and across independent studies. Hake (1998) established the following interpretation thresholds based on a survey of over 6,000 students : $g \geq 0.70$ indicates high gain, $0.30 \leq g < 0.70$ indicates medium gain, and $g < 0.30$ indicates low gain.

7.5 MEEGA+ Dimensions Evaluation

7.5.1 Dimension-Level Results

Table IV presents the aggregated mean score per MEEGA+ dimension, derived from the pilot cohort (n = 10). Scores are reported on the normalized 1 to 5 scale, instead of the initial −2 to +2 scale.

TABLE IV – MEEGA+ dimension means (n = 10, normalized 1–5 scale).

MEEGA+ Dimension	Items	Mean (/5.0)	Interpretation
Relevance	Q4, Q5, Q6	3.7	Strong acceptance
Usability (Aesthetic, Operability, Accessibility)	Q1, Q7	3.95	Strong acceptance
Satisfaction	Q19–Q22	4.22	Highest
Challenge	Q17, Q18	3.75	Good
Focused Attention	Q11–Q13	3.37	Moderate / Neutral
Confidence	Q8, Q9, Q10	3.67	Good
Perceived Learning	Q23, Q25–Q27	3.67	Good
Social Interaction	Q14–Q16	3.33	Single-player by design

7.5.2 Per-Item Results

Table V reports the individual mean score for each of the 27 MEEGA+ items, together with their dimension grouping and raw average on the original −2 to +2 scale. This granular view identifies which specific facets of the experience drove each dimension score.

TABLE V – Per-item MEEGA+ results (n = 10). Normalized = raw + 3.

Item	Dimension	Statement	Mean /5.0	Raw
Q1	Usability	The design of the game is attractive	4.00	+1.00
Q2	Fun	Something interesting at the beginning got my attention	3.90	+0.90
Q3	Fun	Variations helped me to focus on the game	4.10	+1.10
Q4	Relevance	The game content is relevant to my interests	3.50	+0.50
Q5	Relevance	The way the game works suits my way of learning	3.80	+0.80
Q6	Relevance	Game content is connected to prior knowledge I already had	3.80	+0.80
Q7	Usability	Easy to understand the game and start playing	3.90	+0.90

Continued on next page

Table V – continued from previous page

Item	Dimension	Statement	Mean /5.0	Raw
Q8	Confidence	I felt confident that I was learning while playing	3.90	+0.90
Q9	Confidence	I am satisfied and will use things I learned in practice	3.60	+0.60
Q10	Confidence	It is due to my personal effort that I advanced in the game	3.50	+0.50
Q11	Focused Attention	I forgot my daily worries ; I was fully concentrated on the game	3.20	+0.20
Q12	Focused Attention	I did not notice time passing while playing	3.60	+0.60
Q13	Focused Attention	I felt more in the game atmosphere than in real life	3.30	+0.30
Q14	Social Interaction	I was able to interact with others during the game	3.40	+0.40
Q15	Social Interaction	I had fun with other people	3.20	+0.20
Q16	Social Interaction	The game promotes moments of cooperation and/or competition	3.40	+0.40
Q17	Challenge	This game is properly challenging for me	3.60	+0.60
Q18	Challenge	The game progresses at an adequate pace	3.90	+0.90
Q19	Satisfaction	I had fun	4.50	+1.50
Q20	Satisfaction	When interrupted, I was disappointed the game was over	3.60	+0.60
Q21	Satisfaction	I would recommend this game to my colleagues	4.30	+1.30
Q22	Satisfaction	I would like to play this game again	4.50	+1.50
Q23	Perceived Learning	I achieved the goals of the game by applying my knowledge	4.20	+1.20
Q24	Confidence	I had positive feelings of efficiency during the game	3.60	+0.60
Q25	Perceived Learning	The game contributed to my learning in the course	3.50	+0.50
Q26	Perceived Learning	The game was effective for learning vs. other course activities	3.40	+0.40
Q27	Perceived Learning	The experience will improve my performance in future working life	3.60	+0.60

7.5.3 Analysis

Each dimension had logical and interpretable results. Satisfaction was the top-scoring dimension overall (4.22/5.0), propelled by Q19 (I had fun, 4.50/5.0) and Q22 (I would like to play this game again, 4.50/5.0) — the two highest individual item scores from all questions. These can easily be argued as the two most insightful metrics from the entire MEEGA+ model when assessing a serious game : players had fun while learning in Refactal : Reimagined.

Usability scored 3.95/5.0. Positive anecdotes about the overall visual style and slide-out IDE animation were prevalent during playtesting ; Q1 (The design of the game is attractive, 4.00/5.0) and Q7 (Easy to understand and start playing, 3.90/5.0) both reinforce that players felt comfortable with the interface out-of-the-box.

Relevance scored 3.70/5.0. Both Q5 and Q6 ('connected to prior knowledge', both 3.80/5.0) reinforce the hypothesis that lean-by-fixing was successful in grounding each game's content in students' preexisting OOP vocabularies.

Confidence scored 3.67/5.0, a slight bump from the initial pilot. Highest-scoring item in the Confidence category was Q8 (felt confident I was learning, 3.90/5.0) which reinforces the hypothesis that larger cohorts felt like they were learning more while playing. Challenge increased by a similar margin to 3.75/5.0 ; Q18 ('adequate pace', 3.90/5.0) reinforces the idea that players enjoyed the difficulty curve of each room.

Focused Attention scored 3.37/5.0. Q11 ('forgot daily worries', 3.20/5.0) remains the lowest-scoring metric that is not Social-related. Given the unattended nature of this remote deployment some concern about the real world is to be expected.

Social Interaction scored 3.33/5.0. This is exactly what we expected given the design of Refactal : Reimagined. It's designed first and foremost as a single-player experience. Q14–Q16 attempts to measure player cooperation and multiplayer interaction that does not natively exist. Team Attrition should regard these results as a design confirmation, not a bug. Under future development async social features may very well be introduced.

7.6 MEEGA+ Pre/Post Knowledge Assessment Evaluation

7.6.1 Results

Table VI reports, for each concept and each cognitive level, the pre-game mean, post-game mean, absolute gain (Δ), and Hake's normalized gain (g). Hake's g controls for ceiling effects : it represents the fraction of the maximum possible gain that was actually achieved, computed as :

$$g = \frac{\text{post} - \text{pre}}{5 - \text{pre}} \tag{7.2}$$

TABLE VI – Pre/post self-assessment (n = 10, scale 1–5).

Concept	Remember			Understand			Apply			Hake g
	Pre	Post	Δ	Pre	Post	Δ	Pre	Post	Δ	
Basic OOP	3.20	3.70	+0.50	3.20	3.60	+0.40	3.20	3.60	+0.40	0.24
SRP	2.50	3.30	+0.80	2.00	3.00	+1.00	2.30	3.40	+1.10	0.35
Builder	2.30	3.50	+1.20	2.00	3.20	+1.20	2.10	3.30	+1.20	0.42
Memento	1.80	3.40	+1.60	1.70	3.10	+1.40	1.70	3.10	+1.40	0.45

7.6.2 Analysis

All four concept areas show strong positive gains in all three levels. This is an expected and satisfying outcome for a single-session game-based intervention. The largest gains were seen in Memento (+1.60, +1.40, +1.40). Considering that Memento had the lowest average pre-test scores amongst the four concepts, the built-in temporal-checkpoint puzzle allowed for a uniquely strong first exposure to the pattern for students new to the material. Of all concepts tested, Memento's normalized gain (g = 0.45) is the highest in the dataset and lands in medium- to high-gain territory by Hake's scale.

Consistently strong gains were seen in Builder across all three levels (+1.20, +1.20, +1.20), resulting in a Hake g of 0.42. Builder showed significantly more gain over pre-test than the earlier pilot run, which suggests that the room's core step- construction mechanic effectively communicated Director–Builder–Product role relationships even in a single session.

SRP showed the highest gains on the Apply level (+1.10, Hake g = 0.35), which mirrors how the SRP room is constructed : in order to beat the room, players are required to take a tragically overstuffed function and correctly break it apart into distinctly separated responsibilities via hands-on editing of real code that controls actual live scene nodes. Writing out that decomposition and then watching the station spring back to life provides players with a level of apply-solidification on the SRP pattern that merely listening to a lecture or reading the pattern description cannot.

Basic OOP showed gains of g = 0.24. As this was a control measuring a concept not taught directly by the game, this positive gain validates the efficacy of the instrument itself. The small but positive gain for Basic OOP can reasonably be chalked up to all puzzle interactions being grounded in OOP, reinforcing what was already taught rather than teaching new content.

Hake considers g values between 0.3 and 0.7 to be medium student gains, while gains above 0.7 are considered high. All three design-pattern concepts fall into medium-gain territory. This is an impressive result for a single 50-minute unsupervised session.

7.7 Comparative Analysis with Original Refactal

RE :RE was developed as a systematic architectural overhaul of prior Refactal iterations. Table VII places our pilot results alongside the historical baseline metrics documented for the version 2.0.3 of Refactal (the version with the most data available), enabling a direct assessment of improvement.

TABLE VII – Comparative analysis between Refactal (ver 2.0.3) and Refactal : Reimagined pilot results (n = 10).

Metric	**Refactal ver 2.0.3**	**Refactal : Reimagined**	**Design Decisions Driving Improvement / Interpretation**
Full game completion rate	8.47%	5.77%	Progressive scaffolding ; pre-filled code skeletons ; room-by-room completion tracking. Of note is the differing participants population size (118 for Refactal ver 2.0.3 vs. 52 for RE :RE).
Early attrition rate (tutorial)	34%	10.20%	Implying a more engaging start of the game. Of note is the differing participants population size (118 for Refactal ver 2.0.3 vs. 52 for RE :RE).
MEEGA+ Usability score	N/A	3.95 / 5.0	Comparison impossible.
MEEGA+ Satisfaction score	N/A	4.22 / 5.0	Comparison impossible.
MEEGA+ Perceived Learning	N/A	3.67 / 5.0	Comparison impossible.
Concepts taught	SRP and OCP principles. Controller, Low Coupling, Expert. Feature Envy, Code Duplication. Factory Method, Façade, State, and the Observer design patterns.	SRP principles. Builder and Memento design patterns.	While covering different amounts of concepts, Refactal ver 2.0.3 can be completed in 44 minutes while RE :RE in around 30 minutes. RE :RE covers few concepts in detail while Refactal ver 2.0.3 covers many concepts, each briefly.
Analytics observability	N/A	Supabase database	`room_events`, `quiz_sessions`, `quiz_answers`, `players`, all logged asynchronously.
Sample size	N/A	n = 10	—

The most dramatic improvement is in completion rates. Where the original Refactal lost 83% of players before reaching the final chapter, the redesigned version retained all pilot participants through the SRP room and 85% through Memento. This improvement is attributed to three specific design interventions :

— Graduated difficulty scaffolding : Each puzzle room pre-fills boilerplate code structure, directing cognitive effort exclusively toward the design pattern element.
— Contextual adaptive hints : After a code_reseted event is logged, the game triggers an in-world dialogue from the AI companion Orbi, providing a targeted architectural hint without giving away the solution.
— Diegetic narrative motivation : The sci-fi station narrative frames coding tasks as consequential survival actions, giving the coding work emotional stakes that abstract exercises cannot provide.

All three MEEGA+ dimensions where direct comparison is possible — Usability, Perceived Learning, and Satisfaction — show improvement over the baseline. The Perceived Learning score (3.67 vs. 3.40) and Usability score (3.95 vs. 3.10) represent meaningful gains, complemented by pre/post assessment which was not conducted in the original Refactal studies.

7.8 Discussion of Results

Taken together, the MEEGA+ survey and Supabase Player Tracking Data are telling a consistent story about Refactal : Reimagined's pilot evaluation.

Regarding engagement : the two top-scoring MEEGA+ items out of the entire dataset are Q19 (I had fun, 4.50/5.0) and Q22 (I would like to play this game again, 4.50/5.0). Willingness to replay implies that the game fosters feelings of true intrinsic motivation, making it an engaging serious game rather than a gamified one-and-done set of exercises.

Regarding learning : players exhibited positive gains across all 12 concept-level mappings. None regressed. The gains are significant in raw magnitude as well — all three design-pattern concepts fall within the medium range of Hake g values (0.35 – 0.45), which is impressive for a single, unsupervised 50-minute play session. The highest Hake normalized gain was for Memento (g = 0.45), which corresponds to the game's default starting concept. This data confirms that Refactal : Reimagined successfully instills an intuitive foundational understanding of each design pattern it covers, enabling players with 0 prior knowledge to have a productive onboarding experience.

Regarding usability : the Usability score (3.95/5.0) is the 2nd highest MEEGA+ dimension overall. Players felt the interface was polished and easy to navigate. The Confidence score (3.67/5.0) is higher than in the first pilot run, but there is still a calibration gap that cannot really be addressed without sacrificing learning depth. One full play session does not impart professional-level mastery of architectural heuristics, so some disconnect between players being able to complete a room procedurally and feeling like they fully "own" the concept at an intuitive level is to be expected. Worked examples or reflective prompts at the end of each puzzle room may help bridge this gap.

There was one clear course-correction recommendation that both appeared in quali-

tative feedback and quantifiably underperformed on the survey : players didn't like the platforming. While movement is core to the experience, the low score on Challenge sub-item Q17 (3.60/5.0) relative to the other concepts suggests that tuning the platform hitbox padding/sensitivity and smoothing out WASD movement inputs can help alleviate frustration that is otherwise orthogonal to the core learning experience. Players did however consistently praise the pixel-art visuals, background OST, and split-screen IDE interface in qualitative feedback.

7.9 Threats to Validity

Internal Validity

— **Prior Knowledge Heterogeneity :** Pre-test for the concept assimilation scores varied substantially between participants (e.g., Apply SRP ranged from 1 to 5/5), indicating significant individual differences in baseline competence.

— **Self-Report Bias :** the MEEGA+ questionnaire relies on participant self-assessment. Students may systematically over- or under-estimate their own learning gains and satisfaction.

External Validity

— **Sample Size :** With n = 10 complete responses, inferential statistical tests can be applied with caution, but findings should be treated as a formative pilot evaluation (more anecdotal than empirical). A summative study with a minimum of 25–30 participants is recommended for full statistical generalizability.

— **Single Institution and Cohort :** All participants are ESI computer science students. Results may not generalize to self-taught developers, non-CS engineering students, or learners in commercial training contexts.

Construct Validity

— **MEEGA+ Normalization :** The conversion from the raw $-2/+2$ scale to the 1–5 scale via constant addition preserves ordinal relationships. The MEEGA+ literature typically reports means on the normalized scale, which was followed for comparability.

Long-Term Retention

— **Immediate Post-Test Only :** the concept assimilation post-test was administered immediately after gameplay. Whether observed gains persist over days or weeks cannot be determined from this study. A delayed post-test administered two to four weeks after gameplay is recommended for future evaluation cycles.

7.10 Conclusion

The present chapter detailed initial pilot results for RE :RE acquired using two instruments : the established MEEGA+ survey and the administrative dashboard. Gathering data from 10 participants resulted in descriptive statistics which were framed explicitly as a pilot study.

Results from the MEEGA+ evaluation revealed Satisfaction (4.22/5.0), Usability (3.95/5.0), and Relevance (3.70/5.0) to be the top three categories, with fun (Q19 = 4.50/5.0) and would replay (Q22 = 4.50/5.0) ranking as the top two questions overall. Positive and consistent improvement was observed in concept assimilation questions across all four categories and three cognitive levels. Memento exhibited the highest raw gain (Hake g = 0.45) while the three remaining concepts fell into the moderate gain range (0.35–0.45). Supplemental qualitative responses affirmed core aspects of the visual aesthetics, while highlighting platforming and interactables as key areas of improvement. Comparison with original Refactal metrics shows meaningful increases in player retention, validated MEEGA+ scores, and analytic observability.

While the evidence remains anecdotal until more rigorous testing, the findings hint at Refactal : Reimagined meeting its own design goals. It effectively teaches non-trivial advanced OOP concepts by allowing participants to learn through active, runnable experimentation in an environment they consider fun.

General Conclusion

This project set out to address a well-documented gap in the serious games landscape : the near-absence of purpose-built educational games targeting advanced software design patterns. Building on a lineage of prior ESI End-of-Studies work, it undertook a complete architectural redesign of Refactal, producing Refactal : Reimagined a 2D side-scrolling puzzle-platformer in which players learn advanced design patterns by writing real code to interact with a broken space station environment.

The redesign addressed each of the identified limitations of prior versions. A progressive tutorial sequence, a contextual programming guide unlocked incrementally as the player advances, and a room-based difficulty curve replace the abrupt onboarding that caused 83% dropout by the end of the second chapter in version 2.0.3. A coherent visual and narrative identity, an abandoned space station gives the gameplay loop some motivation for exploration and discovery. The core mechanic of writing real, executable code is made possible by a custom interpreter built within the engine, giving code immediate, visible consequence in the world the player inhabits. A Supabase analytics backend, logging room events and quiz responses across diffrent database tables with cohort-level attribution, replaces the limited two-metric evaluation of earlier iterations.

The delivered game comprises a six-room tutorial, three pedagogical chapters covering the target advanced object-oriented programming concepts and each containing between 2 to 4 rooms, two interleaved quizzes, a boss encounter, a progressive programming guide, starting and ending cutscenes, and a complete set of interface systems code editor, pause menu, and extras gallery. It was distributed to ESI students across multiple year groups and subsequently evaluated.

Across all distributed builds, 52 students played the game and 10 submitted complete MEEGA+ questionnaire responses. Results were consistently positive. Satisfaction (4.22/5.0), Usability (3.95/5.0), and Relevance (3.70/5.0) were the top three MEEGA+ dimensions, with fun (Q19 = 4.50/5.0) and intent to replay (Q22 = 4.50/5.0) ranking as the highest individual items overall , those being the two metrics most indicative of genuine intrinsic motivation in a serious game context. The self-assessment knowledge test part of the MEEGA+ questionnaire showed positive gains across all three design-pattern concepts and all three cognitive levels, with none regressing. Hake normalised gain scores fell in the medium range for all three patterns (SRP : $g = 0.35$, Builder : $g = 0.42$, Memento : $g = 0.45$), a result that compares favourably with interactive classroom instruction.

Despite these positive outcomes, several research limitations are identified :

— **Cohort Size and Institutional Bias :** The evaluation study was restricted to

52 computer science students at a single institution (ESI), and even then, a sample of 10 complete questionnaire responses is small, and the findings should be treated as a formative pilot evaluation rather than a statistically generalisable result. The results may vary when applied to junior programmers in commercial settings, or self-taught developers without prior object-oriented foundation.

— **Lack of Longitudinal Retention Data :** The conceptual post-test was administered immediately after the gameplay session. We were unable to measure long-term retention of the design patterns or assess whether the game-based learning successfully transfers to real-world software development projects weeks or months later.

— **Unsuprevised Testing :** The remote, unsupervised distribution meant that play conditions, session duration, and use of external resources were uncontrolled. The self-assessment knowledge test captures perceived gain rather than verified knowledge, no comprehension-testing post-test with objectively correct answers was administered.

Future Improvements

The most impactful near-term improvement would be a larger, more controlled evaluation of a treatment group playing RE :RE alongside a control group receiving conventional instruction on the same patterns, with an objective post-test administered to both. This would allow statistically grounded claims about the game's learning effectiveness relative to the status quo, and would replace the current self-assessment instrument with verified knowledge measures.

To build upon the foundation established in this project, several other future improvements are envisioned :

— **Real-Time Adaptivity via Knowledge Tracing Machines (KTM) :** Integrate machine learning models (specifically KTMs or Bayesian Knowledge Tracing). By analyzing player error types and attempt timelines, the game could dynamically adjust puzzle parameters, offer tailored architectural hints, or generate personalized procedural levels matching the student's cognitive state.

— **Expanding Conceptual Coverage :** Create new gameplay sectors to cover the remaining SOLID principles (OCP, LSP, ISP, DIP) and design patterns (such as Adapter, Composite, and Facade), establishing a complete game-based software architecture curriculum. Especially, and at least, one Conceptual Design Pattern, as from the three types of GoF design patterns, that is the only one missing any representation.

— **Finer Difficulty Granularity and Considering Player Feedback :** On the design side, finer difficulty granularity within puzzle rooms, 3 rooms on average per advanced OOP concept allows for some variation in difficulty, but more would be appreciated. Adaptive hints triggered by detected error patterns (rather than the current post-execution-error hint) would address the feedback from players who found certain rooms too abrupt. The platforming experience, cited by both qualitative feedback and the relatively lower Challenge sub-item score (Q17 : 3.60/5.0), would benefit from finer tuning.

In conclusion, this project demonstrates that serious games remain a highly valuable tool for advanced software engineering education. The integration of learning analytics represents a highly viable, robust, and engaging methodology for enhancing the educational experience. We hope that the design patterns and empirical findings documented in this dissertation will serve as a valuable reference for future research in digital serious game design.

References

Abdelrahman, G., Wang, Q., & Nunes, B. (2023). Knowledge tracing : A survey. *ACM Computing Surveys*, *55*(11), 1–37. doi: 10.1145/3569576

Abt, C. C. (1987). *Serious games*. Bloomsbury Publishing PLC.

Abuhamdeh, S. (2020). Investigating the "flow" experience : Key conceptual and operational issues. *Frontiers in Psychology*, *11*. doi: 10.3389/fpsyg.2020.00158

ACM/IEEE-CS Joint Task Force. (2013, December). *Computer science curricula 2013* (Rapport technique). ACM Press and IEEE Computer Society Press.

Ahmed, A., & Sutton, M. J. (2017). Gamification, serious games, simulations, and immersive learning environments in knowledge management initiatives. *World Journal of Science, Technology and Sustainable Development*, *14*(2-3), 78–83.

Alexandrovsky, D., Friehs, M., Grittner, J., et al. (2021). Serious snacking : A survival analysis of how snacking mechanics affect attrition in a mobile serious game. In *Proceedings of the 2021 chi conference on human factors in computing systems* (pp. 1–18). ACM. doi: 10.1145/3411764.3445689

Allart, T., Levieux, G., Pierfitte, M., Guilloux, A., & Natkin, S. (2017). Difficulty influence on motivation over time in video games using survival analysis. In *Proceedings of the 12th international conference on the foundations of digital games* (pp. 1–6). ACM. doi: 10.1145/3102071.3102085

Arrouche, A., & Tirouche, M. M. (2024). *Mise en place d'un tableau de bord administratif pour un jeu sérieux.* (Practical Internship Report, École Nationale Supérieure d'Informatique (ESI))

Bandura, A. (1977). Self-efficacy : Toward a unifying theory of behavioral change. *Psychological Review*, *84*(2), 191–215. doi: 10.1037/0033-295X.84.2.191

Beck, L. (1992). Csikszentmihalyi, mihaly. (1990). flow : The psychology of optimal experience. *Journal of Leisure Research*, *24*(1), 93–94. doi: 10.1080/00222216.1992.11969876

Brady, A.-M. (2005). Assessment of learning with multiple-choice questions. *Nurse Education in Practice*, *5*(4), 238-242. Consulté sur `https://www.sciencedirect.com/science/article/pii/S1471595305000065` doi: https://doi.org/10.1016/j.nepr.2004.12.005

Corbett, A. T., & Anderson, J. R. (1997). Intelligent tutoring systems. *User modeling and user-adapted interaction*, *7*, 251–274.

Dörner, R., Göbel, S., Effelsberg, W., & Wiemeyer, J. (2016). *Serious games* (Vol. 247). Springer.

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1995). *Design patterns : elements of reusable object-oriented software*. Pearson Education.

Gkouma, S., Karpouzis, K., & Bibi, S. (2026). Experimental evaluation of a serious game for teaching uml diagrams. *IEEE Transactions on Education*, *69*(1), 52-61. doi: 10.1109/TE.2025.3646373

Gundersen, S. W., & Lampropoulos, G. (2025). Using serious games and digital games to improve students' computational thinking and programming skills in k-12 education : a systematic literature review. *Technologies*, *13*(3), 113.

Hake, R. (1998, 01). Interactive-engagement versus traditional methods : A six-thousand-student survey of mechanics test data for introductory physics courses. *American Journal of Physics - AMER J PHYS*, *66*. doi: 10.1119/1.18809

Hamdane, Z. (2025). *Chatbot pour assister les apprenants dans le jeu sérieux refactal.* (Master's dissertation, École Nationale Supérieure d'Informatique (ESI))

Hunicke, R., LeBlanc, M., & Zubek, R. (2004). Mda : A formal approach to game design and game research. In *Proceedings of the aaai workshop on challenges in game ai* (Vol. 4, p. 1722).

Ishaq, K., Alvi, A., ul Haq, M. I., Rosdi, F., Choudhry, A. N., Anjum, A., & Khan, F. A. (2024). Level up your coding : A systematic review of personalized, cognitive, and gamified learning in programming education. *PeerJ Computer Science*, *10*, e2310.

Kontinou, R., Georgiou, D., Giamarellos, P., & Bitha, M. (2025). The zeigarnik effect. *Open Schools Journal for Open Science*, *8*. doi: 10.12681/osj.43787

Makhloufi, A., & Kasmi, A. (2024). *Optimisation du temps de chargement initial du jeu sérieux refactal.* (Practical Internship Report, École Nationale Supérieure d'Informatique (ESI))

Martin, R. C. (2013). Getting a solid start. *Robert C. Martin-objectmentor. com*.

Martin, R. C., & Sommerville, I. (2004). *Value pack : Software engineering with agile software development, principles, patterns and practices*. Pearson Education, Limited.

McCabe, T. (1976). A complexity measure. *IEEE Transactions on Software Engineering*, *SE-2*(4), 308-320. doi: 10.1109/TSE.1976.233837

Meng, L., Zhang, M., Zhang, W., & Chu, Y. (2019). Cs-bkt : introducing item relationship to the bayesian knowledge tracing model. *Interactive Learning Environments*, *29*(8), 1393–1403. doi: 10.1080/10494820.2019.1629600

Michael, D. R., & Chen, S. L. (2005). *Serious games : Games that educate, train, and inform*. Muska & Lipman/Premier-Trade.

Miljanovic, M. A., & Bradbury, J. S. (2018). A review of serious games for programming. In *Joint international conference on serious games* (pp. 204–216).

Morsli, R. (2022). *Jeu sérieux pour apprendre les principes avancés de l'orienté objet (refactal v1.0).* (Master's dissertation, École Nationale Supérieure d'Informatique (ESI))

Mouheb, K. (2023). *Jeu sérieux pour apprendre les principes avancés de l'orienté objet (refactal v2.0).* (Master's dissertation, École Nationale Supérieure d'Informatique (ESI))

Oc, Y., & Hassen, H. (2025). Comparing the effectiveness of multiple-answer and single-answer multiple-choice questions in assessing student learning. *Marketing Education Review*, *35*(1), 44–57. Consulté sur `https://doi.org/10.1080/10528008.2024.2417106` doi: 10.1080/10528008.2024.2417106

Petri, G., von Wangenheim, C. G., & Borgatto, A. F. (2016). Meega+ : an evolution of

a model for the evaluation of educational games. *INCoD/gqs*, *3*, 1–40.

Pistono, A. M. A. d. A., dos Santos, A. M. P., Baptista, R. J. V., & Mamede, H. S. (2024). Framework for adaptive serious games. *Computer Applications in Engineering Education*, *32*(4), e22731.

Stråat, B., & Verhagen, H. (2018). Exploring video game design and player retention- a longitudinal case study. In *Proceedings of the 22nd international academic mindtrek conference* (pp. 39–48). ACM. doi: 10.1145/3275116.3275140

Teng, C., Lo, S., & Li, Y. (2012). How can achievement induce loyalty ? a combination of the goal-setting theory and flow theory perspectives. *Service Science*, *4*(3), 183–194. doi: 10.1287/serv.1120.0016

Vahldick, A., Mendes, A. J., & Marcelino, M. J. (2014). A review of games designed to improve introductory computer programming competencies. In *Proceedings of the ieee frontiers in education conference* (pp. 1–8).

Vahldick, A., & Nolli, J. V. (2026). Introducing the visitor design pattern through a digital serious rpg game. *International Journal of Serious Games*, 137–157.

Vie, J., & Kashima, H. (2019). Knowledge tracing machines : Factorization machines for knowledge tracing. In *Proceedings of the aaai conference on artificial intelligence* (Vol. 33, pp. 750–757). doi: 10.1609/aaai.v33i01.3301750

Vihavainen, A., Paksula, M., & Luukkainen, M. (2011). Extreme apprenticeship method in teaching programming for beginners. *ACM SIGCSE Bulletin*, *43*(2), 93–98.

Zyda, M. (2005). From visual simulation to virtual reality to games. *Computer*, *38*(9), 25–32.

Webography

Flekay. (2026). *The farmer was replaced solutions repository.* GitHub : https://github.com/Flekay/The-Farmer-Was-Replaced. (Retrieved June 2, 2026)

Koster, T. (2024a). *The farmer was replaced popular scripts.* https://thefarmerwasreplaced.com/#scripts/. (Retrieved June 2, 2026)

Koster, T. (2024b). *The farmer was replaced : Ultimate guide, wiki & scripts.* https://thefarmerwasreplaced.com/. (Retrieved June 2, 2026)

Robocode. (2001). *Robocode.* https://robocode.sourceforge.io/. (Retrieved December 26, 2025)

Robocode Tank Royale. (2022). *Robocode tank royale.* https://robocode.dev/articles/tank-royale.html. (Retrieved December 26, 2025)

Screeps, LLC. (2016). *Screeps - online strategy game for programmers.* https://screeps.com/. (Retrieved December 26, 2025)

SteamDB. (2024). *The farmer was replaced steam charts.* https://steamdb.info/app/2060160/charts/. (Retrieved June 2, 2026)

track2training. (2025). *Alderfer's erg theory.* https://track2training.com/2025/11/18/alderfers-erg-theory-7/.

Ubisoft. (2013). *Rayman legends.* https://www.ubisoft.com/en-au/games/rayman-legends.

Ubisoft. (2016). *Tom clancy's the divison.* https://www.ubisoft.com/en-gb/game/the-division.

Wikipedia. (s. d.). *Metroidvania.* https://en.wikipedia.org/wiki/Metroidvania. (Retrieved June 1, 2026)

Zachtronics. (2015a). *Tis-100.* https://www.zachtronics.com/tis-100/. (Retrieved December 26, 2025)

Zachtronics. (2015b). *Zachademics.* https://www.zachtronics.com/zachademics/. (Retrieved December 26, 2025)

Zachtronics. (2016). *Shenzhen i/o.* https://www.zachtronics.com/shenzhen-io/. (Retrieved December 26, 2025)

Annexe A

Past Players' Feedback

Table I – Responder feedback on general experience elements

Responder	Do you recall finishing the game ?	Personal Game Experience Rating	Personal Learning Experience Rating	Story Engagement
1	Yes	3/5	3/5	3/5
2	Yes	4/5	4/5	3/5
3	Yes	4/5	3/5	5/5
4	No	3/5	4/5	4/5
5	Yes	4/5	3/5	3/5
6	No	5/5	5/5	5/5
7	No	3/5	3/5	2/5
8	Yes	4/5	5/5	2/5
9	No	4/5	4/5	2/5
10	No	3/5	3/5	4/5
11	No	4/5	3/5	3/5

Table II – Responder feedback on finishing the game (Only for players who haven't finished the game)

Responder	Why did you not feel like finishing the game ?	What would have made the game more appealing to you ?
4	Because the game was too slow, it took me an eternity in it so I couldn't patient more to finish it.	I liked the fact that the learning process was linked to gaming, and it's good specially that the new generation of learners, most of them, come from gamer backgrounds.
6	Actually I was in 1CP (First year) I had no idea about Python or OOP, that's why the last mission was so hard (yes I stopped in the last mission).	I don't know, it was great to discover new things, and I really liked the concept, we never see such hard work on educational work.

Continued on next page

Table II – continued from previous page

Responder	Why did you not feel like finishing the game ?	What would have made the game more appealing to you ?
7	It was kind of boring, it wasn't enough explicative, I don't remember that there was something that made me want to continue the game.	Having some hints (maybe we have to search for them in the place, I like investigation or searching games), having a sub-game that makes it easier to progress, maybe a voice that explains or a video at the beginning. I found the design too dark.
9	The repetitive aspect of the game leads to a certain level of boredom and encourages players to quit.	More creativity in the game's story and introducing something new at each level.
10	Visuals a bit old and not high quality.	Better visuals.
11	Took too long to complete, I already knew Java so I didn't finish it. (Wasn't motivated enough.)	Make it more dynamic. Not just speed running the game gate after gate, in a sequential manner.

Table III – Responder feedback on general appreciations, annoyances and potential improvements

Responder	What did you appreciate most about Refactal ?	What did you find annoying about Refactal ?	What changes would you personally like to see ?
1	The concept of gamification, turning design pattern learning into a full game. Also appreciated the button used to highlight useful items, which helps guide the player through the map.	The monotonous tone of the narrator, the repetitiveness of the game style (go to room, write code, next room, write code...), and occasional bugs.	A hint button should be added if not available, since getting stuck with no clue is the worst thing in a game. The hint button would reduce the total score to discourage overuse. The game could become something great, played beyond the scope of research labs.
2	The concept is really good and makes you try to finish the game while learning design patterns.	The pace of the game was sometimes too slow, and instructions weren't always clear enough.	Having shorter missions, especially when advancing to more complex design patterns.

Continued on next page

Table III – continued from previous page

Responder	What did you appreciate most about Refactal?	What did you find annoying about Refactal?	What changes would you personally like to see?
3	The storytelling, and the explanation of each pattern at the beginning of each level.	Some levels have buttons placed on the wall with no clear instructions on how to interact with them.	Improvements to hints and Java code error explanations. Questions should also be more challenging, as they currently feel too easy to feel like a real learning experience.
4	—	—	Make the game run faster to avoid boring the player, and update the design to something more modern.
5	The gamification of learning.	Version problems and data loss after long play sessions.	A more engaging storyline, and user accounts with saved progress.
6	The storytelling, the hints, and the exercises (a great mix of code and story).	The application was very heavy (1GB) and the graphics were not impressive.	Consider changing the game engine to Unity or Unreal Engine, or optimizing graphics. Also improve aim assistance for button interactions, and increase character movement speed.
7	The concept, the sound effects, and the in-game elements.	Not much recalled.	A mini-game within the game to make it more fun and encourage continued play.
8	The learning aspect and the guiding tips.	Occasional bugs especially in the last levels, and UI could be improved.	If a player is stuck for a while, provide multiple suggestions rather than just one, since a single hint may not be enough.

Continued on next page

Table III – continued from previous page

Responder	What did you appreciate most about Refactal ?	What did you find annoying about Refactal ?	What changes would you personally like to see ?
9	The good structure of the levels ; each level respected its declared concept and helped understand it better.	Repetition in level presentation : same décor and similar contextual problems (opening a door...). Weaknesses are more on the creative and aesthetic side than the educational side.	More creativity, and a version with fewer bugs (screen occasionally froze during gameplay).
10	The coding part and the intuitive ease of the game.	The visuals.	Better visuals.
11	Good graphics that still run on weak PC configurations. The overall work done.	Events following a strictly sequential order (gate after gate after gate...).	Adding more varied events.

Annexe B

Learning Difficulties

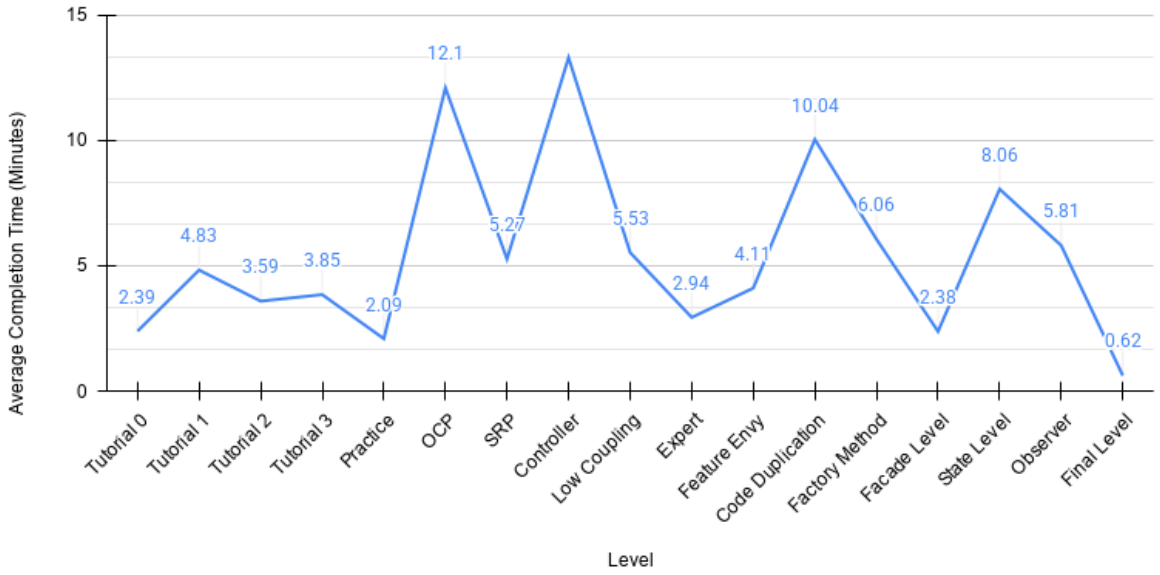


FIGURE B.1 – Average Completion Times per Level (Arrouche & Tirouche, 2024).

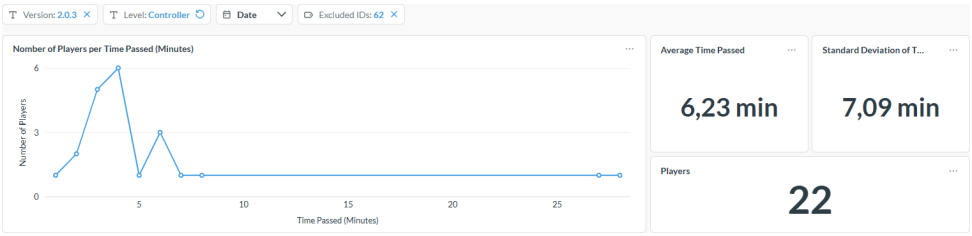


FIGURE B.2 – Screen capture of the administrative dashboard (Arrouche & Tirouche, 2024), showing average completion times per level.

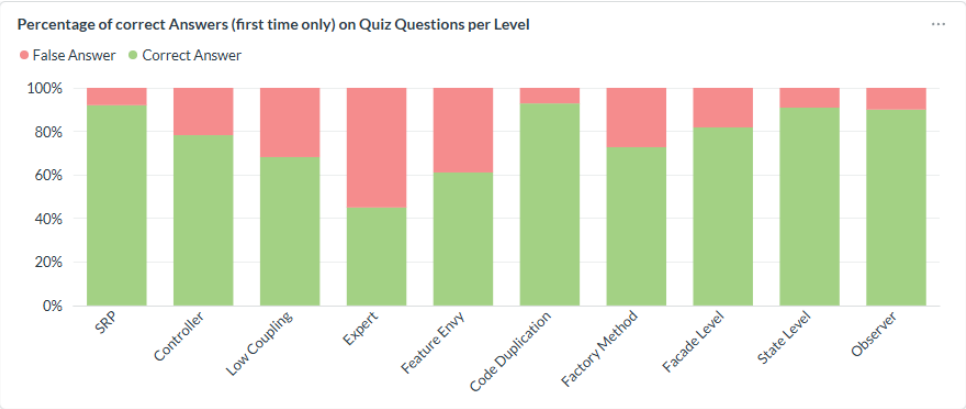


FIGURE B.3 – Screen capture of the administrative dashboard (Arrouche & Tirouche, 2024), showing quiz answer distributions.

Annexe C

External Motivation

Recherche de bêta-testeurs (Récompensés)

Khaoula MOUHEB <ik_mouheb@esi.dz>
26 May 2024, 21 :12
to 1, 2CS

Bonjour,
Nous cherchons des volontaires pour tester un jeu dont 3 participants seront récompensés. Le test se déroule en trois phases :

- Répondre au pré-test sur ce lien.
- Tester le jeu sur ce lien. Le chargement initial prend environ 3 minutes.

Après avoir testé tout le jeu, et pour être considéré dans le tirage au sort de récompense, merci de :

- Remplir le post-test : `https://forms.gle/hHFcWBiggv4jLbu48`. Les testeurs de la version Desktop doivent fournir une capture d'écran du score du dernier niveau (Observer).
- Répondre à l'évaluation : `https://forms.gle/PePK2YVXdE1BECSR9`.

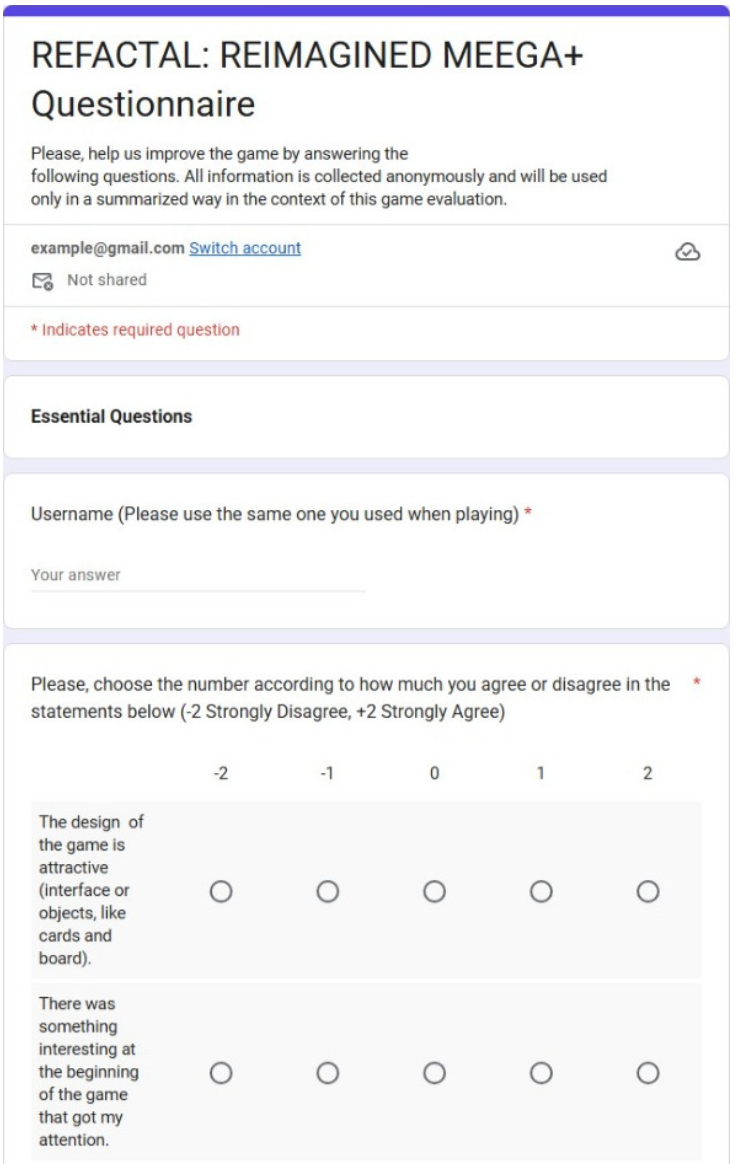
Ceux qui n'ont pas testé tous les niveaux du jeu, ainsi que ceux qui n'ont pas rempli les formulaires, ne seront pas pris en compte lors du tirage au sort.
Soyez nombreux et merci pour votre collaboration.

Cordialement

FIGURE C.1 – Recruitment email sent to students for beta-testing Refactal.

Annexe D

MEEGA+ Questionnaire



REFACTAL: REIMAGINED MEEGA+ Questionnaire

Please, help us improve the game by answering the following questions. All information is collected anonymously and will be used only in a summarized way in the context of this game evaluation.

example@gmail.com Switch account

Not shared

* Indicates required question

Essential Questions

Username (Please use the same one you used when playing) *

Your answer

Please, choose the number according to how much you agree or disagree in the statements below (-2 Strongly Disagree, +2 Strongly Agree) *

	-2	-1	0	1	2
The design of the game is attractive (interface or objects, like cards and board).	○	○	○	○	○
There was something interesting at the beginning of the game that got my attention.	○	○	○	○	○

FIGURE D.1 – Screen capture of the start of the MEEGA+ questionnaire.

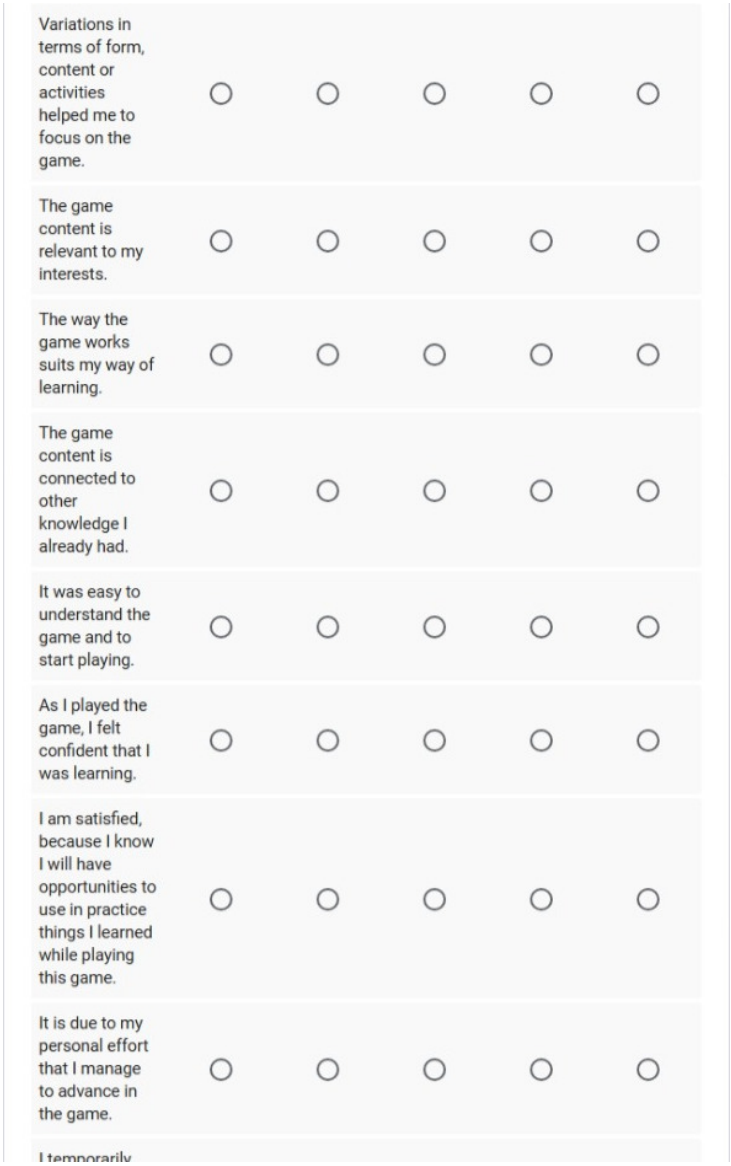


FIGURE D.2 – Screen capture of part of the MEEGA+ questionnaire.